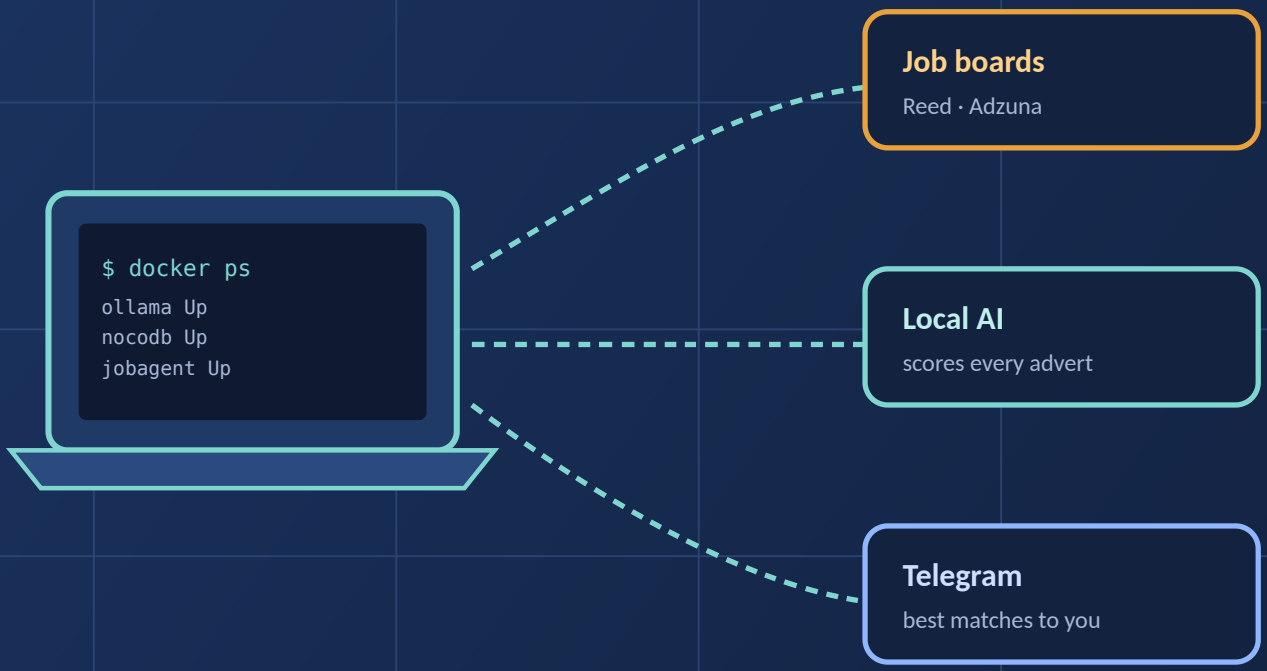


BEGINNER'S DIY HANDBOOK · EDITION 1.0

Build Your Own AI Job-Search Assistant

Turn an old laptop into a private server that searches job boards twice a day, scores every advert against your CV with a free local AI, and sends the best matches to your phone. No graphics card, no paid AI, no experience needed.

Proxmox · Docker · Ollama · NocoDB · Telegram

November 2026

About this handbook

This handbook takes you from an ordinary old computer to a working, self-hosted job-search assistant. It is written for complete beginners: every new word is explained the first time it appears, every command is shown in full, and every stage ends with a check that tells you whether it worked.

What you will have at the end

A small server, running 24 hours a day, which twice a day:

1. searches the Reed and Adzuna job boards for the job titles you choose;
2. throws away adverts that clearly do not fit (too senior, wrong field, security clearance needed, and so on);
3. asks an AI model, running privately on your own computer, to score each remaining advert against your real profile from 0 to 100;
4. saves every good match (70 or more, by default) into a simple online-spreadsheet tracker;
5. sends you a Telegram message with the best matches and links to the adverts.

> **Be clear about what it does not do**
>
> The working system described here is **Part 1: the Job Finder**. It does **not** rewrite your CV, does **not** ask you to approve applications and does **not** apply for jobs. Those are planned future parts and are described honestly as plans in Chapter 19. You read the matches, decide, and apply yourself. This is deliberate: nothing is ever sent to an employer without you.

How to read the instructions

You will type commands in three different places. Mixing them up is the most common beginner mistake (it happened while building the original system), so every command box is labelled with where to type it:

PROXMOX HOST
```
root@pve:~# this box is typed on the Proxmox server itself (the "host")
```

CONTAINER
```
root@agent-box:~# this box is typed inside the container that runs the apps
```

YOUR PC
```
this box is typed on your normal Windows, Mac or Linux computer
```

FILE CONTENTS
```
this box shows what a file should contain
```

EXPECTED OUTPUT
```
this box shows what you should SEE after a command
```

Do not type the prompt (`root@pve:~#`); type only what comes after it. Anything written in capitals and starting with `YOUR_` , such as `YOUR_SERVER_IP` , is a placeholder: replace it with your own value, without the word YOUR.

Placeholder	Replace with	Where you get it
`YOUR_SERVER_IP`	The address of your Proxmox server, e.g. 192.168.1.50	Stage C, screen 5
`YOUR_CONTAINER_IP`	The address of the apps container	Stage D
`YOUR_TAILSCALE_IP`	A 100.x.y.z address for remote access	Stage E
`YOUR_TELEGRAM_BOT_TOKEN`	Your bot's secret token	Stage G
`YOUR_TELEGRAM_CHAT_ID`	Your personal chat number	Stage G
`YOUR_REED_API_KEY` , `YOUR_ADZUNA_APP_ID` , `YOUR_ADZUNA_APP_KEY`	Job-board keys	Stage H
`YOUR_NOCODB_API_TOKEN` , `YOUR_NOCODB_TABLE_ID`	Tracker keys	Stage F2

Coloured boxes mean: **tip** (helpful extra), **warning** (read before continuing), **danger** (can delete data or leak secrets), **success check** (how to know it worked) and **real experience** (something that actually went wrong while building the original system).

What has been tested

This handbook is based on a real system that was built and run on a 16 GB laptop. Labels tell you how sure each part is:

BUILT & USED ran on the original system OPTIONAL not needed for the core system NOT VERIFIED standard procedure, not tested on the original hardware PLANNED does not exist yet

The program code in the download package was rebuilt from the original design for public release and tested against simulated job-board, AI, tracker and Telegram services (37 automated checks). The full details are in the separate *Testing Report*.

At a glance: what you need

Hardware

- One 64-bit computer you can dedicate to this: old laptop, desktop or mini PC. **8 GB RAM or more recommended** (4 GB possible with limits).
- Ideally an SSD with at least 64 GB (128 GB+ is comfortable).
- An Ethernet cable to your router (Wi-Fi is not supported by Proxmox for this setup).
- A USB stick of 4 GB or more (it will be erased).
- A second computer or phone with a web browser to manage the server.

Accounts (all free)

- Telegram account (on your phone).
- Reed developer API key (email sign-up).
- Adzuna developer account (optional second source).
- Tailscale account (optional, for access away from home).

Time

Part	Time
Read chapters 1-4	45 min
Install Proxmox (Stages B-C)	1 hour
Container + Docker (D-E)	45 min
Project + tracker + Telegram (F-G)	1 hour
AI model download (H)	10-30 min
Profile writing + first run (J)	1-2 hours

Money

Software: £0. Electricity for an idle laptop running all day: very roughly £2-£5 per month (Chapter 3). Optional: a small SSD (about £25-£40) if your computer has an old hard disk.

The route through this book

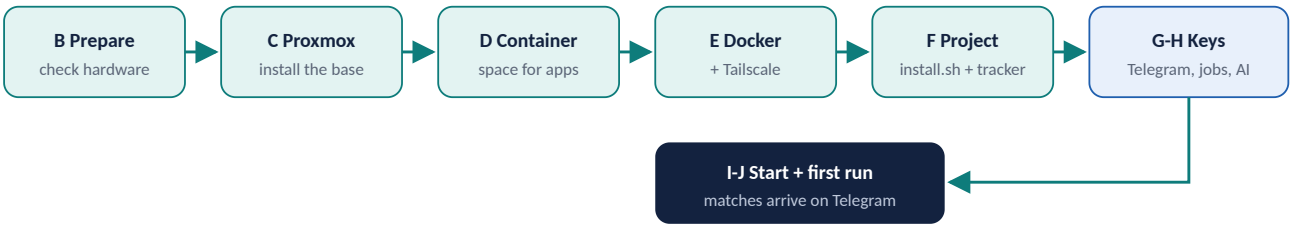


Figure 0.1. The installation stages. Each one depends on the one before it, so please do them in order. Stage A (understanding) is Chapters 1-3.

Contents

About this handbook 2

What you will have at the end 2

How to read the instructions 2

What has been tested 3

At a glance: what you need 4

The route through this book 4

Contents 5

PART I · UNDERSTAND

Chapter 1 · What you are building 10

Features: what exists and what is planned 10

Why build it yourself? 11

Honest limitations 11

Chapter 2 · How it all fits together 12

The layers, one by one 12

What happens during one run 14

How the network fits in 15

Chapter 3 · Choosing your hardware 16

Minimum and recommended 16

What different machines can do 16

Where the AI runs: three choices 17

Choosing a model 18

Settings for low-memory machines 18

Storage: learn from the original system 18

Power and running costs 19

PART II · BUILD

Chapter 4 · Terminal and SSH basics 21

What a terminal is 21

Opening a terminal 21

Copying and pasting 21

Editing a file with nano 22

Ten commands you will meet 22

SSH: a terminal from your own PC 23

Chapter 5 · Stage B: prepare the computer 24

Check that virtualisation is supported 24

Switch on virtualisation in the BIOS/UEFI 24

Storage 25

Network 25

Laptops: power, lid and battery 25

Chapter 6 · Stage C: install Proxmox 27

Download the installer 27
Make a bootable USB stick 27
Start the server from the USB stick 28
The installer, screen by screen 28
First login to the web interface 29
Switch to the free update channel and update 30
Laptop settings: lid, USB and console 31
Alternative routes 32

Chapter 7 · Stage D: create the apps container 34
Step 1: put the project package on the server 34
Step 2: download the Debian template 34
Step 3: choose your network option 35
Step 4: create the container 36
Step 5: extra container settings 37
Step 6: start the container and update it 38

Chapter 8 · Stage E: install Docker and Tailscale 40
Docker 40
Tailscale (optional, recommended) 41

Chapter 9 · Stage F: install the project 43
Copy the package into the container 43
What is in the package, and where it goes 43
Run the installer 44
Manual installation (what install.sh does, by hand) 45
The secrets file: .env 46
Your search settings: config.json 46
Your profile: master.md 47
Fixed answers: answers.yaml 47

Chapter 10 · Stage F2: set up the NocoDB tracker 49
Create the admin account 49
Create the base and the table 49
Create an API token 50
Find the table ID 50

Chapter 11 · Stage G: create your Telegram bot 51
What a Telegram bot is 51
Create the bot with @BotFather 51
Start the Job Finder and find your chat ID 52
Send a test message 53
What the messages look like 53
About approve and reject 53

Chapter 12 · Stage H: job-board keys and the AI 55
Get a Reed API key 55
Get Adzuna keys (optional) 55
How the AI part works 55
Download, list and test models 56

Test everything at once 56
Optional: Open WebUI, and making it fast 57

Chapter 13 · Stage I: start, stop and automate 59
How automatic start-up works 59
Changing the schedule 59
Running a search by hand 59
Reading the logs 60
Stopping and restarting 61
Optional: managing stacks with Dockge 61
Recovery after a reboot or power cut 61

Chapter 14 · Stage J: your first real run 63
Step 1: write your master profile 63
Step 2: set your job preferences 63
Step 3: start the first search 64
Step 4: review the matches 64
Step 5: apply, and record it 64
Step 6: tune over the first week 65

PART III · KEEP IT RUNNING

Chapter 15 · Troubleshooting 67
Proxmox and hardware 67
Network 69
Container and Docker 70
The Job Finder 72
The AI 73
Telegram 74
Resources and other messages 75

Chapter 16 · Maintenance and backups 76
A monthly routine 76
Updating the project code 76
Changing preferences or your CV safely 77
Backups 77
Restoring 78
Recovering from a dead disk or failed server 78

Chapter 17 · Security and privacy 79
The security design in one paragraph 79
Where secrets live 79
Ten rules 80
Your personal data 80

Chapter 18 · Responsible use 81
Principles 81

Chapter 19 · Roadmap: what comes next 82
Infrastructure improvements 82
Part 2: CV and cover-letter tailor 82

Part 3: Telegram approve / skip buttons 82
Part 4: apply robot with guardrails 82
Other ideas 82

Chapter 20 · The download package 84
Contents 84
How to use it 84
Differences from the original private build 84

APPENDICES · REFERENCE

Appendix A · Complete source files 87

Appendix B · Command cheat sheet 108
Proxmox host 108
Container: Docker and the Job Finder 108

Appendix C · Services, ports and file locations 110
Services 110
Important files 110

Appendix D · Glossary 112

Appendix E · Final checklist 114

Appendix F · Useful links and index 116
Official links 116
Index 116

PART I

Understand

Before touching any computer, spend half an hour understanding what you are building and choosing the right hardware. This is Stage A of the installation.

- Chapter 1: what the system does and does not do
- Chapter 2: how the parts fit together
- Chapter 3: what computer you need

CHAPTER 1

What you are building

Job hunting involves a lot of repetitive searching: the same job titles, on the same websites, every day, scrolling past adverts that are too senior or need a clearance you do not have. This project hands that repetitive part to a small computer, so that you only spend time on adverts that actually fit you.

Features: what exists and what is planned

The table below is the single source of truth about the project's features. Everything marked BUILT is in the download package and is covered step by step in this handbook.

Feature	Status	Notes
Search job boards automatically, twice a day	BUILT	Reed (main source) and Adzuna (optional). Times are set in `config.json`.
Collect job titles, companies, locations, links and full descriptions	BUILT	For Reed, the full advert text is downloaded before scoring.
Remove duplicates and adverts already seen	BUILT	Remembered in `state/seen.json`.
Rule-based filters	BUILT	Required title words, banned seniority words, "skip" phrases such as SC/DV clearance.
AI scoring of each advert against your profile (0-100)	BUILT	Local model through Ollama; gives reasons, gaps and a level-fit opinion.
Rank and keep only the best matches	BUILT	Default threshold 70.
Save matches to a tracker	BUILT	NocoDB table "Applications", Status "Found".
Telegram summary of the best matches	BUILT	Top 15 with scores, reasons and links.
Private chat with the AI in a web page	OPTIONAL	Open WebUI. Not needed for the Job Finder.
Tailor your CV and cover letter to each job	PLANNED	Part 2. Gotenberg (PDF maker) is installed as an optional extra, ready for it.
Telegram approve / reject buttons	PLANNED	Part 3.
Apply after approval	PLANNED	Part 4, limited to email, Greenhouse, Lever and Ashby forms. Never LinkedIn or Indeed bots.

So, today, the workflow is: **the server finds and ranks, you decide and apply.** In the tracker you then change the Status of each job yourself (Approved, Applied, Skipped, Interview and so on). Chapter 14 shows exactly how.

Why build it yourself?

- **Privacy.** Your CV and your job search never leave your own computer for AI processing. The AI model runs locally.
- **Cost.** Everything is free software. There is no monthly AI subscription.
- **Learning.** You will end up with practical experience of Linux, virtualisation, containers, APIs and AI models, which is valuable in its own right (and worth a line on a CV in IT roles).
- **Control.** You decide the rules: which titles, which companies, how strict the scoring is.

Honest limitations

- **UK only, out of the box.** Reed is a UK job board and the Adzuna search is set to the UK (`gb`). Other countries would need code changes.
- **Small AI models make mistakes.** A 3-billion-parameter model running on a laptop processor is good at "does this advert fit this profile?", but it is not perfect. Treat scores as a helpful first sort, not a final verdict. Always read the advert.
- **It is slow on old processors.** Expect about 40-70 seconds per advert on a 4-core laptop chip. That is why the system scores at most 40 adverts per run and carries the rest over.
- **Job boards only.** It does not read company career pages, LinkedIn or Indeed.
- **Laptop servers need care.** Laptops are designed to sleep, run on battery and use slow USB disks. Chapter 5 shows the settings that keep one running 24/7.

CHAPTER 2

How it all fits together

The system is a stack of layers, like a set of boxes inside boxes. Understanding the layers makes every later step obvious, so this chapter explains each one in plain English.

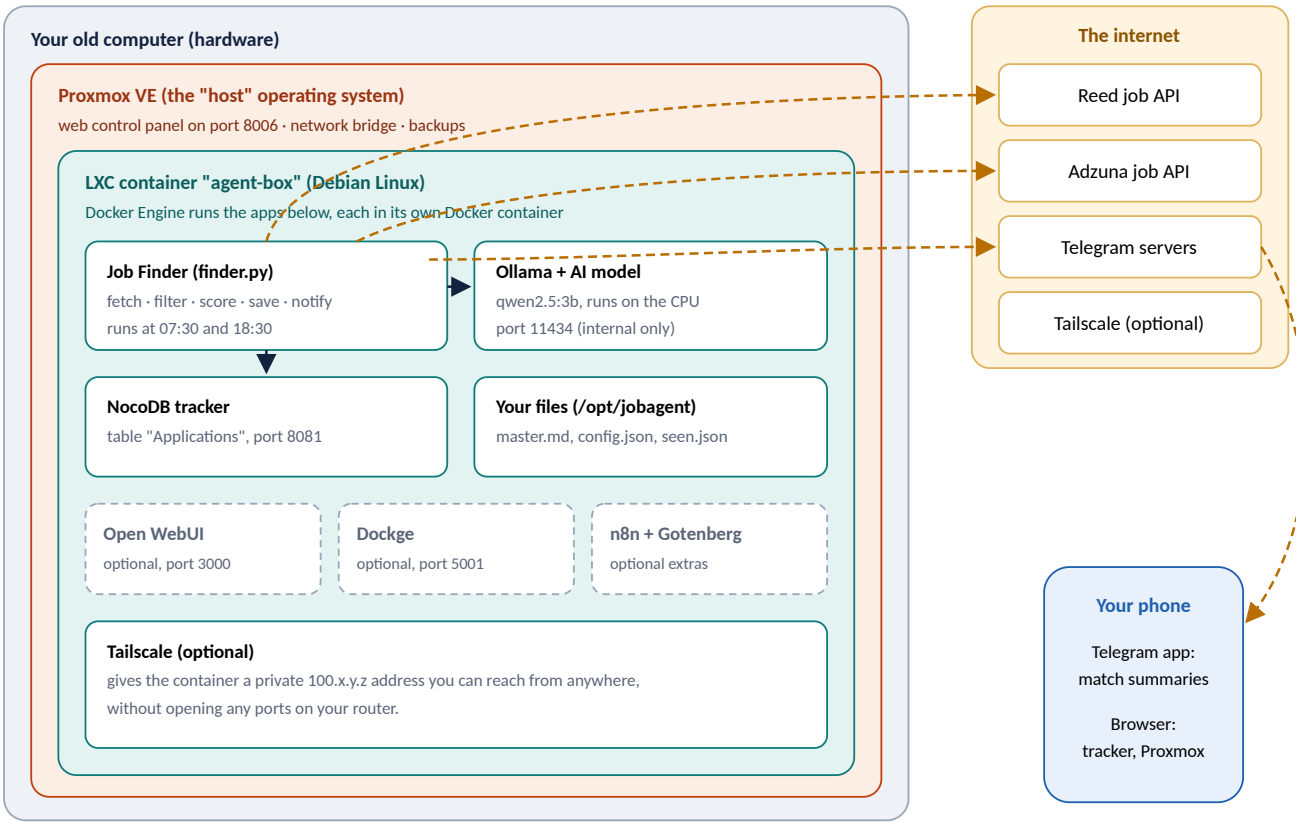


Figure 2.1. Architecture. Solid arrows stay inside your computer; dashed arrows go over the internet. The Job Finder talks to the job boards and Telegram over the internet, and to Ollama and NocoDB over a private network inside the container. No incoming connections from the internet are needed.

The layers, one by one

Hardware: your old computer

The physical machine. It needs a 64-bit processor with **hardware virtualisation** (a feature, called Intel VT-x or AMD-V, that lets one computer pretend to be several). Almost every processor made since about 2010 has it, but it is sometimes switched off in the BIOS. Chapter 5 shows how to check.

Proxmox VE: the host

Proxmox Virtual Environment is a free operating system that replaces Windows on the server. Its only job is to run and manage other, smaller systems inside it, and it is controlled from a web page that you open in a browser on another device. We call the Proxmox machine the **host**.

Why use it? It makes the server easy to rebuild, back up and experiment with. If you break the apps, you can delete and recreate the container in minutes without reinstalling everything. If you prefer, you can skip Proxmox and install Debian Linux directly (Chapter 6 explains this alternative).

Virtual machines and containers

A **virtual machine (VM)** is a complete pretend computer, with its own pretend hard disk and its own operating system. It is very isolated, but it reserves a lot of memory.

An **LXC container** (Linux Container) is a lighter alternative: it is an isolated Linux system that shares the host's core (the kernel). It starts in seconds and uses far less memory. For a low-spec machine this matters, so the original system uses one LXC container, called **agent-box** in this handbook, with ID **100**.

Docker: apps in boxes

Docker runs each application in its own sealed box, also called a container (a Docker container, not to be confused with the LXC container it lives in). Each box contains the app and everything it needs, already set up by the app's authors. Instead of installing Ollama, NocoDB and so on by hand, you write a short recipe file called `compose.yaml` and Docker downloads and starts the app. A group of apps started from one recipe is called a **stack**.

Running Docker inside an LXC container ("nesting") is common and works well, as long as the container has the *nesting* option switched on.

The apps

App	What it is, in plain English	Needed?
Job Finder	The project's own small Python program. It does the searching, filtering, scoring and messaging.	**Yes**
Ollama	A program that runs AI language models on your own computer and answers questions from other programs.	**Yes**
AI model	The "brain" file Ollama loads, e.g. `qwen2.5:3b` (a 1.9 GB file). Bigger models are smarter but slower and need more memory.	**Yes**
NocoDB	A free, self-hosted spreadsheet-style database. Your list of matched jobs lives here.	**Yes**
Open WebUI	A ChatGPT-style web page for chatting with your local AI.	Optional
Dockge	A web page for starting, stopping and editing Docker stacks without typing commands.	Optional
n8n	A visual automation tool ("if this, then that"). Installed in the original system but not used by the Job Finder.	Optional
Gotenberg	Turns web pages into PDF files; kept ready for the planned CV-tailoring part.	Optional
Tailscale	A private network between your own devices, so you can reach the server from anywhere without exposing it to the internet.	Optional

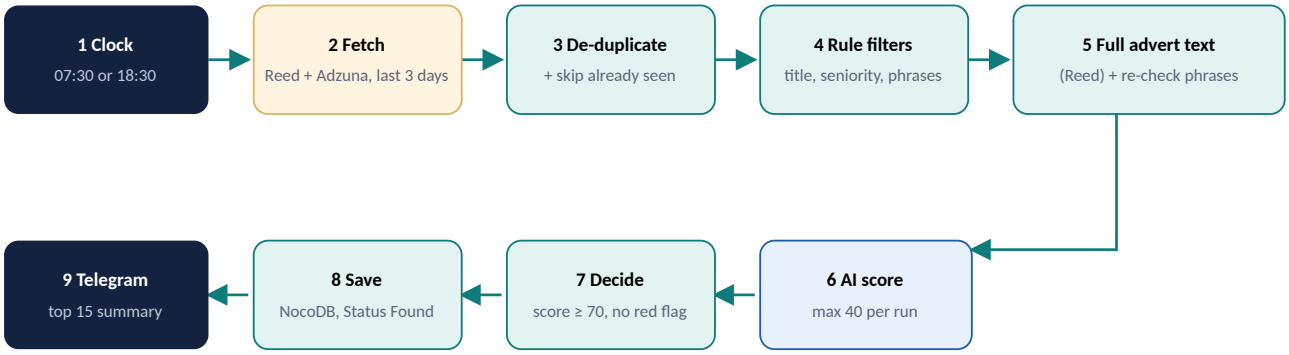
APIs and keys

An **API** (Application Programming Interface) is a door into a website that is meant for programs instead of people. Instead of a web page, the Reed API returns tidy data such as "job title, company, salary". Websites hand out **API keys** (long secret codes) so they know who is asking. Your keys live in one protected file called `.env` and nowhere else.

Telegram bots

A **Telegram bot** is a Telegram account controlled by a program instead of a person. You create one in two minutes by chatting with Telegram's official **@BotFather** account, and it gives you a **token** (the bot's password). The Job Finder uses the token to send messages to one chat only: yours, identified by your **chat ID** number.

What happens during one run



Real first run on the original system: 818 adverts fetched → 40 passed the filters → scored one by one (about 40-70 s each) → matches saved and sent. Adverts rejected by rules or scored by the AI are remembered in seen.json and never processed again. Adverts that were not scored (over the 40 limit, or the AI timed out) are NOT remembered, so they are tried again at the next run.

Figure 2.2. One run of the Job Finder, step by step. You can also start a run by hand at any time (Chapter 13).

How the AI scores an advert

For each advert, the Job Finder sends Ollama two messages. The first ("system" message) contains recruiter instructions followed by your complete `master.md` profile. The second contains the advert's title, company, location and the first 2,500 characters of the description. The AI must answer in a fixed JSON format (a simple structured text format), for example:

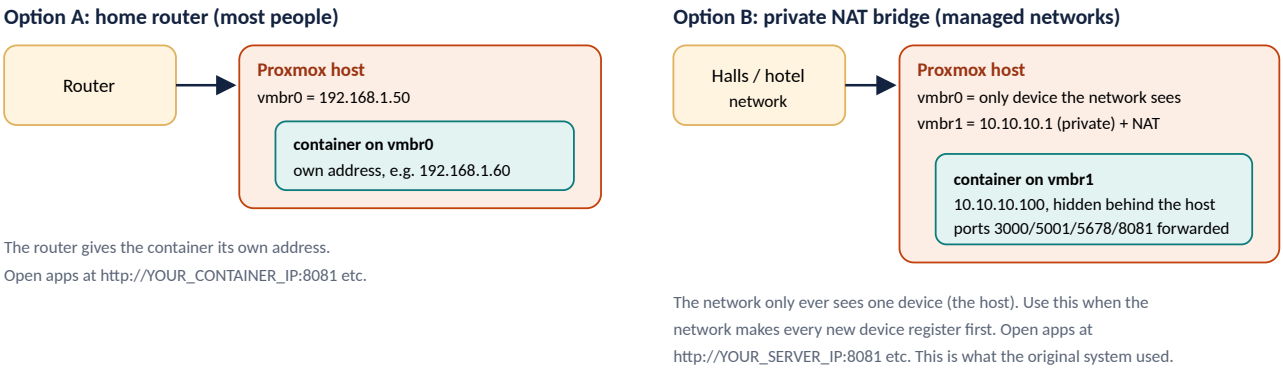
EXAMPLE AI ANSWER

```
{"score": 78,
 "level_fit": "good",
 "reasons": ["Has SIEM and Wireshark experience", "Graduate level matches"],
 "gaps": ["No Splunk certification"],
 "sponsorship_red_flag": false}
```

Because your profile always comes first and never changes between adverts, Ollama can reuse the work it did reading it (this is called *prompt caching*), so the first advert of a run is slower and the rest are faster.

How the network fits in

Your server needs to reach the internet (job boards, Telegram, software downloads) and you need to reach the server's web pages from your phone or laptop. Two layouts are supported:



> **Figure 2.3. The two network options.** Choose A unless you are on a student-halls, hotel or workplace network that blocks unknown devices. You choose in Stage D.

Some technical words you will meet:

- **IP address**: a device's number on the network, like a house number (e.g. 192.168.1.50).
- **Port**: a numbered door on that address. One device can offer several services on different ports: NocoDB on 8081, Open WebUI on 3000.
- **Gateway**: the router, i.e. the way out to the internet.
- **Bridge** (`vmbr0`, `vmbr1`): a virtual network switch inside Proxmox that containers plug into.
- **NAT** (Network Address Translation): the trick your home router uses so many devices share one address. In Option B the Proxmox host does the same for its containers.
- **DHCP**: the automatic way a router hands out addresses to devices.

CHAPTER 3

Choosing your hardware

You do not need a powerful computer or a graphics card. The original system runs on a five-year-old business laptop. This chapter helps you judge what your machine can do, and how to make a smaller machine work.

Minimum and recommended

	Minimum (works, with limits)	Recommended	Original system
Processor	64-bit, 2 cores, VT-x/AMD-V	4 cores / 8 threads, 2017 or newer	10th-generation Intel Core i5 laptop chip (4C/8T)
Memory (RAM)	4 GB	8-16 GB	16 GB
Storage	64 GB, any disk	128-500 GB SSD, internal	USB hard disk (a mistake, see below)
Network	Wired Ethernet	Wired Ethernet	Wired Ethernet
Graphics card	Not needed	Not needed	Built-in Intel graphics, unused

Proxmox itself officially needs a 64-bit processor with virtualisation support and a minimum of about 2 GB RAM for evaluation; everything else depends on what you run on it.

What different machines can do

Machine	Suggested setup	AI model	Expect	Verified?
4 GB RAM old laptop or desktop	Skip Proxmox: install Debian directly (Chapter 6, route 2). `install.sh --minimal`. No Open WebUI, no extras.	`qwen2.5:1.5b` (~1 GB)	Works but tight. Weaker scoring. ~25-40 s per advert. Keep the per-run limit at 20-40.	NOT VERIFIED
8 GB RAM laptop, desktop or mini PC (e.g. Intel N100)	Proxmox + container with 5-6 GB RAM. `--minimal` recommended.	`qwen2.5:3b` (~1.9 GB)	Comfortable for the Job Finder. Open WebUI usable if added.	NOT VERIFIED
16 GB RAM laptop or desktop	Proxmox + container with 12 GB RAM, 6 cores. Full install.	`qwen2.5:3b` for scoring; `qwen2.5:7b` (4.7 GB) possible for chat	~40-70 s per advert on a 4-core laptop chip. Room for all optional apps.	BUILT & USED
Existing Proxmox server	Just add one container (Stage D onwards).	Depends on free RAM	Same as above.	NOT VERIFIED

Machine	Suggested setup	AI model	Expect	Verified?
Cloud VPS (rented server)	No Proxmox: Ubuntu/Debian VPS, install.sh directly. Needs 4+ GB RAM, 2+ vCPU.	`qwen2.5:1.5b` or `3b`	CPU-only VPS are often slow at AI. Your CV is stored on someone else's server.	NOT VERIFIED

> **No promises for every machine**
> Only the 16 GB laptop setup was actually built and used. The other rows are reasoned estimates from model sizes and the measured speed of the original system. Very old processors (before about 2012), 32-bit machines and ARM boards such as the Raspberry Pi are not covered by this handbook.

Where the AI runs: three choices

1. Locally, on the server (default)

Ollama runs on the server's processor. Free, private, works offline, but slow on old machines. This is what the handbook sets up and what was tested.

2. On another, stronger computer in your home

If you have a gaming PC or a newer laptop, you can install Ollama on it and point the Job Finder at it by changing one line in `.env`:

/OPT/STACKS/JOBAGENT/.ENV

```
OLLAMA_URL=http://YOUR_OTHER_PC_IP:11434
```

The server then only does the light work (fetching, filtering, saving). The other computer must be switched on at run times and Ollama on it must accept network connections (on that computer, set the environment variable `OLLAMA_HOST=0.0.0.0`). NOT VERIFIED on the original system, but it uses exactly the same Ollama interface.

3. A paid cloud AI service (not included)

Services such as OpenAI, Anthropic or Google sell AI by the amount of text processed. They are much faster and smarter than a laptop model. **The Job Finder in this package does not support them**: it only talks to Ollama. Adding a cloud provider would mean changing the `score_job` function in `finder.py` and storing a new key in `.env`. It is a reasonable improvement, but it is not part of the tested system, and it means your CV text is sent to that company with every advert. If you do this yourself, estimate cost as: *adverts per day × (profile + advert length)*. With a 1,500-word profile and 40 adverts twice a day, that is roughly 200,000-250,000 words of input per day; check the provider's current price list.

Choosing a model

Model	Download	RAM while running*	Quality for scoring	Use on
`qwen2.5:1.5b`	~1.0 GB	~1.5-2 GB	Basic; more mistakes	4 GB machines
`qwen2.5:3b` **(default)**	~1.9 GB	~2.5-3 GB	Good; follows the JSON format reliably	8 GB+
`llama3.2:3b`	~2.0 GB	~2.5-3 GB	Good alternative	8 GB+
`qwen2.5:7b`	~4.7 GB	~6 GB	Better reasoning, about 2-3 × slower	16 GB+

* Approximate, including the 8,192-token working memory the Job Finder asks for. "b" means billions of parameters: a rough measure of the model's size. On the original laptop, 3B models read prompts at about 35-45 tokens per second and wrote answers at about 8-12 tokens per second (a token is roughly three-quarters of a word).

Settings for low-memory machines

- Run `bash install.sh --minimal --model qwen2.5:1.5b`: no Open WebUI, no Dockge, small model.
- Keep `OLLAMA_NUM_PARALLEL=1` (already the default) so only one AI request runs at a time.
- Lower `OLLAMA_KEEP_ALIVE` from `24h` to `10m` in `/opt/stacks/ai/compose.yaml` so the model leaves memory between runs (the next run then starts a few seconds slower).
- Keep your `master.md` short and factual (under about 1,000 words) so it fits in the AI's working memory.
- If memory is still tight, open `/opt/jobagent/app/finder.py` and change `AI_NUM_CTX = 8192` to `4096` and `DESCRIPTION_LIMIT = 2500` to `1500`. Your profile plus the advert must fit in that number of tokens, so do this only with a short profile.
- Do not install n8n or Gotenberg (they only start with `--profile extras` anyway).

Storage: learn from the original system

> ★ **Real experience: the USB hard disk failure**
>
> The original server booted Proxmox from an external USB hard disk. Twice in one day the disk disconnected while running. Every command then failed with `Input/output error`, Docker died, and even `dmesg` failed with "Bus error". The only fix was to hold the power button for 10 seconds, reseat the cable and repair the file system. Likely causes were running on battery, USB power saving ("autosuspend") and a fragile disk enclosure.

So: **install on an internal SSD if you possibly can.** If you must use USB, use a USB SSD (not a spinning hard disk), plug it directly into a blue USB 3 port (not a hub), keep the laptop on its charger, and apply the USB autosuspend fix in Chapter 5. A 128 GB SATA or NVMe SSD costs little and is the single best upgrade for this project.

Space needed: Proxmox ~10 GB, container 30-50 GB (Docker images ~5 GB, each AI model 1-5 GB, NocoDB data tiny), plus room for backups.

Power and running costs

A laptop idling as a server typically draws about 8-20 watts, more while the AI is working. A rough monthly cost is *watts × 24 × 30 ÷ 1000 × price per kWh*. For example, 15 W at 25p per kWh: 15 × 720 ÷ 1000 × £0.25 ≈ **£2.70 per month**. Desktops often draw 30-60 W idle (£5-£11 per month). Check your own tariff; these are estimates, not measurements.

> **Laptop battery health**
>
> A laptop on charge 24/7 keeps its battery at 100%, which ages it faster. Some laptops, mostly business models, support charge limits of about 75-80% that can be set from Linux with a tool called TLP. This is optional and NOT VERIFIED here.

PART II

Build

Stages B to J, in order. Every stage ends with a success check. Do not move on until the check passes: problems are much easier to fix while they are fresh.

- Chapter 4: terminal and SSH basics (read first)
- Chapters 5-8: the server (Stages B-E)
- Chapters 9-12: the project, tracker, Telegram and AI (Stages F-H)
- Chapters 13-14: starting it and your first run (Stages I-J)

CHAPTER 4

Terminal and SSH basics

Most of this project is set up by typing short commands. If you have never done that, this chapter gives you the five skills you need. Nothing here can break anything.

What a terminal is

A **terminal** (also called a shell, console or command line) is a text window where you type an instruction, press `Enter`, and the computer replies with text. Linux servers are normally managed this way because it is precise and can be copied exactly from a guide like this one.

The text at the start of each line is the **prompt**. It tells you who you are and which machine you are typing on:

READING A PROMPT

```
root@pve:~#
 |    |  | \__ "#" means you are the all-powerful root user
 |    |  \____ "~" means you are in your home folder
 |    \_______ the machine's name (here the Proxmox host "pve")
 \____________ your user name
```

★ **Real experience: typing in the wrong place**

While building the original system, a network command meant for the Proxmox host was pasted into the container. It failed with `iptables: Bad rule`. Always look at the prompt before pressing Enter: `root@pve` is the host, `root@agent-box` is the container.

Opening a terminal

- **On the Proxmox host:** in the Proxmox web page, click your server's name (e.g. **pve**) in the left tree, then **>_ Shell**. A terminal opens in the browser.
- **In the container:** click the container (e.g. **100 (agent-box)**) and then **Console**; or, from the host shell, type `pct enter 100`. Type `exit` to go back to the host.
- **On your own PC:** Windows: press `Start`, type *Terminal* (Windows 11) or *PowerShell* (Windows 10). Mac: open *Terminal* from Applications > Utilities. Linux: open *Terminal*.

Copying and pasting

Where	Copy	Paste
Proxmox web shell / console	Select text with the mouse	`Ctrl`+`Shift`+`V`, or right-click > Paste
Windows Terminal / PowerShell	Select, then `Ctrl`+`C`	`Ctrl`+`V` or right-click
Mac Terminal	`Cmd`+`C`	`Cmd`+`V`

In a terminal, `Ctrl`+`C` on its own means **stop the running command**, not copy. That is useful: if something seems stuck, press `Ctrl`+`C`. Copying text out of this PDF works, but check long commands arrived on one line.

> **Making the Proxmox web shell text bigger**
> Click your user name (top right, **root@pam**) > **My Settings** > **xterm.js Settings** > **Font-Size**.

Editing a file with nano

nano is a simple text editor that runs inside the terminal. You will use it to edit configuration files.

CONTAINER

```
nano /opt/jobagent/app/config.json
```

1. Move around with the arrow keys. The mouse does not move the cursor.
2. Type to insert text; `Backspace` deletes.
3. Save: press `Ctrl`+`O`, then `Enter` to confirm the file name.
4. Exit: press `Ctrl`+`X`. If it asks "Save modified buffer?", press `Y` then `Enter`.
5. Search: `Ctrl`+`W`, type the word, `Enter`.

The bottom two lines of nano always list the shortcuts; `^` means `Ctrl`.

Ten commands you will meet

Command	What it does
`ls -la /opt`	List the files in a folder, with details
`cd /opt/stacks/ai`	Change into a folder
`cat file.txt`	Print a whole (short) file on screen
`tail -f file.log`	Show the end of a file and keep following it (stop with `Ctrl`+`C`)
`nano file`	Edit a file
`apt update && apt upgrade -y`	Update the system's software (`&&` means "then, if that worked")
`hostname -I`	Show this machine's IP address(es)
`df -h`	Show disk space
`free -h`	Show memory use
`docker ps`	List running Docker apps

Linux is case-sensitive: `Config.json` and `config.json` are different files. Spaces matter too.

SSH: a terminal from your own PC

SSH (Secure Shell) lets you open the server's terminal from your own computer over the network, encrypted. It is more comfortable than the browser shell (proper copy and paste, bigger window) and is also how you copy files to the server. Proxmox has SSH switched on from the start; Windows 10/11, macOS and Linux all have the SSH program built in.

YOUR PC

```
ssh root@YOUR_SERVER_IP
```

The first time, it asks *"Are you sure you want to continue connecting (yes/no)?"*. Type `yes` and press `Enter`. Then type the Proxmox root password (nothing appears while you type; that is normal) and press `Enter`.

EXPECTED OUTPUT

```
Linux pve 6.x.x-pve #1 SMP PREEMPT_DYNAMIC ...
root@pve:~#
```

Success check

You see the `root@pve:~#` prompt. Type `exit` to disconnect.

Copying a file from your PC to the server uses `scp` (secure copy), which works the same way:

YOUR PC

```
scp ai-job-agent.zip root@YOUR_SERVER_IP:/root/
```

CHAPTER 5

Stage B: prepare the computer

Before installing anything, make sure the computer can do virtualisation, that you have saved anything you need from it, and that it is set up to stay awake on a wired network.

> **Installing Proxmox erases the disk you install it on**
> Everything on that disk (Windows, photos, documents) will be deleted. Copy anything you want to keep onto another drive or a cloud service **now**. If you have a Windows licence you might want later, note that reinstalling Windows needs a USB installer and, for older machines, a product key.

Check that virtualisation is supported

If the computer still runs Windows

1. Press `Ctrl`+`Shift`+`Esc` to open Task Manager. Click **More details** if you see a small window.
2. Open the **Performance** tab and click **CPU**.
3. Look at the bottom right for **Virtualisation: Enabled** (or "Virtualization").

"Enabled" means you are ready. "Disabled" means the processor supports it but it is switched off in the BIOS (next section). If the line is missing, also check the processor model online: it must be 64-bit with Intel VT-x or AMD-V.

If the computer runs Linux

YOUR PC (LINUX)

```
grep -Ec '(vmx|svm)' /proc/cpuinfo
```

Any number above `0` means the processor supports it.

Switch on virtualisation in the BIOS/UEFI

The **BIOS** (or its modern version, **UEFI**) is the small settings program that runs before any operating system. You open it by pressing a key immediately after switching on. Common keys:

Make	BIOS key	One-time boot menu key
Lenovo	`F1` (ThinkPad) or `F2`	`F12`
Dell	`F2`	`F12`
HP	`F10` (or `Esc` then choose)	`F9`
Acer / ASUS	`F2` or `Del`	`F12` / `F8` / `Esc`
Desktop motherboards	`Del`	`F8`, `F11` or `F12`

Tap the key repeatedly right after pressing the power button. In the BIOS, look under *Security*, *Advanced*, *CPU Configuration* or *Virtualization* for:

- **Intel Virtualization Technology (VT-x)** or **SVM Mode / AMD-V**: set to **Enabled**.
- **VT-d / IOMMU**: may be enabled; not required.
- **Secure Boot**: Proxmox 9 supports Secure Boot, so you can usually leave it on. If the installer refuses to start, turn Secure Boot off and try again.
- **Boot mode**: UEFI (leave as is on modern machines).

Save and exit, usually with `F10`.

> **Wake on power**
> Many BIOSes have an option such as *"Power on with AC attach"* or *"After power loss: Power On"*. Turning it on means the server starts by itself after a power cut. Optional but useful.

Storage

Decide which disk Proxmox will be installed on. An internal SSD is best (Chapter 3). If the computer has an old spinning hard disk and you can swap it for an SSD, do it before installing. If you have to use an external USB disk, read the USB section of Chapter 6 after installation.

Network

- Connect the computer to your router (or the wall socket in halls) with an **Ethernet cable**. Proxmox is designed for wired networks, and Wi-Fi is not supported for this setup.
- Find out the basic numbers of your network now, from another computer on the same network. On Windows, open Terminal and type `ipconfig`; note the **IPv4 Address** (e.g. 192.168.1.23), **Subnet Mask** (255.255.255.0 = /24) and **Default Gateway** (e.g. 192.168.1.1). On a Mac: System Settings > Network > your connection > Details > TCP/IP.
- Pick an address for the server that no other device uses. On home routers, a high number such as `.50` or `.200` is usually free. Better still, after installation, reserve that address for the server in your router's settings ("DHCP reservation").

> **Managed networks (student halls, hotels, workplaces)**
> Some networks only allow devices that have been registered on a web page, and redirect everything else to a sign-in page. On such networks you will use Network Option B in Stage D. Some also change your address from time to time; the troubleshooting chapter explains how to update it.

Laptops: power, lid and battery

A laptop is designed to sleep. As a server it must not. You will configure Proxmox to ignore the lid in Chapter 6; for now:

- Keep it **plugged into the charger** all the time. The original server's disk failures happened while it was running on battery.
- Put it somewhere ventilated, not on a bed or carpet.

Stage B success check

- [ ] Virtualisation shows as enabled (or you enabled it in the BIOS).
- [ ] Personal files are backed up elsewhere.
- [ ] The computer is on its charger and connected by Ethernet.
- [ ] You have written down the gateway and a free IP address for the server.

CHAPTER 6

Stage C: install Proxmox

Proxmox VE is the free operating system that will turn the computer into a server. You download it on another computer, copy it to a USB stick, start the server from that stick and answer a few questions.

Download the installer

1. On your everyday computer, open **https://www.proxmox.com/en/downloads**. This is the only official source.
2. Under **Proxmox Virtual Environment**, download the latest **Proxmox VE ISO Installer** (version 9.2 at the time of writing; any newer 9.x is fine). The file is about 1.7 GB and ends in `.iso`. Do not pick the ARM64 version unless your computer has an ARM processor (laptops and PCs with Intel or AMD chips do not).
3. Optional but recommended: check the download is complete and genuine. The download page shows a **SHA256SUM** code. On Windows, open Terminal in your Downloads folder and run the command below, then compare the long code with the one on the website. They must match exactly.

YOUR PC (WINDOWS)

```
Get-FileHash .\proxmox-ve_9.2-1.iso -Algorithm SHA256
```

YOUR PC (MAC / LINUX)

```
shasum -a 256 ~/Downloads/proxmox-ve_9.2-1.iso
```

Use the exact file name you downloaded; the version number may be newer.

Make a bootable USB stick

An `.iso` file is an image of an installation disc. You cannot just copy it onto a USB stick: you must "write" it with a tool that makes the stick bootable. **This erases the USB stick.**

Windows: Rufus

1. Download Rufus from **https://rufus.ie** (the portable version needs no installation) and open it.
2. **Device**: choose your USB stick. Double-check it is the stick, not another drive.
3. **Boot selection**: click **SELECT** and choose the Proxmox `.iso`.
4. Click **START**. If Rufus asks about the writing mode, choose **Write in DD Image mode** (the Proxmox documentation recommends this). Confirm the warning about erasing.
5. Wait until the bar says **READY**, then close Rufus.

Mac, Linux or Windows: balenaEtcher

1. Download balenaEtcher from **https://etcher.balena.io** and open it.

2. **Flash from file** > choose the `.iso`; **Select target** > your USB stick; **Flash!**
3. On a Mac, if it then says the disk is unreadable, click **Eject**. That is normal.

Start the server from the USB stick

1. Plug the USB stick into the server (and the Ethernet cable and charger).
2. Switch it on and immediately tap the **one-time boot menu** key (table in Chapter 5, e.g. `F12`).
3. Choose the USB stick from the list (it may be called "UEFI: USB ..." or by the stick's brand).
4. The Proxmox welcome menu appears. Choose **Install Proxmox VE (Graphical)** and press `Enter`.

> ? **It says "No support for hardware-accelerated KVM virtualization detected"**
> Virtualisation is still off in the BIOS. You can continue, but containers then have to do without it; better to switch it on (Chapter 5) and start again.

The installer, screen by screen

Screen 1: End User License Agreement

Proxmox is open source (AGPLv3 licence). Click **I agree**.

Screen 2: Target Harddisk



Figure 6.1. Illustration (simplified; your disk names will differ). **1**: choose the disk to erase and install on. **2**: Options. **3**: keep **ext4**. ZFS is excellent on servers with lots of RAM but uses extra memory; on a low-spec single-disk machine ext4 (which the original system used, with LVM) is the right choice.

If you see several disks, identify yours by its size and brand. A USB stick of 16 GB or a USB disk will also appear in the list: do not pick the installer stick.

Screen 3: Location and Time Zone

Country: **United Kingdom** (or yours). Time zone: **Europe/London** (or yours). Keyboard layout: **U.K. English** (or yours: it matters for typing the password later!). Next.

Screen 4: Administration Password and Email

- **Password**: this is the **root** (administrator) password for the whole server. Use a long, unique password, e.g. four random words. Write it down somewhere safe. In this handbook it is never shown; you type it when asked.
- **Email**: an address where the server can send alerts (e.g. failed backups). You can use your own address; it is stored only on your server.

Screen 5: Management Network Configuration

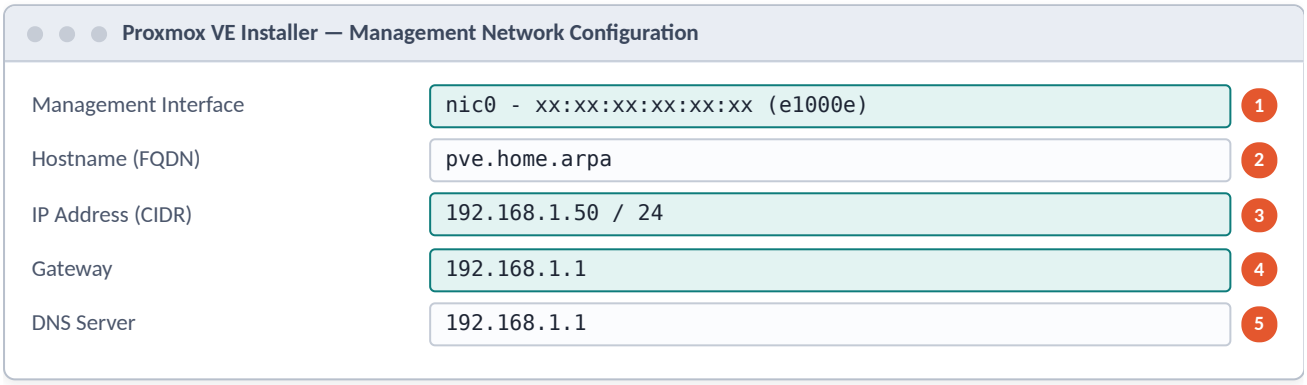


Figure 6.2. Illustration with example values. **1**: the wired network card (newer Proxmox names it `nic0`; older names look like `enp0s31f6`). **2**: a name for the server; `pve.home.arpa` is fine. **3**: the free address you chose in Chapter 5, followed by `/24`. **4**: your router's address. **5**: your router, or `1.1.1.1`.

The installer usually pre-fills these from your network. If the values look sensible, keep them but note down the IP address: this is `YOUR_SERVER_IP` from now on.

Screen 6: Summary

Check everything, tick **Automatically reboot after successful installation**, and click **Install**. It takes 5-15 minutes. When the computer restarts, **remove the USB stick**.

The server now shows a black screen with a login prompt and a message like:

SERVER SCREEN AFTER REBOOT

```
Welcome to the Proxmox Virtual Environment. Please use your web browser to
configure this server - connect to:

  https://192.168.1.50:8006/

pve login: _
```

You do not need to log in on the server itself. From now on you manage it from another device.

First login to the web interface

1. On your everyday computer or phone (same network), open a browser and go to `https://YOUR_SERVER_IP:8006`. Note **https** and **:8006**.

2. The browser warns that the connection is not private. This is expected: Proxmox uses a self-made certificate. Click **Advanced** > **Continue to ... (unsafe)** (Chrome/Edge) or **Accept the Risk and Continue** (Firefox).
3. Log in with user name **root**, the password from Screen 4, and realm **Linux PAM standard authentication**.
4. A box says **"No valid subscription"**. This is normal for free use: click **OK**. (Paid subscriptions give access to the enterprise update channel and support; you do not need one.)

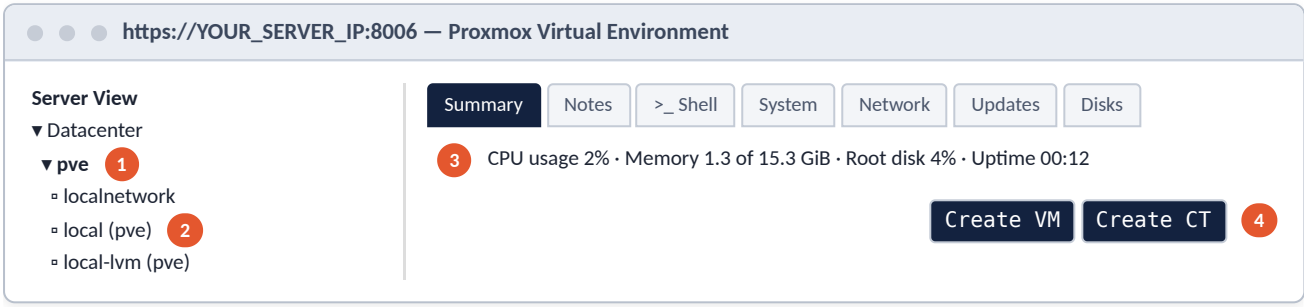


Figure 6.3. Illustration of the main screen. **1**: your server (node), with tabs including **Shell**, **Network** and **Updates**. **2**: storage: *local* holds templates and backups, *local-lvm* holds container disks. **3**: live usage. **4**: buttons to create a VM or a CT (container), top right.

Switch to the free update channel and update

Out of the box, Proxmox is set to the paid "enterprise" update channel, which refuses updates without a subscription. Switch to the free **no-subscription** channel:

1. Click your server (**pve**) > **Updates** > **Repositories**.
2. Click **Add**, choose **No-Subscription**, click **Add**.
3. Select each line whose name contains **enterprise** (Proxmox VE and Ceph) and click **Disable**.
4. Go to **Updates**, click **Refresh**, wait for "TASK OK", then click **Upgrade**. A shell window opens; type `y` and `Enter` if asked.
5. When finished, click **Reboot** (top right) if the update included a new kernel.

The same update from the host shell:

PROXMOX HOST

```
apt update && apt full-upgrade -y
```

Stage C success check

PROXMOX HOST

```
pveversion
hostname -I
```

EXPECTED OUTPUT

```
pve-manager/9.x.x/...   (running kernel: 6.x or newer ...)
192.168.1.50
```

The version starts with 9 and the address is the one you chose. You can log in to `https://YOUR_SERVER_IP:8006` from another device.

Laptop settings: lid, USB and console

Keep running with the lid closed

By default, closing the lid may suspend the laptop, which takes your server offline. Tell the system to ignore the lid:

PROXMOX HOST

```
nano /etc/systemd/logind.conf
```

Find the lines below (they start with `#`, which means "switched off"). Remove the `#` and set the values to `ignore`:

/ETC/SYSTEMD/LOGIND.CONF

```
HandleLidSwitch=ignore
HandleLidSwitchExternalPower=ignore
HandleLidSwitchDocked=ignore
```

Save (`Ctrl`+`O`, `Enter`), exit (`Ctrl`+`X`) and apply:

PROXMOX HOST

```
systemctl restart systemd-logind
```

NOT VERIFIED on the original laptop, but this is the standard Debian setting. Test it: close the lid, wait a minute, and check the web interface still responds.

Stop USB power saving (important if you boot from a USB disk)

Linux can "autosuspend" idle USB devices to save power. For a USB system disk this can cause sudden disconnections (see the real experience in Chapter 3). Switch it off immediately:

PROXMOX HOST

```
echo -1 > /sys/module/usbcore/parameters/autosuspend
```

That lasts until the next reboot. To make it permanent, add a kernel option. Proxmox installed on ext4 normally starts with the **GRUB** boot loader. Check which one you have:

PROXMOX HOST
```
proxmox-boot-tool status
```

If it says it is not configured / uses GRUB (the usual case with ext4):

PROXMOX HOST
```
nano /etc/default/grub
```

/ETC/DEFAULT/GRUB (CHANGE THIS LINE)
```
GRUB_CMDLINE_LINUX_DEFAULT="quiet usbcore.autosuspend=-1"
```

PROXMOX HOST
```
update-grub
```

If it lists systemd-boot (usual with ZFS): add `usbcore.autosuspend=-1` to the end of the single line in `/etc/kernel/cmdline`, then run `proxmox-boot-tool refresh`.

After the next reboot, verify:

PROXMOX HOST
```
cat /proc/cmdline
cat /sys/module/usbcore/parameters/autosuspend
```

EXPECTED OUTPUT
```
BOOT_IMAGE=/boot/vmlinuz-... root=/dev/mapper/pve-root ro quiet usbcore.autosuspend=-1
-1
```

★ **Real experience**

On the original system the permanent version of this fix was planned but not yet verified when the second disk dropout happened. Do it now, and check `/proc/cmdline` after rebooting.

Optional: bigger console text

If you ever use the laptop's own screen, the default text is tiny. Run `dpkg-reconfigure console-setup` and choose UTF-8, the default character set, font **TerminusBold** and size **14x28** (the original system's choice).

Alternative routes

Route 1: no second computer

You need another device only to make the USB stick and to open the web page. Options: make the stick on a friend's, library or college computer (buying a ready-made "Proxmox USB stick" is *not* recommended, because you cannot know what is on it). After installation, a **smartphone browser** can manage Proxmox

perfectly well at `https://YOUR_SERVER_IP:8006` (there is also a mobile layout). If the computer you are converting is your only computer, you can make the stick on it before installing, and use your phone afterwards.

Route 2: skip Proxmox (4 GB machines, VPS, or "just one box")

The apps do not need Proxmox. Instead, install **Debian 13** (from **https://www.debian.org**, "netinst" image, same USB method; choose "SSH server" and "standard system utilities", untick any desktop environment) or **Ubuntu Server 24.04 LTS**. Then skip Stage D: run everything from Stage E onwards directly on that system, using `sudo -i` to become root. You lose the easy rebuild and container backups, but save about 1 GB of RAM.

Route 3: you already have Proxmox

Skip to Stage D and create one more container.

CHAPTER 7

Stage D: create the apps container

All the project's apps live in one lightweight Linux container on the Proxmox host. In this stage you copy the project package to the server, download a Debian template, choose a network option and create the container.

Step 1: put the project package on the server

The project package is the file `ai-job-agent.zip` that comes with this handbook (Chapter 20 lists its contents). Download it to your everyday computer, then copy it to the Proxmox host with `scp` (Chapter 4):

YOUR PC (IN THE FOLDER WHERE THE ZIP IS)

```
scp ai-job-agent.zip root@YOUR_SERVER_IP:/root/
```

Or, if the package is published at a web address, download it straight onto the host:

PROXMOX HOST

```
wget -O /root/ai-job-agent.zip "PACKAGE_DOWNLOAD_URL"
```

`PACKAGE_DOWNLOAD_URL` means the link where you obtained this handbook and package.

Unpack it on the host (only needed for the optional network script):

PROXMOX HOST

```
apt install -y unzip
cd /root && unzip -o ai-job-agent.zip && ls ai-job-agent
```

EXPECTED OUTPUT

```
README.md  dockge  host  install.sh  jobagent  stacks
```

Step 2: download the Debian template

A **template** is a ready-made, minimal Linux system that Proxmox copies to create a container. We use **Debian 13** ("trixie"), the same Linux family Proxmox itself is built on.

1. In the left tree, click **local (pve)** > **CT Templates** > **Templates**.
2. In the search box type `debian-13`.
3. Select **debian-13-standard** (the newest version shown) and click **Download**. Wait for "TASK OK" and close the window.

Command-line alternative, with the safety filter that picks the correct processor type:

PROXMOX HOST

```
pveam update
T=$(pveam available --section system | awk '/debian-13-standard.*amd64/ {print $2}' | tail
-1)
```

```
echo "$T"
pveam download local "$T"
```

★ **Real experience: the wrong processor type**

On the original system, a looser filter picked the **arm64** build of the template. Proxmox printed *"Detected container architecture: arm64"* and the container refused to start. The fix was to destroy it and download the **amd64** template. Intel and AMD PCs always need **amd64**; check that the template name ends in `_amd64.tar.zst`.

Step 3: choose your network option

	Option A: normal (vmbr0)	Option B: private NAT (vmbr1)
Use when	Home router, normal network	Network blocks or "registers" each new device (halls, hotel, work)
Container gets	Its own address from the router	A private address 10.10.10.100 behind the host
Apps reached at	`http://YOUR_CONTAINER_IP:PORT`	`http://YOUR_SERVER_IP:PORT` (ports forwarded)
Extra setup	None	Add bridge vmbr1 on the host (below)
Original system		Used this

★ **Real experience: why Option B exists**

On a student-halls network, every new network card (identified by its MAC address) is treated as an unregistered device, and its web traffic is redirected to a registration page. Inside a container on vmbr0 this showed up as `apt update` failing with *"Hash Sum mismatch"* and *"Clearsigned file isn't valid"*: the "updates" were actually the network's sign-in page. Putting containers behind the host on a private NAT bridge means the network only ever sees the Proxmox host, which is already registered.

Option B only: create the private bridge on the host

Easy way: run the script from the package. It shows what it will do, asks for confirmation and backs up the network file first.

PROXMOX HOST

```
bash /root/ai-job-agent/host/setup-nat-bridge.sh
```

Manual way (exactly what the original system uses). Open the host's network configuration:

PROXMOX HOST

```
cp /etc/network/interfaces /etc/network/interfaces.bak
nano /etc/network/interfaces
```

Go to the very end of the file and add:

/ETC/NETWORK/INTERFACES (APPEND AT THE END)

```
auto vmbr1
iface vmbr1 inet static
        address 10.10.10.1/24
        bridge-ports none
        bridge-stp off
        bridge-fd 0
        post-up   iptables -t nat -A POSTROUTING -s 10.10.10.0/24 -o vmbr0 -j MASQUERADE
        post-down iptables -t nat -D POSTROUTING -s 10.10.10.0/24 -o vmbr0 -j MASQUERADE
        post-up   iptables -t nat -A PREROUTING -i vmbr0 -p tcp -m multiport --dports
3000,5001,5678,8081 -j DNAT --to-destination 10.10.10.100
        post-down iptables -t nat -D PREROUTING -i vmbr0 -p tcp -m multiport --dports
3000,5001,5678,8081 -j DNAT --to-destination 10.10.10.100
```

What each line means: `address` gives the host the address 10.10.10.1 on the new private network; `bridge-ports none` means it is not connected to any physical cable; the **MASQUERADE** rule lets containers reach the internet through the host (NAT); the **DNAT** rule forwards visitors on ports 3000, 5001, 5678 and 8081 of the host to the container at 10.10.10.100. Now switch on forwarding and apply:

PROXMOX HOST

```
echo "net.ipv4.ip_forward=1" > /etc/sysctl.d/99-forward.conf
sysctl -p /etc/sysctl.d/99-forward.conf
ifreload -a
ip addr show vmbr1
```

EXPECTED OUTPUT

```
... vmbr1: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
    inet 10.10.10.1/24 scope global vmbr1
```

Adding a port later

The original system added port 8081 after setup. Edit the `--dports` list on **both** DNAT lines in `/etc/network/interfaces`, then reboot the host (or remove and re-add the live rule with `iptables -t nat -D ...` / `-A ...` using the old and new lists). Do this on the host, not in the container.

Step 4: create the container

Click **Create CT** (top right). The wizard has several tabs; fill them in as follows.

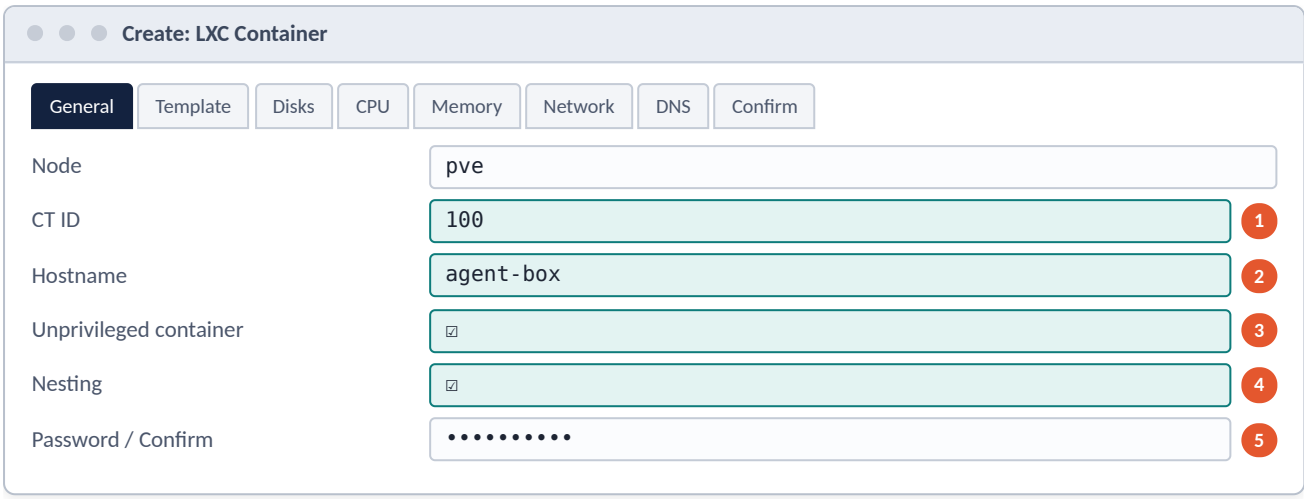


Figure 7.1. Illustration of the General tab. **1**: the container's number (100 is the first free one). **2**: any name; this handbook uses *agent-box*. **3**: keep ticked (safer). **4**: must be ticked, or Docker will not work inside. **5**: a root password for the container (can differ from the host's).

Tab	Setting	16 GB machine (original)	8 GB machine	Why
General	CT ID / Hostname	100 / agent-box; Unprivileged ☑; Nesting ☑; password		See figure
Template	Storage / Template	local / debian-13-standard_..._amd64.tar.zst		Must be amd64
Disks	Storage / Disk size	local-lvm / 50 GB	local-lvm / 30-40 GB	Docker images + AI models
CPU	Cores	6 (of 8 threads)	all but 1	AI speed depends on cores
Memory	Memory / Swap (MiB)	12288 / 4096	6144 / 2048	Leave ~2 GB for Proxmox
Network	Option A	Name eth0, Bridge **vmbr0**, IPv4 **DHCP** (or Static with a free address), IPv6 SLAAC or Static none		Router gives an address
	Option B	Name eth0, Bridge **vmbr1**, IPv4 **Static**, IPv4/CIDR `10.10.10.100/24`, Gateway `10.10.10.1`		Must match the forwarding rules
DNS	DNS servers	Option A: leave blank (use host settings). Option B: `1.1.1.1`		Name lookups
Confirm	Start after created	Leave unticked for now		We add two settings first

Click **Finish** and wait for "TASK OK".

For a 4 GB machine, Proxmox is not recommended (Chapter 6, Route 2). If you do use it: 2560 MiB memory, 1024 swap, 25 GB disk, all cores.

Step 5: extra container settings

1. Click the new container **100 (agent-box)** > **Options**.

2. Double-click **Start at boot**, tick it, OK. The container now starts whenever the server starts.
3. Double-click **Features**: make sure **nesting** is ticked and also tick **keyctl** (helps Docker in unprivileged containers). OK.

Allow Tailscale inside the container (optional, recommended)

Tailscale (Stage E) needs a virtual network device called TUN, which containers cannot use by default. On the host:

PROXMOX HOST

```
nano /etc/pve/lxc/100.conf
```

Add these two lines at the end (100 is your CT ID):

/ETC/PVE/LXC/100.CONF (APPEND)

```
lxc.cgroup2.devices.allow: c 10:200 rwm
lxc.mount.entry: /dev/net/tun dev/net/tun none bind,create=file
```

For reference, the whole file then looks roughly like this (Option B shown):

/ETC/PVE/LXC/100.CONF (EXAMPLE)

```
arch: amd64
cores: 6
features: keyctl=1,nesting=1
hostname: agent-box
memory: 12288
nameserver: 1.1.1.1
net0:
name=eth0,bridge=vmbr1,gw=10.10.10.1,hwaddr=XX:XX:XX:XX:XX:XX,ip=10.10.10.100/24,type=veth
onboot: 1
ostype: debian
rootfs: local-lvm:vm-100-disk-0,size=50G
swap: 4096
unprivileged: 1
lxc.cgroup2.devices.allow: c 10:200 rwm
lxc.mount.entry: /dev/net/tun dev/net/tun none bind,create=file
```

Step 6: start the container and update it

Select the container and click **Start**. Then open a terminal inside it, either with **Console** (log in as `root` with the container password) or from the host shell:

PROXMOX HOST

```
pct enter 100
```

? **pct needs the number**

`pct enter agent-box` or `pct enter new` does not work; always use the CT ID number.

The prompt changes to `root@agent-box:~#`. Update the container and test the internet:

CONTAINER

```
apt update && apt upgrade -y
ping -c 3 deb.debian.org
hostname -I
```

EXPECTED OUTPUT

```
...
3 packets transmitted, 3 received, 0% packet loss ...
192.168.1.60          (Option A: an address from your router)
10.10.10.100          (Option B)
```

For Option A, write down this address: it is `YOUR_CONTAINER_IP`. Reserve it in your router so it never changes, or set it as Static in the container's Network tab.

Stage D success check

- [ ] `apt update` finishes without "Hash Sum mismatch" errors (if not, use Option B).
- [ ] `ping` shows 0% packet loss.
- [ ] The container's Options show Start at boot = Yes and Features nesting=1, keyctl=1.

CHAPTER 8

Stage E: install Docker and Tailscale

Docker is the engine that runs all the apps. Tailscale is optional but gives you safe access from anywhere. Everything in this chapter is typed inside the container.

Docker

What it is: software that runs applications in sealed, pre-packaged boxes (Docker containers) built from **images** that the apps' authors publish. **Why:** installing Ollama, NocoDB and the rest becomes one command each, and they cannot interfere with each other. **Where it fits:** it runs inside the LXC container and hosts every app in Figure 2.1.

Docker's official convenience script installs the current Docker Engine and the **Compose** plugin (the part that reads `compose.yaml` recipes):

CONTAINER

```
apt install -y curl ca-certificates
curl -fsSL https://get.docker.com -o get-docker.sh
sh get-docker.sh
```

This takes a few minutes. (The package's `install.sh` does the same thing automatically if Docker is missing, so this step is also safe to skip; doing it by hand lets you see that it works.)

Verify Docker

CONTAINER

```
docker --version
docker compose version
docker run --rm hello-world
```

EXPECTED OUTPUT

```
Docker version 29.x.x, build ...
Docker Compose version v2.x or newer (any version number is fine)
...
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

> **If hello-world fails with a permission or "operation not permitted" error**
>
> Nesting is not enabled for the container. On the host: container > Options > Features > tick nesting (and keyctl), then restart the container.

Tailscale (optional, recommended)

What it is: a free service that builds a private, encrypted network (a "tailnet") between devices signed into your account, wherever they are. **Why:** you can open your tracker from your phone on mobile data, without opening any ports on your router or exposing anything to the internet. **Where it fits:** installed in the container, it gives the container a second address of the form `100.x.y.z`.

1. Create a free account at **https://tailscale.com** (sign in with Google, Microsoft, GitHub or Apple).
2. Install the Tailscale app on your phone and/or laptop and sign in with the same account.
3. Install it in the container:

CONTAINER

```
curl -fsSL https://tailscale.com/install.sh | sh
tailscale up
```

EXPECTED OUTPUT

```
To authenticate, visit:

        https://login.tailscale.com/a/xxxxxxxxxxx
```

1. Open that link on any device, sign in and approve the new machine.
2. Get the container's Tailscale address:

CONTAINER

```
tailscale ip -4
```

EXPECTED OUTPUT

```
100.x.y.z
```

This is `YOUR_TAILSCALE_IP`. From any device signed into your tailnet, apps are now at `http://YOUR_TAILSCALE_IP:8081` and so on, from anywhere.

> **"failed to connect to local tailscaled" or "/dev/net/tun does not exist"**
> The two TUN lines from Stage D are missing in `/etc/pve/lxc/100.conf`, or the container was not restarted afterwards.

Also on the Proxmox host (recommended)

The container's Tailscale address reaches the apps, but not the Proxmox web interface. To manage Proxmox remotely too, run the same two commands on the host:

PROXMOX HOST

```
curl -fsSL https://tailscale.com/install.sh | sh
tailscale up
tailscale ip -4
```

Then use `https://HOST_TAILSCALE_IP:8006`. `NOT VERIFIED`: planned on the original system but not yet done.

Stage E success check

- [ ] `docker run --rm hello-world` prints "Hello from Docker!".
- [ ] (Optional) `tailscale status` lists your devices, and your phone shows the container as connected in the Tailscale app.

CHAPTER 9

Stage F: install the project

Now the actual job agent. You copy the package into the container, run the installer, and learn where every file lives and what it does.

Copy the package into the container

In Stage D you put `ai-job-agent.zip` on the host. `pct push` copies a file from the host into a container:

PROXMOX HOST

```
pct push 100 /root/ai-job-agent.zip /root/ai-job-agent.zip
```

Then, inside the container:

CONTAINER

```
apt install -y unzip
cd /root && unzip -o ai-job-agent.zip && cd ai-job-agent
ls
```

EXPECTED OUTPUT

```
README.md  dockge  host  install.sh  jobagent  stacks
```

If you skipped Proxmox (Route 2), copy the zip straight to the Debian/Ubuntu machine with `scp` and unzip it there.

What is in the package, and where it goes

In the package	Installed to	Purpose
`install.sh`	(run once)	The installer
`stacks/ai/compose.yaml`	`/opt/stacks/ai/`	Ollama + Open WebUI recipe; creates the shared network
`stacks/jobs/compose.yaml`	`/opt/stacks/jobs/`	NocoDB (+ optional n8n, Gotenberg)
`stacks/jobagent/Dockerfile`, `compose.yaml`	`/opt/stacks/jobagent/`	Builds and runs the Job Finder container
`stacks/jobagent/.env.example`	`/opt/stacks/jobagent/.env`	**Your secrets** (made from the example)
`dockge/compose.yaml`	`/opt/dockge/`	Optional Docker web manager
`jobagent/app/finder.py`	`/opt/jobagent/app/`	The Job Finder program
`jobagent/app/config.json`	`/opt/jobagent/app/`	**Your search settings**
`jobagent/cv/master.md`	`/opt/jobagent/cv/`	**Your profile** (template to replace)
`jobagent/cv/answers.yaml`	`/opt/jobagent/cv/`	Your fixed form answers (for the planned apply part)

In the package	Installed to	Purpose
`jobagent/tools/*.py`	`/opt/jobagent/tools/`	Helpers: find chat ID, test all connections
	`/opt/jobagent/state/`	`seen.json` (memory of processed adverts) and `run.log`
`host/setup-nat-bridge.sh`	(host only)	Optional Network Option B

Inside the Job Finder's Docker container, the folder `/opt/jobagent` appears as `/data`. That is why commands such as `docker exec jobagent python /data/app/finder.py` use `/data`, while you edit the same file at `/opt/jobagent/app/finder.py`.

Run the installer

Choose the command for your machine (Chapter 3):

CONTAINER

```
bash install.sh                              # 16 GB: everything
bash install.sh --minimal                    # 8 GB: no Open WebUI, no Dockge
bash install.sh --minimal --model qwen2.5:1.5b   # 4 GB
```

It prints what it will do and asks *Continue? [y/N]*; type `y`. Then, step by step, it:

1. installs a few basic tools (`curl`, `locales`, `tzdata`, `nano`);
2. sets the time zone (Europe/London unless you add `--tz`) and creates English locales (this stops "perl: warning: Setting locale failed" messages, which the original system had);
3. installs Docker if it is not already there;
4. creates the folders in the table above and copies the files, **never overwriting** your own `config.json`, `master.md`, `answers.yaml` or `.env` if they already exist (so it is safe to run again);
5. creates `.env` and makes it readable by root only (`chmod 600`);
6. starts Ollama, NocoDB and (unless `--minimal`) Open WebUI and Dockge, then downloads the AI model (1-5 GB; 5-30 minutes);
7. builds the Job Finder container but only starts it once your keys are filled in.

EXPECTED OUTPUT (END)

```
==> Building the Job Finder container
    OK Image built
    !! Job Finder NOT started yet. Still to fill in /opt/stacks/jobagent/.env:
       REED_API_KEY TELEGRAM_TOKEN TELEGRAM_CHAT_ID NOCODB_TOKEN NOCODB_TABLE_ID

================================================================
 Installation finished.
   NocoDB tracker : http://192.168.1.60:8081
   Open WebUI     : http://192.168.1.60:3000
   Dockge         : http://192.168.1.60:5001
 (With the NAT network option, use the Proxmox host's address instead.)
 ...
```

Success check

CONTAINER

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

EXPECTED OUTPUT

```
NAMES        STATUS          PORTS
dockge       Up 2 minutes    0.0.0.0:5001->5001/tcp
nocodb       Up 2 minutes    0.0.0.0:8081->8080/tcp
open-webui   Up 3 minutes    0.0.0.0:3000->8080/tcp
ollama       Up 3 minutes    127.0.0.1:11434->11434/tcp
```

All say **Up**. (`--minimal` shows only ollama and nocodb.) The `jobagent` container appears after Stage G.

Manual installation (what install.sh does, by hand)

If you prefer to understand every step, or the script stops with an error, these commands do the same job. Run them from inside the unzipped `ai-job-agent` folder.

CONTAINER

```
apt install -y curl ca-certificates locales tzdata nano
ln -sf /usr/share/zoneinfo/Europe/London /etc/localtime
sed -i 's/^# *\(en_GB.UTF-8 UTF-8\)/\1/; s/^# *\(en_US.UTF-8 UTF-8\)/\1/' /etc/locale.gen
&& locale-gen

mkdir -p /opt/stacks/{ai,jobs,jobagent} /opt/dockge \
         /opt/jobagent/{app,cv,state,output,tools}
cp stacks/ai/compose.yaml   /opt/stacks/ai/
cp stacks/jobs/compose.yaml /opt/stacks/jobs/
cp stacks/jobagent/{Dockerfile,compose.yaml,.env.example} /opt/stacks/jobagent/
cp dockge/compose.yaml      /opt/dockge/
cp jobagent/app/{finder.py,config.json} /opt/jobagent/app/
cp jobagent/cv/{master.md,answers.yaml} /opt/jobagent/cv/
cp jobagent/tools/*.py      /opt/jobagent/tools/
cp /opt/stacks/jobagent/.env.example /opt/stacks/jobagent/.env
chmod 600 /opt/stacks/jobagent/.env
chown -R 1000:1000 /opt/jobagent

cd /opt/stacks/ai   && docker compose up -d     # creates the "shared" network
cd /opt/stacks/jobs && docker compose up -d
cd /opt/dockge      && docker compose up -d     # optional
docker exec ollama ollama pull qwen2.5:3b
cd /opt/stacks/jobagent && docker compose build
```

Order matters

Start the AI stack first. It creates the private Docker network called `shared` that the other stacks join (they list it as `external`). Starting NocoDB first fails with *"network shared declared as external, but could not be found"*.

The secrets file: .env

Where: `/opt/stacks/jobagent/.env`. **Open:** `nano /opt/stacks/jobagent/.env`. It holds every key and password the Job Finder uses. You fill it in during Stages F2, G and H.

Line	Change?	Value
`REED_API_KEY`	Yes	Your Reed key (Stage H)
`ADZUNA_APP_ID`, `ADZUNA_APP_KEY`	Optional	Adzuna's short App ID and long key; leave the placeholders to skip Adzuna
`TELEGRAM_TOKEN`, `TELEGRAM_CHAT_ID`	Yes	Stage G
`NOCODB_URL`	No	`http://nocodb:8080`: the app's name on the private Docker network
`NOCODB_TOKEN`, `NOCODB_TABLE_ID`	Yes	Stage F2
`OLLAMA_URL`	No*	`http://ollama:11434` (*unless Ollama runs on another PC)
`SCORING_MODEL`	Optional	`qwen2.5:3b`, or the model you downloaded
`TZ`	Optional	Your time zone; the run times follow this clock

Rules: one setting per line, no spaces around `=`, no quotes, no trailing spaces. **Apply changes** with `cd /opt/stacks/jobagent && docker compose up -d --force-recreate` (a plain restart does *not* reload `.env`). **Verify** with the setup check in Stage H.

> **Never share this file**
> Do not paste it into chats (including AI chats), screenshots, emails, forums or GitHub. On the original system, keys were pasted into a chat during setup, and they all had to be replaced afterwards (Chapter 17).

Your search settings: config.json

Where: `/opt/jobagent/app/config.json`. **Open:** `nano /opt/jobagent/app/config.json`. The format is JSON: text in "double quotes", items separated by commas, **no comma after the last item** of a list. The example targets entry-level cyber security roles in the UK; change it to your field.

Setting	Example	Meaning
`run_times`	`["07:30", "18:30"]`	When to search each day (24-hour clock, server time zone).
`search_keywords`	`"SOC Analyst"`, ...	Each one is a separate search on each job board. 5-10 is a good number.
`location`	`""` or `"Leeds"`	Empty = whole UK. A town or postcode limits the search to around it.

Setting	Example	Meaning
`distance_miles`	`20`	Radius around `location` (ignored when location is empty).
`title_must_contain_one_of`	`"Security"`, `"Analyst"`	An advert is kept only if its title contains at least one of these words (not case-sensitive). Use an empty list `[]` to switch this off.
`avoid_title_keywords`	`"Senior"`, `"Lead"`	Whole words that reject an advert if they appear in the title.
`skip_if_description_contains`	`"SC clearance"`	Phrases that reject an advert if found anywhere in its text. The example includes visa-sponsorship refusals: delete those lines if sponsorship does not matter to you.
`max_days_old`	`3`	Ignore adverts older than this.
`max_jobs_to_score_per_run`	`40`	AI scoring limit per run. 40 × ~60 s ≈ 40 minutes on a laptop CPU. The rest wait for the next run.
`min_match_score`	`70`	Minimum AI score to save and report a job.
`telegram_top_n`	`15`	How many matches to list in the Telegram message.

Verify your edit is valid JSON, then apply it:

CONTAINER

```
python3 -m json.tool /opt/jobagent/app/config.json > /dev/null && echo "config OK"
docker restart jobagent        # once the jobagent container exists
```

If you see an error such as *Expecting ',' delimiter: line 12 column 5*, look at that line for a missing comma or quote.

Your profile: master.md

Where: `/opt/jobagent/cv/master.md`. This is the most important file in the project: the AI judges every advert against it. Replace the whole example with your own facts. Writing it is covered in Chapter 14; for now just know where it is. It is plain text with `#` headings (Markdown format).

> **The golden rule**
> Everything in `master.md` must be true. The AI is told to judge only from this file and never to assume skills that are not in it. Future CV tailoring will only be allowed to select, reorder and rephrase what is here, never to add. If it is not in this file, it is not true.

Fixed answers: answers.yaml

Where: `/opt/jobagent/cv/answers.yaml`. Your standard answers to application-form questions (right to work, notice period, salary expectations, willingness to relocate). The current Job Finder does **not** use this file; it is prepared for the planned apply part, which must copy these exact answers and never let the AI make

them up. Fill it in truthfully now while you are thinking about it. YAML format: `key: value`, indentation with spaces (never tabs).

CHAPTER 10

Stage F2: set up the NocoDB tracker

NocoDB is a free spreadsheet-style database that runs on your server. The Job Finder adds each good match as a row; you use it to keep track of what you have applied for.

> **Screens change between versions**
> NocoDB is updated often. Button names below match recent versions; if yours look slightly different, look for the same words nearby. The field names, however, must be exactly as shown, because the Job Finder uses them.

Create the admin account

1. In a browser, open `http://YOUR_CONTAINER_IP:8081` (Option A) or `http://YOUR_SERVER_IP:8081` (Option B), or `http://YOUR_TAILSCALE_IP:8081`.
2. The first visit shows a sign-up page. Enter an email address and a strong password. This creates the **super admin** account. It is stored only on your server; nothing is sent anywhere.

Create the base and the table

1. Create a new **Base** (a project, like a spreadsheet file) and name it **Job Agent**.
2. Inside it, create a **Table** named **Applications**.
3. New tables come with a first column called **Title**. Click its header > **Edit** and rename it to **Company**.
4. Add the other fields with the **+** button at the end of the header row. Choose the type from the list, type the name exactly (capitals and spaces matter), and save.

Field name (exact)	Type	Filled by
`Company`	Single line text (the renamed Title)	Job Finder
`Role`	Single line text	Job Finder
`Link`	URL	Job Finder
`Source`	Single line text	Job Finder (Reed / Adzuna)
`Match Score`	Number	Job Finder
`Status`	Single select, options: *Found*, *Tailored*, *Approved*, *Applied*, *Skipped*, *Manual*, *Rejected*, *Interview*	Job Finder sets *Found*; you change it
`CV File`	Single line text	Reserved for planned CV tailoring

Field name (exact)	Type	Filled by
`Applied At`	DateTime	You
`Notes`	Long text	Job Finder (AI reasons, gaps, location); you can add more
`Job ID`	Single line text	Job Finder (e.g. `reed-12345678`)

NocoDB — Job Agent › Applications

Company	Role	Link	Source	Match Score	Status	Notes	Job ID
Example Ltd	Graduate SOC Analyst	https://...	Reed	86	FOUND	Level fit: good ...	reed-1000001
Sample Group	Cyber Security Analyst	https://...	Adzuna	74	APPLIED	Reasons: SIEM ...	adzuna-500001
Demo Services	IT Security Analyst	https://...	Reed	71	SKIPPED	Gaps: needs CCNA	reed-1000002

Figure 10.1. Illustration of the finished table with invented example rows. Columns not filled automatically are hidden here for space.

Create an API token

The Job Finder needs a token (a password for programs) to add rows.

1. Click your account (bottom-left or top-right) > **Account Settings** > **Tokens** (in some versions: *Team & Settings* > *Tokens*).
2. Click **Create token** / **Add new token**, name it `jobagent`, and save.
3. Copy the token (a long string of letters and numbers). This is `YOUR_NOCODB_API_TOKEN`.

Find the table ID

Every table has an internal ID, usually starting with the letter **m** (e.g. `m1a2b3c4d5e6f7g`).

- Open the **Applications** table and look for **Details** > **APIs** (or **REST APIs** / **API Snippets**). The example address shown there contains `/api/v2/tables/THE_ID/records`.
- Or look at the browser address bar while the table is open: the last part of the address after `/table/` (or the last `/`) is usually the table ID.

Put both values into `.env`:

/OPT/STACKS/JOBAGENT/.ENV

```
NOCODB_TOKEN=YOUR_NOCODB_API_TOKEN
NOCODB_TABLE_ID=YOUR_NOCODB_TABLE_ID
```

✓ **Success check**

You can see the empty **Applications** table with all ten fields, and both values are in `.env`. The connection itself is tested by `check_setup.py` at the end of Stage H.

CHAPTER 11

Stage G: create your Telegram bot

The Job Finder reports to you through Telegram. You create your own private bot by chatting with Telegram's official bot-making account, then tell the Job Finder the bot's token and your chat number.

What a Telegram bot is

A bot is a Telegram account run by a program. Anyone can make one for free. It has a display name, a unique username ending in `bot`, and a secret **token** that acts as its password: whoever has the token controls the bot. A bot cannot start a conversation with you; you must press **Start** in its chat first. After that, the bot can send you messages using your **chat ID**, a number that identifies your chat with it.

Create the bot with @BotFather

1. Install Telegram on your phone and sign in.
2. Search for **@BotFather**. Choose the one with the blue verified tick; ignore look-alikes.
3. Press **Start**, then send `/newbot`.
4. Send a display name, e.g. *My Job Agent*.
5. Send a username ending in `bot`, e.g. *myname_jobs_2026_bot*. It must be unique; try another if taken.
6. BotFather replies with the token. Copy it. It looks like `1234567890:AAH...` (a number, a colon and about 35 characters). This is `YOUR_TELEGRAM_BOT_TOKEN`.
7. Recommended: send `/setjoingroups`, choose your bot, then **Disable**, so nobody can add it to groups.

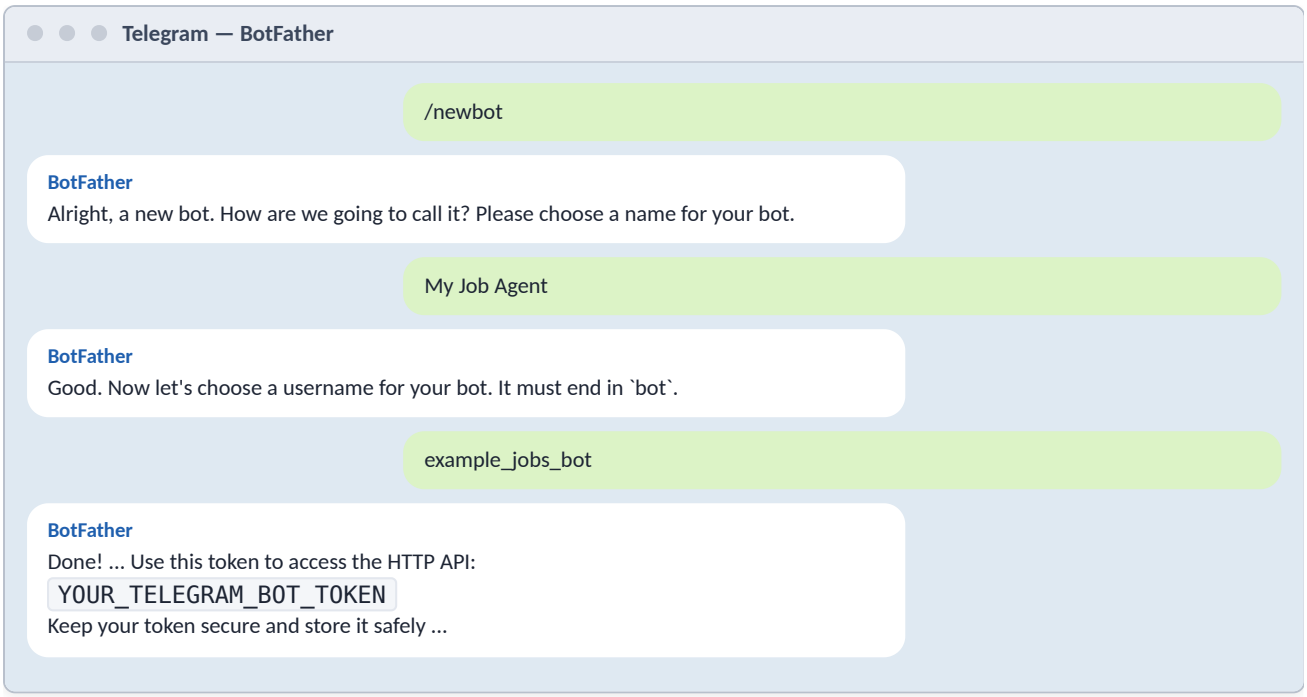


Figure 11.1. Illustration of the BotFather conversation (wording shortened; real token replaced with a placeholder).

> **Protect the token**
>
> Anyone with the token can send messages as your bot and read messages sent to it. Never screenshot or share it. If it leaks, send `/revoke` to BotFather, choose your bot, and put the new token in `.env`.

Start the Job Finder and find your chat ID

1. Put the token into `.env`: `nano /opt/stacks/jobagent/.env`, replace `YOUR_TELEGRAM_BOT_TOKEN`, save and exit.
2. In Telegram, search for your bot's username, open it, press **Start** and send `hello`.
3. Start the Job Finder container (it simply waits for its run times, so it is safe to start before everything is filled in):

CONTAINER

```
cd /opt/stacks/jobagent
docker compose up -d --build
docker exec jobagent python /data/tools/get_chat_id.py
```

EXPECTED OUTPUT

```
Chats that recently messaged your bot:
  chat ID: 123456789        name: YourFirstName        type: private

Put YOUR chat ID (type 'private') into .env as TELEGRAM_CHAT_ID=...
```

1. Copy the number into `.env` as `TELEGRAM_CHAT_ID=`, save, and apply it:

CONTAINER

```
docker compose up -d --force-recreate
```

> **"No messages found"?**
> Make sure you pressed Start and sent a message to *your* bot (not BotFather), wait a few seconds, and run the command again.

Send a test message

CONTAINER

```
docker exec jobagent python /data/tools/check_setup.py --send
```

The Telegram line should say OK and your phone receives *"Job agent online - test message"*. Other lines may still say FAIL until you finish Stage H; that is expected.

What the messages look like

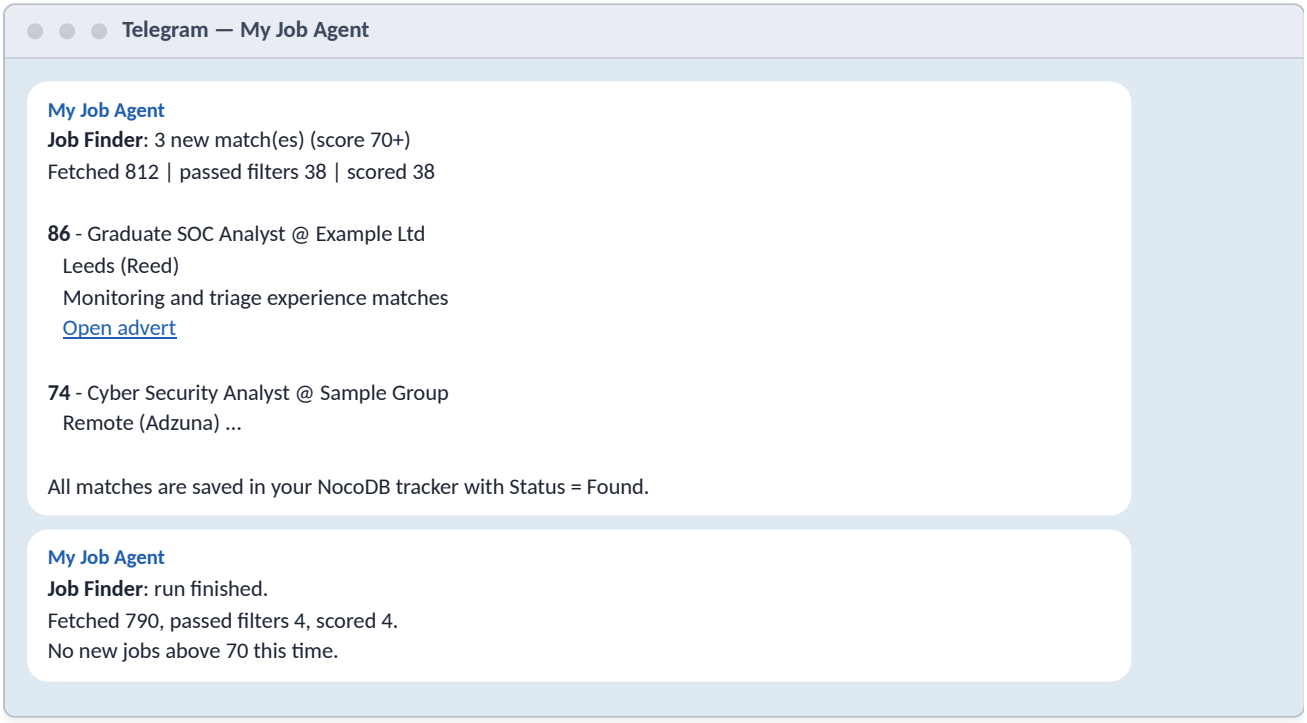


Figure 11.2. Illustration of a match summary and a "nothing new" message, using invented companies. One message is sent at the end of every run.

About approve and reject

The current system only **sends** messages; it does not read your replies, and there are no approve or reject buttons. You approve a job by applying for it yourself and setting its Status in NocoDB (Chapter 14). Approve/reject buttons are PLANNED (Part 3, Chapter 19). When built, they must check that every button

press comes from your own chat ID and ignore everyone else; disabling group joins now (step 7 above) is part of that protection.

CHAPTER 12

Stage H: job-board keys and the AI

The last ingredients: permission to search Reed (and optionally Adzuna), and a working AI model. At the end of this stage one command tests every connection.

Get a Reed API key

Reed.co.uk is one of the UK's largest job boards and gives free API keys to developers.

1. Go to **https://www.reed.co.uk/developers/jobseeker**.
2. Sign in or register, fill in the short form (you can describe the use as "personal job search tool") and accept the terms.
3. Your API key is shown and/or emailed to you: a long code with dashes. This is `YOUR_REED_API_KEY`.
4. Put it in `.env` as `REED_API_KEY=`.

How it is used: the Job Finder sends the key as the user name of a standard web login with an empty password (this is how Reed's API works), asks for up to 100 full-time adverts per keyword, then downloads the full text of each advert that passes the filters.

Get Adzuna keys (optional)

Adzuna collects adverts from many sites and adds variety. Skip this if you want to keep things simple.

1. Go to **https://developer.adzuna.com** and register.
2. Your dashboard shows an **Application ID** and an **Application Key**.
3. Put them in `.env`: `ADZUNA_APP_ID=` the **short** ID (8 characters) and `ADZUNA_APP_KEY=` the long key.

> ★ **Real experience**
>
> On the original system the Adzuna App ID was left blank because the wrong value had been copied: the App ID is the short 8-character code, not the long key. With either value missing, the Job Finder simply skips Adzuna and logs *"skipping Adzuna"*.

How the AI part works

Ollama runs in its own Docker container and keeps the model loaded in memory for 24 hours after use (`OLLAMA_KEEP_ALIVE=24h`; the original used 10 minutes at first, which meant reloading the model each run). The Job Finder sends each advert to Ollama's `/api/chat` address with these settings:

Setting	Value	Why
Model	`SCORING_MODEL` from `.env`	Default `qwen2.5:3b`: good balance on a CPU
`format`	`json`	Forces a structured answer the program can read

Setting	Value	Why
`temperature`	0.1	Low randomness: consistent scores
`num_ctx`	8192	Working memory in tokens; profile + advert must fit (Ollama's default is smaller)
`num_predict`	300	Maximum answer length. Fixes a real bug: in JSON mode the model sometimes produced endless blank space until a 15-minute timeout
Timeout	360 s	Gives up on one advert after 6 minutes; it is retried next run
Advert text	first 2,500 characters	Keeps each request fast

Download, list and test models

The installer downloads the model you chose. To manage models by hand:

CONTAINER

```
docker exec ollama ollama list                # what is installed
docker exec ollama ollama pull qwen2.5:3b     # download a model
docker exec ollama ollama rm llama3.2:3b      # delete one you do not use
```

EXPECTED OUTPUT (OLLAMA LIST)

```
NAME          ID              SIZE      MODIFIED
qwen2.5:3b    357c53fb659c    1.9 GB    2 minutes ago
```

Test the AI directly (Ollama only listens inside the container, so run this in the container):

CONTAINER

```
curl -s http://127.0.0.1:11434/api/chat -d '{"model":"qwen2.5:3b","stream":false,
 "messages":[{"role":"user","content":"Reply with exactly five words."}]}'
```

After a few seconds (longer the first time, while the model loads) you get a line of JSON containing `"content":"..."` with a short reply. To switch the Job Finder to another model, pull it, set `SCORING_MODEL=` in `.env` and run `docker compose up -d --force-recreate` in `/opt/stacks/jobagent`.

Costs: local models cost nothing per use; you only pay for electricity (Chapter 3). There are no API bills because no cloud AI is used.

Test everything at once

With all values in `.env`, apply them and run the setup check:

CONTAINER

```
cd /opt/stacks/jobagent && docker compose up -d --force-recreate
docker exec jobagent python /data/tools/check_setup.py
```

EXPECTED OUTPUT

```
AI Job Agent - setup check
  Files                 OK    10 keywords, master.md 612 words
  Reed API              OK    48211 jobs available
  Adzuna API            OK    1250 jobs available       (or: SKIP not configured)
  Ollama + model        OK    qwen2.5:3b is installed
  NocoDB tracker        OK    tracker table reachable
  Telegram              OK    bot @example_jobs_bot (add --send to send a test message)
All required checks passed.
```

Each FAIL line includes a hint. The most common ones are in the table below and in Chapter 15.

Message	Meaning and fix
Reed: key rejected (401)	Key mistyped or has spaces. Copy it again.
Adzuna: rejected	App ID and key swapped, or long key used as App ID.
model ... not downloaded	Run the `ollama pull` command shown, or fix the model name in `.env`.
Ollama: cannot connect	AI stack not running (`cd /opt/stacks/ai && docker compose up -d`) or the containers are not on the `shared` network.
NocoDB: table not found	Wrong table ID (see Chapter 10).
NocoDB: token rejected	Create a new token.
Telegram: token rejected	Token mistyped or revoked.

Optional: Open WebUI, and making it fast

Open WebUI is a chat page for your local AI, useful for experimenting (for example, asking it to proofread your `master.md`). Open `http://YOUR_CONTAINER_IP:3000` (or the host/Tailscale address). The first account you create becomes the administrator; it is stored only on your server.

> ★ **Real experience: chat replies took a minute, or talked about "a list of functions"**
> Ollama's log showed that a simple "hi" was being sent as about **2,050 tokens**: Open WebUI was quietly adding instructions for tools, web search and other features. On a laptop CPU reading 35-45 tokens per second, that is 50-60 seconds before the reply even starts. Sending "hi" directly to Ollama used about 30 tokens and replied instantly. Switching off the extras fixed it.

To make Open WebUI fast on a CPU:

1. **Admin Panel > Settings > Models**: open each model and untick capabilities such as *Builtin Tools*, *Web Search*, *Code Interpreter* and *Image Generation*; remove any attached tools. Save.
2. **Admin Panel > Settings**: switch **Web Search** off and **Code Interpreter** off.

3. **Admin Panel > Settings > Interface**: switch off title generation, tags generation, follow-up generation and autocomplete generation. Leave *Task Model* on "Current Model" (choosing a specific model there gave the error *"This model is not publicly available"* on the original system).
4. Your own **Settings**: empty the system prompt and switch **Memory** off.

Check the result by sending "hi" and then looking at Ollama's log:

CONTAINER

```
docker logs ollama --tail 30 2>&1 | grep "task.n_tokens" | tail -1
```

A value around 30 (instead of 2,000) means the fix worked. The `2>&1` is needed because Docker writes these logs to the "error" stream, which `grep` otherwise does not see.

✓ **Stage H success check**

`check_setup.py` ends with **All required checks passed.**

CHAPTER 13

Stage I: start, stop and automate

Everything is installed. This chapter explains how the system starts by itself, how to check it, how to read its logs, and how to stop and restart it safely.

How automatic start-up works

Three layers each start the next one after a reboot or power cut:

1. **Proxmox** starts the container because you ticked *Start at boot* (Stage D).
2. **Docker** starts automatically inside the container (it is installed as a system service).
3. Every app has `restart: unless-stopped` in its `compose.yaml`, so Docker restarts it unless you deliberately stopped it.

The Job Finder runs in **scheduler mode**: it stays running, checks the clock every 20 seconds and starts a run at each time in `run_times`. Nothing else (no cron job) is needed.

Verify after any reboot:

CONTAINER

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

EXPECTED OUTPUT

```
NAMES        STATUS
jobagent     Up 3 minutes
dockge       Up 3 minutes
nocodb       Up 3 minutes
open-webui   Up 3 minutes
ollama       Up 3 minutes
```

Changing the schedule

Edit `run_times` in `/opt/jobagent/app/config.json`, e.g. `["06:45", "12:30", "19:00"]`, check the JSON (Chapter 9) and run `docker restart jobagent`. Choose times when the server is idle. Each run takes from a few minutes to about 40 minutes, depending on how many adverts pass the filters.

CONTAINER

```
docker logs jobagent --tail 5
```

EXPECTED OUTPUT

```
[2026-11-03 07:29:41] Scheduler started. Checking the clock every 20 seconds.
```

Running a search by hand

In the foreground (you watch it; it stops if you close the terminal or browser):

CONTAINER

```
docker exec jobagent python /data/app/finder.py --once
```

In the background (keeps going if you close the browser; output goes to a log file):

CONTAINER

```
docker exec -d jobagent sh -c "python /data/app/finder.py --once >> /data/state/run.log
2>&1"
tail -f /opt/jobagent/state/run.log
```

Press `Ctrl`+`C` to stop *watching* the log; the run continues.

> ★ **Real experience: run it once**
>
> Starting the background command twice by accident starts two runs side by side. They compete for the processor, both run slowly and both send results. Check nothing is already running first: `docker top jobagent | grep -c -- "--once"` should print `0` when no manual run is active.

Reading the logs

What	Command
Job Finder (scheduled runs)	`docker logs -f jobagent`
Job Finder (background manual runs)	`tail -f /opt/jobagent/state/run.log`
Ollama (AI)	`docker logs --tail 50 ollama`
NocoDB	`docker logs --tail 50 nocodb`
Search inside a log	`docker logs jobagent 2>&1 | grep -i error`

A healthy run looks like this (from the test run of the package, with simulated job boards):

EXAMPLE RUN LOG

```
[2026-11-03 09:54:16] Run started (model qwen2.5:3b, threshold 70)
[2026-11-03 09:54:16] Reed: 'SOC Analyst' -> 8 results
[2026-11-03 09:54:16] Reed: 'Security Operations Analyst' -> 0 results
...
[2026-11-03 09:54:16] Adzuna: 'SOC Analyst' -> 2 results
...
[2026-11-03 09:54:16] Fetched 8 unique jobs, 8 not seen before
[2026-11-03 09:54:16] 4 jobs passed the filters
[2026-11-03 09:54:16] [1/4]  82 | SOC Analyst @ Acme Ltd (0s)
[2026-11-03 09:54:16] [2/4] skipped after full text: skip phrase 'unable to sponsor'
[2026-11-03 09:54:16] [3/4]  55 | Security Analyst @ Epsilon (0s)
[2026-11-03 09:54:16] [4/4]  91 | Graduate Cyber Security Analyst @ Eta & Sons (0s)
[2026-11-03 09:54:16] Run finished: 3 scored, 2 saved as matches
```

On real hardware each scored line takes 40-70 seconds instead of 0 s, and the first one is slower while the profile is read for the first time.

Stopping and restarting

Goal	Command	Where
Restart the Job Finder (after editing config.json, master.md or finder.py)	`docker restart jobagent`	Container
Reload .env	`cd /opt/stacks/jobagent && docker compose up -d --force-recreate`	Container
Stop one stack	`cd /opt/stacks/ai && docker compose stop`	Container
Start it again	`cd /opt/stacks/ai && docker compose up -d`	Container
Stop the Job Finder until you start it again	`docker stop jobagent`	Container
Shut down the container cleanly	`pct shutdown 100`	Host
Reboot the whole server	`reboot`	Host
Switch the server off	`shutdown now`	Host
Reboot in 10 minutes / cancel	`shutdown -r +10` / `shutdown -c`	Host

> **Do not just pull the plug**
> Always shut down from the web interface (top right **Shutdown**) or with `shutdown now`. Cutting power can corrupt files, especially on USB disks. Holding the power button is a last resort for a completely frozen machine.

Optional: managing stacks with Dockge

Open `http://YOUR_CONTAINER_IP:5001` (or host/Tailscale address) and create an admin account on the first visit. Dockge shows the stacks in `/opt/stacks` (**ai**, **jobs**, **jobagent**) with Start, Stop, Restart and Update buttons, live logs, and an editor for each `compose.yaml`. It does the same as the commands above; use whichever you prefer.

Recovery after a reboot or power cut

1. Wait 2-3 minutes after power-on, then open `https://YOUR_SERVER_IP:8006`. If it does not load, see Chapter 15 ("Cannot open the Proxmox web page").
2. Check the container shows a green running icon. If not: select it > **Start**.
3. In the container, run `docker ps` and check all apps are Up.
4. If you use a USB system disk, check the autosuspend fix is still active: `cat /sys/module/usbcore/parameters/autosuspend` on the host should print `-1`.
5. Optionally start one manual run in the background to catch up.

✓ **Stage I success check**

After `reboot` on the host, all five (or, with `--minimal`, three) apps are Up without you doing anything, and `docker logs jobagent` shows "Scheduler started".

CHAPTER 14

Stage J: your first real run

Time to make it yours: write your profile, set your preferences, run a search, and learn the daily routine from Telegram message to submitted application.

Step 1: write your master profile

Open the file and replace the entire example with your own information:

CONTAINER

```
nano /opt/jobagent/cv/master.md
```

Tips for a profile that scores well and stays honest:

- **Be specific and complete.** "Investigated 30+ phishing emails using Microsoft Defender" helps the AI far more than "security experience". Include tools, numbers, dates and results.
- **Include everything true** that a recruiter in your field might look for: modules, projects, part-time jobs, volunteering, home lab work. Leave out anything you could not discuss in an interview.
- **No exaggeration.** If you have only read about a tool, do not list it. The AI scores you against the file, so inflated profiles produce matches you cannot win.
- **No sensitive data.** Do not include ID or passport numbers, bank details, your full home address or date of birth. Contact lines are for future CV generation; the Job Finder never sends them anywhere (the profile only goes to your own local AI).
- **Keep it under about 1,500 words** (1,000 on 4 GB machines) so it fits comfortably in the AI's working memory.
- Copying from your CV is a good start: open your CV, copy the text, and paste it into nano (`Ctrl`+`Shift`+`V` in the web console). Then add detail.

Apply: `docker restart jobagent`. Check the word count with `wc -w /opt/jobagent/cv/master.md`.

Step 2: set your job preferences

Edit `/opt/jobagent/app/config.json` (all settings are explained in Chapter 9):

1. **Target roles**: put 5-10 job titles you would genuinely apply for in `search_keywords`. Use the words employers use in adverts.
2. **Relevance filter**: put the core words of those titles in `title_must_contain_one_of` so unrelated adverts are dropped before the slow AI step.
3. **Seniority**: keep or adjust `avoid_title_keywords`. If you are experienced, remove "Senior" and "Lead".
4. **Location**: `"location": ""` searches the whole UK (good if you will relocate). For a commutable area: `"location": "Manchester", "distance_miles": 25`.

5. **Deal-breakers**: add phrases to `skip_if_description_contains` for requirements you can never meet. Remove the sponsorship phrases if sponsorship is irrelevant to you.
6. **Threshold**: start with `"min_match_score": 70`. If you get too many weak matches, raise it to 75; if almost none, lower it to 65.

Validate and apply:

CONTAINER

```
python3 -m json.tool /opt/jobagent/app/config.json > /dev/null && echo "config OK"
docker restart jobagent
```

Step 3: start the first search

Do not wait for the next scheduled time; start a background run now and watch it:

CONTAINER

```
docker exec -d jobagent sh -c "python /data/app/finder.py --once >> /data/state/run.log
2>&1"
tail -f /opt/jobagent/state/run.log
```

You will see the searches, then the filter counts, then one line per scored advert. On the original laptop the first real run fetched 818 adverts, 40 passed the filters, and scoring took roughly 30-45 minutes. You can close the browser; the run continues. Real example scores from that run: *SOC Analyst at a recruitment agency: 70-75*; *SOC Analyst at another agency: 45-60*; *L2 SOC Analyst: 70*.

Step 4: review the matches

When the run finishes, Telegram shows the summary (Figure 11.2). For each match:

1. Tap **Open advert** and read the whole advert yourself. The score is a first sort, not a decision.
2. In NocoDB, open the row and read **Notes**: the AI's level fit, reasons and gaps. Check the reasons are actually true for you and the gaps are fair.
3. Check the advert is genuine: a real company, a sensible salary, no request for payment, no request to move to WhatsApp or another chat app for an "interview". Job scams exist on every board.
4. Decide: set **Status** to *Approved* (you will apply), *Skipped* (not for you) or *Manual* (needs a special process such as an online test).

> **Sort and filter the tracker**
> In NocoDB, use **Sort** on *Match Score* (descending) and **Filter** on *Status is Found* to see only new matches, best first. Save it as a view called "To review".

Step 5: apply, and record it

1. Apply through the advert's normal "Apply" button, with a CV you have checked yourself.

2. If you adapt your CV for the job, change only how true facts are presented: choose and order the most relevant ones and use the advert's wording where it honestly describes what you did. Never add skills, tools, qualifications or experience you do not have, and never change dates or job titles.
3. Read the final application once more before you click Submit.
4. In NocoDB set **Status** to *Applied*, fill **Applied At**, and add anything useful to **Notes** (contact name, reference number).
5. Later, update the status to *Interview* or *Rejected*. Over time the tracker shows which kinds of jobs respond to you, which helps you tune your keywords.

> **You are the approval step**
> The current system never submits anything. Every application is made by you, after you have read the advert and checked your CV. Keep it that way even if you later automate more (Chapter 18).

Step 6: tune over the first week

You notice	Change
Lots of irrelevant adverts reach the AI	Tighten `title_must_contain_one_of`; add skip phrases.
Good jobs never appear	Add more `search_keywords`; check they are not blocked by `avoid_title_keywords` (e.g. a title containing "Lead" in another sense).
Scores seem too generous or too harsh	Adjust `min_match_score`; make `master.md` more specific.
Runs take too long	Lower `max_jobs_to_score_per_run`; tighten filters; use a smaller model.
You want to re-score everything (e.g. after rewriting your profile)	Delete the memory: `rm /opt/jobagent/state/seen.json` (adverts already in NocoDB will be added again if they still match).

> **Stage J success check**
> A run completed, you received a Telegram summary, matching jobs appear in NocoDB with Status *Found*, and you have reviewed at least one and updated its Status. Congratulations: the system is working.

PART III

Keep it running

Fixing problems, maintaining the system, keeping it secure, using it responsibly, and where the project goes next.

- Chapter 15: troubleshooting (35 problems)
- Chapter 16: maintenance and backups
- Chapter 17: security and privacy
- Chapter 18: responsible use
- Chapter 19: roadmap
- Chapter 20: the download package

CHAPTER 15

Troubleshooting

Each problem below has five parts: what it looks like, why it happens, how to find the cause, how to fix it, and how to confirm it is fixed. Problems marked REAL actually happened on the original system.

General approach

1. **Read the exact error message.** Copy it; search for it in this chapter (use your PDF reader's search).
2. **Check where you are typing** (host or container prompt).
3. **Check the basics:** `docker ps` (apps running?), `df -h` (disk full?), `free -h` (memory?).
4. **Read the logs:** `docker logs --tail 50 <name>`.
5. **Change one thing at a time** and re-test.

Proxmox and hardware

Cannot open the Proxmox web page REAL

LOOKS LIKE `https://YOUR_SERVER_IP:8006` times out or "site can't be reached".

WHY Wrong address (forgot https or :8006), server off, cable unplugged, or the network changed. On the original system this happened after moving to new accommodation: the static address set during installation no longer matched the new network.

DIAGNOSE On the server's own screen, log in as root and run `hostname -I` and `ip route`. Compare the address and gateway with what other devices on this network use (Chapter 5).

FIX Edit the address and gateway in the `vmbr0` section of `/etc/network/interfaces`, and the same address in `/etc/hosts` (the line with your host name). Then apply:

PROXMOX HOST (ON ITS OWN SCREEN)

```
nano /etc/network/interfaces      # address x.x.x.x/24 and gateway under
vmbr0
nano /etc/hosts                   # same address before the host name
ifreload -a
systemctl restart pveproxy
```

VERIFY The web page opens at the new address. Reserve the address in the router so it does not change again.

Installer: "No support for hardware-accelerated KVM virtualization"

LOOKS LIKE Warning during Proxmox installation.

WHY VT-x / AMD-V is switched off in the BIOS, or the CPU lacks it.

DIAGNOSE / FIX Enable it in the BIOS (Chapter 5). On the installed host, `grep -Ec '(vmx|svm)' /proc/cpuinfo` must be above 0.

VERIFY Reinstall or reboot; the warning no longer appears.

"Input/output error" on every command, Docker dead, "Bus error" REAL

LOOKS LIKE Simple commands such as `ls` fail with *Input/output error*; `dmesg` itself fails with *Bus error*; the web page may still half-work.

WHY The system disk has disappeared, usually a USB disk disconnecting (power saving, battery power, loose cable, failing enclosure). It happened twice in one day on the original server.

DIAGNOSE If anything still works: `dmesg | tail -50` shows "USB disconnect" or "I/O error, dev sda". Usually nothing works.

FIX

1. Hold the power button for 10 seconds (a normal shutdown is impossible without the disk).
2. Plug in the charger. Reseat the USB cable in a USB 3 port directly on the computer.
3. Power on. If the boot stops in **emergency mode**, see the next problem.
4. Apply the USB autosuspend fix permanently (Chapter 6) and plan a move to an internal SSD (Chapter 19).
5. Check the disk's health: `apt install -y smartmontools` then `smartctl -a /dev/sda` (for USB enclosures add `-d sat`). Look for "PASSED" and for non-zero *Reallocated_Sector_Ct* or *Current_Pending_Sector*, which mean the disk is failing.

VERIFY `cat /sys/module/usbcore/parameters/autosuspend` prints -1; the server stays up for several days.

Boot stops in "emergency mode" REAL

LOOKS LIKE "You are in emergency mode… Give root password for maintenance".

WHY The file system was not closed cleanly (power loss or disk dropout) and needs checking.

FIX Type the root password, then:

PROXMOX HOST (EMERGENCY SHELL)

```
fsck -y /dev/mapper/pve-root
reboot
```

VERIFY Normal boot; the web page loads; containers start.

Server goes offline when the lid is closed or after a while

LOOKS LIKE	Everything stops responding until you open the lid or press a key.
WHY	The laptop is suspending.
FIX	Set the lid options in `/etc/systemd/logind.conf` to `ignore` and restart `systemd-logind` (Chapter 6). Check BIOS power settings for sleep timers.
VERIFY	Close the lid for 10 minutes; the web page still loads.

Network

apt: "Hash Sum mismatch" or "Clearsigned file isn't valid" REAL

LOOKS LIKE	`apt update` in the container fails with these errors, or downloads return web pages.
WHY	The network sees the container as a new, unregistered device and redirects its web traffic to a registration/sign-in page (a "captive portal").
DIAGNOSE	`curl -sI http://deb.debian.org | head -5` in the container: a *302* or *Location:* pointing at a portal address confirms it.
FIX	Use Network Option B (Stage D): move the container to `vmbr1` with address 10.10.10.100/24 and gateway 10.10.10.1 (container > Network > edit net0), after creating vmbr1 on the host. Alternatively, register the container's MAC address on the network's portal if that is allowed.
VERIFY	`apt update` completes cleanly.

Container has no internet

LOOKS LIKE	`ping -c 3 1.1.1.1` fails, or `ping deb.debian.org` says "Temporary failure in name resolution".
WHY	Wrong gateway or bridge; Option B forwarding or NAT not active; no DNS server.
DIAGNOSE	In the container: `ip addr`, `ip route`, `cat /etc/resolv.conf`. On the host (Option B): `sysctl net.ipv4.ip_forward` must be 1, and `iptables -t nat -L -n` must show the MASQUERADE rule.
FIX	Correct the container's Network tab (bridge, IP, gateway) and DNS tab (e.g. 1.1.1.1). For Option B, re-check the vmbr1 block and run `ifreload -a` on the host. If 1.1.1.1 pings but names fail, it is DNS only.
VERIFY	Both pings succeed.

The apps' web pages (8081, 3000, 5001) do not open

LOOKS LIKE	Browser times out or refuses the connection.
WHY	Wrong address (Option A uses the *container's* IP; Option B the *host's*); app not running; port missing from the forwarding rule; your device is not on the same network or not on Tailscale.
DIAGNOSE	In the container: `docker ps` (is it Up, and does it show the port?) and `curl -sI http://127.0.0.1:8081 | head -1`. On the host (Option B): `iptables -t nat -L PREROUTING -n` must list the port.
FIX	Start the app; use the right address; add the port to both DNAT lines (Stage D tip).
VERIFY	The page loads from your phone/laptop.

iptables: "Bad rule" or command not found REAL

LOOKS LIKE	A network command from this book fails immediately.
WHY	It was typed inside the container (`root@agent-box`) instead of on the host (`root@pve`).
FIX	Type `exit` to leave the container, check the prompt shows the host, and run it again.
VERIFY	No error.

SSH: "Connection timed out", "Connection refused" or "Permission denied"

LOOKS LIKE	`ssh root@YOUR_SERVER_IP` fails.
WHY	Timed out: wrong address or different network. Refused: SSH not running. Permission denied: wrong password or keyboard layout. "REMOTE HOST IDENTIFICATION HAS CHANGED": you reinstalled the server.
FIX	Check the address; on the host run `systemctl status ssh`; retype the password carefully. For the changed-identity warning, run `ssh-keygen -R YOUR_SERVER_IP` on your PC and connect again.
VERIFY	You get the `root@pve` prompt.

Container and Docker

Container will not start: "Detected container architecture: arm64" REAL

WHY	The wrong (ARM) template was downloaded.
FIX	Destroy the container (select it > More > Remove), delete the wrong template from local > CT Templates, download the `..._amd64.tar.zst` template (Stage D) and create the container again.
VERIFY	The container starts; `dpkg --print-architecture` inside prints `amd64`.

"pct enter new" or "pct enter agent-box" fails REAL

FIX Use the numeric ID: `pct enter 100`. List IDs with `pct list`.

Docker will not run inside the container

LOOKS LIKE `docker run hello-world` fails with permission, "operation not permitted" or cgroup errors; or install.sh says "Docker is installed but not running".

WHY *nesting* (and sometimes *keyctl*) is not enabled for the LXC container.

FIX Host: container > Options > Features > tick nesting and keyctl; restart the container; then `systemctl restart docker` inside.

VERIFY hello-world prints its message.

"network shared declared as external, but could not be found"

WHY The AI stack, which creates the `shared` network, has not been started yet.

FIX `cd /opt/stacks/ai && docker compose up -d`, then start the other stack again. (Or create it by hand: `docker network create shared`.)

VERIFY `docker network ls` lists `shared`.

Downloading images fails ("pull access denied", timeouts, "no space left")

WHY No internet or DNS in the container, Docker Hub temporarily rate-limiting, a typo in an image name, or a full disk.

FIX Fix connectivity first (above); check `df -h`; wait and retry; compare the image names with Appendix A.

VERIFY `docker compose pull` completes.

Apps are not running after a reboot

DIAGNOSE Host: is the container running? Container: `systemctl status docker`; `docker ps -a` shows stopped apps and exit codes.

FIX Tick *Start at boot* for the container; `systemctl enable --now docker`; check each compose file has `restart: unless-stopped`; start stacks with `docker compose up -d`. An app you stopped with `docker stop` stays stopped until started.

VERIFY Reboot the host; everything is Up after 3 minutes.

The Job Finder

The search returns no results ("Fetched 0 unique jobs")

DIAGNOSE	Look for lines like *Reed: 'X' failed: 401* (bad key), *skipping Reed* (key missing), or *-> 0 results* for every keyword. Run `check_setup.py`.
FIX	Fix the key; widen `search_keywords`; raise `max_days_old` (3 days is strict); clear `location` or increase `distance_miles`.
VERIFY	A manual run shows non-zero results.

Everything is filtered out ("0 jobs passed the filters")

WHY	Filters too strict, or all adverts already in `seen.json` ("N not seen before" is 0).
FIX	Check `title_must_contain_one_of` matches the titles you search for; remove over-broad skip phrases. To re-process everything: `rm /opt/jobagent/state/seen.json`.

Python errors: "No module named requests", "JSONDecodeError", "FileNotFoundError"

WHY	*requests*: you ran `python3 finder.py` directly in the LXC container instead of inside the `jobagent` Docker container. *JSONDecodeError* … *config.json*: a typo in config.json. *FileNotFoundError*: a file is missing from `/opt/jobagent`.
FIX	Always run through Docker: `docker exec jobagent python /data/app/finder.py --once`. Validate config with `python3 -m json.tool /opt/jobagent/app/config.json`. Re-run `install.sh` to restore missing program files (it keeps your own files).

Changes to .env have no effect

WHY	`docker restart` does not re-read `.env`. Or the value has quotes/spaces.
FIX	`cd /opt/stacks/jobagent && docker compose up -d --force-recreate`. Check a value safely without printing secrets: `docker exec jobagent sh -c 'echo ${#TELEGRAM_CHAT_ID}'` prints only its length.

Scheduled runs never happen

DIAGNOSE	`docker logs jobagent --tail 20`: is "Scheduler started" there? `docker exec jobagent date`: is the time zone right?
WHY / FIX	Container stopped (start it); wrong time format (must be `"07:30"`, with leading zero and quotes); `TZ` wrong in `.env`; server asleep at that time.
VERIFY	Set a run time two minutes ahead, restart, and watch the log.

Two runs at the same time REAL

LOOKS LIKE Doubled log lines, very slow scoring, two Telegram summaries.

FIX `docker restart jobagent` stops all runs inside it (the scheduler restarts cleanly). Start manual runs only once; check with `docker top jobagent`.

Matches are not saved in NocoDB

LOOKS LIKE Log: *NocoDB: could not save … 400 / 404 / 401*.

WHY 400: a field name differs (spelling, capitals, spaces) or *Found* is missing from the Status options. 404: wrong table ID. 401/403: bad token.

FIX Compare your fields with the table in Chapter 10 exactly; fix the ID or token; recreate the agent container after editing `.env`.

VERIFY `check_setup.py` NocoDB line is OK, and the next match appears as a row.

"Job applications fail"

NOTE The system does not submit applications (Chapter 1), so there is no automatic application to fail. If an employer's own form fails when you apply by hand, try another browser, check file size limits for your CV, or contact the employer.

The AI

Scoring is extremely slow or times out REAL

LOOKS LIKE Minutes per advert; *scoring failed … Read timed out*.

WHY On the original system the model occasionally produced endless whitespace in JSON mode, hitting a 15-minute timeout. The package already contains the fix (answer limited to 300 tokens, 6-minute timeout, advert cut to 2,500 characters). Other causes: model too large for the CPU, two runs at once, memory swapping.

FIX Use `qwen2.5:3b` or smaller; one run at a time; give the container more cores; lower `max_jobs_to_score_per_run`. Timed-out adverts are retried automatically next run.

VERIFY Log lines show 40-70 s per advert on a 4-core laptop chip.

"scoring failed: AI did not return JSON"

WHY Small models occasionally answer badly.

FIX Nothing, if it is rare: the advert is retried next run. If frequent, try another model (e.g. `llama3.2:3b`) or a bigger one.

"model not found" / Ollama 404

FIX `docker exec ollama ollama list`; pull the model or correct `SCORING_MODEL` in `.env` (exact name including the `:3b` tag) and recreate the agent container.

Scores look wrong, or the AI's reasons mention things you never did

WHY Small models can misread or "hallucinate" (state things that are not true). Vague profiles make this worse.

FIX Make `master.md` concrete and well organised; try a larger model if you have the RAM; treat the score as a first sort and always read adverts yourself. Never copy AI reasons into an application without checking them.

Open WebUI is slow, replies about "a list of functions" REAL

FIX Turn off tools, web search, code interpreter and background generation features (Chapter 12). Check `task.n_tokens` in the Ollama log falls to about 30 for "hi".

Open WebUI: "This model is not publicly available" REAL

FIX Set the Task Model back to "Current Model" in Admin Panel > Settings > Interface, or make the model public in Admin Panel > Models.

Telegram

No Telegram messages arrive

DIAGNOSE Log line *Telegram: token or chat ID missing* or *send failed (HTTPError)*. Run `check_setup.py --send`.

WHY / FIX Token wrong or revoked (copy again); chat ID wrong (run `get_chat_id.py`); you never pressed Start in the bot chat (do it); you blocked the bot (unblock it); `.env` not reloaded (`--force-recreate`). Also check the run actually finished: the message is sent at the end.

VERIFY The test message arrives.

get_chat_id.py: "409 Conflict" or finds nothing

WHY 409: another program is already receiving the bot's messages, for example an n8n workflow with a Telegram Trigger, or a webhook. Nothing found: no recent message to the bot.

FIX Deactivate the other workflow (or remove the webhook by opening `https://api.telegram.org/botYOUR_TELEGRAM_BOT_TOKEN/deleteWebhook` in a private browser window), send "hello" to the bot again, retry.

Resources and other messages

Out of memory (RAM)

LOOKS LIKE Ollama restarts during scoring; errors like *model requires more system memory*; everything very slow; `free -h` shows almost no "available" memory and swap in use.

FIX Use a smaller model; stop optional apps (`docker stop open-webui dockge n8n`); lower `OLLAMA_KEEP_ALIVE` ; give the container more memory (host: container > Resources > Memory) if the host has spare; see Chapter 3's low-memory settings.

VERIFY `free -h` shows free "available" memory during a run; `docker stats --no-stream` shows each app's usage.

Disk full ("No space left on device")

DIAGNOSE `df -h` in the container and on the host; `docker system df` shows what Docker uses.

FIX Remove unused models (`ollama rm`); remove old images: `docker image prune -a` (only removes images no container uses); delete old backups in Proxmox (local > Backups); truncate a huge log: `: > /opt/jobagent/state/run.log` . To enlarge the container disk: host > container > Resources > Root Disk > Volume Action > Resize.

VERIFY At least 15-20% free.

"perl: warning: Setting locale failed" REAL

WHY / FIX English locales are not generated. Harmless, but noisy. Run `apt install -y locales` , then `dpkg-reconfigure locales` and tick *en_GB.UTF-8* and *en_US.UTF-8* (install.sh does this automatically).

grep on "docker logs" finds nothing REAL

WHY / FIX Many apps write logs to the error stream. Add `2>&1` : `docker logs ollama 2>&1 | grep something` .

CHAPTER 16

Maintenance and backups

A few minutes a month keeps the system healthy. This chapter gives a simple routine and explains how to back up and restore.

A monthly routine

Task	How	Where
Update Proxmox	pve > Updates > Refresh > Upgrade (or `apt update && apt full-upgrade -y`); reboot if a new kernel was installed	Host
Update the container OS	`apt update && apt upgrade -y`	Container
Update the apps (Docker images)	For each stack: `cd /opt/stacks/ai && docker compose pull && docker compose up -d` (same for `jobs`, `/opt/dockge`); for the agent: `cd /opt/stacks/jobagent && docker compose build --pull && docker compose up -d`	Container
Clean old images	`docker image prune -f`	Container
Check disk space	`df -h` (host and container); `docker system df`	Both
Check memory and CPU	`free -h`; `docker stats --no-stream`; the host's Summary page graphs; optional: `apt install -y btop` then `btop`	Both
Check disk health	`smartctl -H /dev/sda` (add `-d sat` for USB)	Host
Skim the logs	`docker logs jobagent --since 168h 2>&1 | grep -iE "error|failed"`	Container
Check backups ran	Datacenter > Backup / local > Backups	Host

Before updating

Make a backup first (below). Open WebUI and NocoDB occasionally change their screens after updates; that is normal. Proxmox major version upgrades (e.g. 9 to 10) need the official upgrade guide; do not do them with a simple Upgrade click.

Updating the project code

When a newer package is released: copy the new zip into the container (Stage F), unzip it to a new folder and run `bash install.sh` again. It replaces `finder.py`, the tools and the compose files, but **keeps** your `config.json`, `master.md`, `answers.yaml` and `.env`. Compare your `config.json` with the new example for any new settings, then `docker restart jobagent`.

Changing preferences or your CV safely

1. Copy the file first: `cp /opt/jobagent/app/config.json /opt/jobagent/app/config.json.bak` (same idea for `master.md`).
2. Edit with nano; for config.json, validate with `python3 -m json.tool`.
3. `docker restart jobagent`.
4. If something breaks, restore: `cp config.json.bak config.json` and restart.

After a big profile change, consider deleting `seen.json` so recent adverts are re-scored against the new profile.

Backups

What matters

Data	Location	Importance
Profile, answers, settings	`/opt/jobagent/cv`, `/opt/jobagent/app/config.json`	High: your work
Secrets	`/opt/stacks/jobagent/.env`	High, and sensitive
Tracker data	Docker volume `jobs_nocodb`	High: your application history
Processed-advert memory	`/opt/jobagent/state/seen.json`	Low (can be rebuilt)
AI models	Docker volume `ai_ollama`	Low (can be downloaded again)

Method 1: whole-container backups in Proxmox (recommended)

Proxmox can save a complete copy of the container (all apps, data and settings) with a tool called **vzdump**.

1. Once now: select the container > **Backup** > **Backup now**; Storage *local*, Mode *Snapshot*, Compression *ZSTD*.
2. Automatically: **Datacenter** > **Backup** > **Add**; select the container, a schedule (e.g. Sunday 03:00, a time that does not overlap a run), storage, mode Snapshot; on the **Retention** tab keep e.g. the last 4.

Command-line equivalent:

PROXMOX HOST

```
vzdump 100 --mode snapshot --compress zstd --storage local
```

> **A backup on the same disk is not a real backup**
> If that disk dies (as USB disks do), the backups go with it. Regularly copy the files from `/var/lib/vz/dump/` to another drive or computer, for example with `scp root@YOUR_SERVER_IP:/var/lib/vz/dump/vzdump-lxc-100-*.tar.zst .` from your PC. A dedicated **Proxmox Backup Server** on another machine is the professional option.

Method 2: just your files

CONTAINER

```
tar czf /root/jobagent-files-$(date +%F).tar.gz /opt/jobagent /opt/stacks
docker run --rm -v jobs_nocodb:/v -v /root:/b alpine tar czf /b/nocodb-$(date +%F).tar.gz -
C /v .
```

Then copy the two files off the server (`pct pull 100 /root/FILE /root/FILE` on the host, then `scp` to your PC). These archives contain your `.env` secrets and personal data: store them privately (an encrypted drive or an encrypted archive).

Volume names are the stack folder name plus the volume name (`jobs_nocodb`, `ai_ollama`); check with `docker volume ls`.

Restoring

- **Whole container**: local > Backups > select the file > **Restore**. Restoring over the existing CT 100 replaces it; you can instead restore to a new ID to compare first. On a freshly reinstalled Proxmox, copy the backup file into `/var/lib/vz/dump/` first, recreate the Option B bridge if you used it, then restore.
- **Files only**: on a fresh install (after Stage F), stop the agent, unpack the archive over `/` with `tar xzf FILE -C /`, restore the NocoDB volume, and start everything with `docker compose up -d` in each stack:

  CONTAINER

  ```
  cd /opt/stacks/jobs && docker compose stop nocodb
  docker run --rm -v jobs_nocodb:/v -v /root:/b alpine tar xzf /b/nocodb-YYYY-MM-
  DD.tar.gz -C /v
  docker compose up -d
  ```

NOT VERIFIED: restore procedures were not exercised on the original system. Test a restore once to a spare container ID, so you know it works before you need it.

Recovering from a dead disk or failed server

1. Replace the disk (ideally with an internal SSD).
2. Reinstall Proxmox (Stage C) with the same IP if possible.
3. Recreate the network bridge if you used Option B.
4. Restore the container backup (or rebuild with Stages D-F and restore your files).
5. Run `check_setup.py`.

CHAPTER 17

Security and privacy

Your server holds your CV, your job history and keys to several accounts. The design keeps it private by default; these rules keep it that way.

The security design in one paragraph

Nothing on the server is reachable from the public internet: you never open ports on your router. On your home network, only the app pages you need are reachable, and the AI engine (Ollama, which has no password) listens only inside the container. Remote access goes through Tailscale, which is encrypted and limited to your own signed-in devices. All secrets live in a single file readable only by root. The AI runs locally, so your CV is not sent to an AI company. Outgoing connections go only to the job boards, Telegram and software update servers.

Where secrets live

Secret	Stored in	If it leaks
Telegram bot token	`/opt/stacks/jobagent/.env`	BotFather: `/revoke`, choose the bot, update `.env`
NocoDB API token	`.env`	NocoDB > Account Settings > Tokens: delete it, create a new one
Reed API key	`.env`	Request a new key from Reed's developer page and stop using the old one
Adzuna ID/key	`.env`	Adzuna developer dashboard: regenerate the key
Proxmox root password	Your head / password manager	Change it: host shell `passwd`
Container root password	Password manager	Container: `passwd`
NocoDB, Open WebUI, Dockge admin passwords	Password manager	Change in each app's account settings
Credentials saved inside n8n (if used)	n8n's own encrypted store	Delete and recreate them in n8n

After rotating any key: update `.env`, run `docker compose up -d --force-recreate` in `/opt/stacks/jobagent`, and run `check_setup.py`.

> ★ **Real experience: keys pasted into a chat**
>
> While building the original system, the Telegram token, NocoDB token and API keys were pasted into a chat conversation to get help. Anything you paste into a website, chat (including AI assistants), forum or support ticket should be treated as exposed. All of those keys had to be rotated. When asking for help, replace secrets with `YOUR_...` placeholders first.

Ten rules

1. **Never expose ports to the internet.** Do not set up router port forwarding to the server. Use Tailscale.
2. **Keep `.env` at mode 600.** Check with `ls -l /opt/stacks/jobagent/.env`: it must start with `-rw-------`. Fix with `chmod 600`.
3. **Never share `.env`**, screenshots of it, or terminal output that contains keys. Never put it in Git or cloud folders.
4. **Use strong, unique passwords** for Proxmox and every app; consider two-factor login for Proxmox (Datacenter > Permissions > Two Factor).
5. **Keep software updated** (Chapter 16). Most attacks use old, known holes.
6. **Keep Ollama private.** The package binds it to `127.0.0.1`; do not change that to `0.0.0.0` on the server.
7. **n8n's `N8N_SECURE_COOKIE=false`** allows logging in over plain http. It is acceptable on a private network or Tailscale only, never on a public address.
8. **The bot talks to one chat.** It only sends to your chat ID. Keep group joining disabled. Any future feature that reacts to Telegram messages must check the sender's chat ID and ignore everyone else.
9. **Backups contain secrets and personal data.** Store them privately and encrypted.
10. **Before publishing anything** (screenshots, blog posts, a fork of this project), search it for your name, email, chat ID, IP addresses, tokens and keys, and replace them with placeholders. Blurring is not enough in PDFs: remove or recreate the content.

Your personal data

- `master.md` and `answers.yaml` contain personal information. They stay on your server and are only read by your local AI and (in future) your own CV generator.
- Job titles and company names from adverts are sent to Telegram in your summary. Telegram stores messages on its servers; if that matters to you, keep the summary minimal (lower `telegram_top_n`).
- If you run the system on a rented VPS or point `OLLAMA_URL` at a computer you do not control, your profile goes there. Choose accordingly.
- Deleting everything: in Proxmox, remove the container (and its backups). That deletes all data, models and secrets in one go. Then revoke the bot token and keys.

CHAPTER 18

Responsible use

An automated job-search tool is only worth having if it helps you without harming employers, job boards or your own reputation. These principles were decided before any automation was written, and they apply to the current system and to every future part.

Principles

1. **Your own accounts and information only.** Use the system for your own job search, with your own keys and your own true profile.
2. **Follow the job platforms' terms.** The current system uses Reed's and Adzuna's official APIs, which exist for this purpose. Read and respect their terms, and do not use the data for anything else (for example, republishing adverts).
3. **Respect rate limits.** Twice-daily searches with a handful of keywords are a light load. Do not set run times every few minutes or add dozens of keywords.
4. **No scraping or bots where they are banned.** LinkedIn and Indeed prohibit automated applications and scraping; this project does not and will not automate them. Large applicant-tracking portals such as Workday, Taleo and SuccessFactors are also excluded.
5. **Never bypass CAPTCHAs or security checks.** A CAPTCHA means "a human must do this". The system must stop and hand over to you.
6. **Verify that adverts are genuine.** Be wary of jobs that ask for money, personal documents before an interview, or moving to a private messaging app.
7. **No fabrication.** The AI must never invent qualifications, skills, experience, dates or answers. The profile and fixed answers are written by you, truthfully.
8. **Explicit approval for every application.** No application may be submitted without you reviewing and approving that specific application. Today you apply manually; future automation must keep an "approve first" mode.
9. **Review before submission.** Read the final CV and application yourself every time. You are responsible for what is sent in your name.
10. **Store personal data securely** (Chapter 17).

> ? **A note on fairness to employers**
>
> Mass-applying to everything that scores above a threshold wastes recruiters' time and rarely works. Use the scores to find the roles you are genuinely suited to, then write a careful application for each.

CHAPTER 19

Roadmap: what comes next

Everything in this chapter is `PLANNED`. None of it exists in the package yet. It is included so you can see the intended direction and the rules any implementation must follow.

Infrastructure improvements

1. **Make the USB autosuspend fix permanent and verified** (Chapter 6), check disk health with `smartctl`, and **replace a USB hard disk with an SSD** (500 GB internal or USB SSD).
2. **Set up real backups**: scheduled vzdump plus copies to another machine, or a Proxmox Backup Server.
3. **Install Tailscale on the Proxmox host** for remote access to the web interface.
4. **Optional battery care**: charge thresholds of about 75-80% with TLP on supported laptops.

Part 2: CV and cover-letter tailor

For each match above the threshold: generate a job-specific CV and cover letter **using only facts from `master.md`** (select, reorder, rephrase; never add), render them from HTML to PDF with the already-installed **Gotenberg** service, save them in `/opt/jobagent/output`, update the NocoDB row (Status *Tailored*, *CV File*), and send the PDF to you on Telegram for checking.

Part 3: Telegram approve / skip buttons

Each tailored application arrives with **Approve** and **Skip** buttons. Every button press must be checked against your chat ID; presses from anyone else are ignored. Approve sets Status *Approved*; Skip sets *Skipped*.

Part 4: apply robot with guardrails

A browser-automation robot (Playwright) that applies only after approval, with these fixed rules:

- Only email applications and Greenhouse, Lever and Ashby forms. No LinkedIn or Indeed bots, no Workday, Taleo or SuccessFactors.
- Form answers come only from `answers.yaml`; never AI-generated; never invented experience.
- A daily limit (10 by default).
- A screenshot of every submitted form as proof, attached to the NocoDB row.
- Any unknown question, CAPTCHA or unusual page: stop and ask on Telegram.
- Start in "approve first" mode, and stay there.

Other ideas

- Optional cloud AI provider in `score_job` for faster scoring (with the privacy trade-off explained in Chapter 3).

- More job sources with official APIs.
- A weekly summary of applications and responses from NocoDB.

CHAPTER 20

The download package

Everything needed to recreate the system is in one zip file, `ai-job-agent.zip`, distributed with this handbook. It contains no credentials: only templates with placeholders.

Contents

AI-JOB-AGENT.ZIP

```
ai-job-agent/
├── README.md                  short version of this handbook
├── install.sh                 installer, run inside the container (Stage F)
├── host/
│   └── setup-nat-bridge.sh    optional Network Option B, run on the host (Stage D)
├── dockge/compose.yaml        optional Docker web manager
├── stacks/
│   ├── ai/compose.yaml        Ollama + Open WebUI
│   ├── jobs/compose.yaml      NocoDB (+ optional n8n, Gotenberg)
│   └── jobagent/
│       ├── Dockerfile         Python 3.12 image for the Job Finder
│       ├── compose.yaml
│       └── .env.example       secrets template (copy to .env)
└── jobagent/
    ├── app/finder.py          the Job Finder
    ├── app/config.json        example search settings
    ├── cv/master.md           profile template
    ├── cv/answers.yaml        fixed answers template
    ├── tools/get_chat_id.py   finds your Telegram chat ID
    ├── tools/check_setup.py   tests all connections
    ├── state/                 (empty; seen.json and run.log appear here)
    └── output/                (empty; reserved for planned CV PDFs)
```

How to use it

1. Download `ai-job-agent.zip` to your PC.
2. Copy it to the Proxmox host (Stage D, Step 1) and into the container (Stage F).
3. Unzip it and run `bash install.sh` (Stage F).
4. Fill in `.env`, `master.md` and `config.json` (Stages F-J).

Differences from the original private build

For transparency, these are the changes made when preparing the public package:

- `finder.py` was rebuilt from the original design and fixes (same flow, settings, field names, prompt structure and patch). It adds an optional `location` / `distance_miles` setting (empty by default = same behaviour as the original), applies `max_days_old` to Reed results as well, re-checks skip phrases on Reed's full advert text, and allows the API addresses to be overridden for testing.

- Two helper tools were added: `get_chat_id.py` and `check_setup.py`. An installer script and the optional NAT-bridge script were added.
- Ollama's port is published on `127.0.0.1` only (the original published it on all interfaces, which was safe there only because of the NAT setup).
- n8n and Gotenberg are optional extras (Compose profile `extras`) instead of always running, to save memory.
- All personal values (profile, answers, chat ID, addresses, bot name, keys) were replaced with templates and placeholders.

The complete source of every file is printed in Appendix A, so you can check exactly what you are running.

APPENDICES

Reference

Complete source code, a command cheat sheet, service and file reference, glossary, the final checklist, useful links and an index.

APPENDIX A

Complete source files

These are the exact contents of every file in the download package, printed automatically from the package itself. Long lines wrap in print; the files in the zip are the copy-and-paste source.

A.1 Installer (run inside the container)

Package path: `install.sh` · 196 lines

INSTALL.SH

```
#!/usr/bin/env bash
# ==============================================================================
#  AI Job Agent - installer
#
#  Run this INSIDE your Debian/Ubuntu container or virtual machine (NOT on the
#  Proxmox host), as root, from the folder you unzipped:
#
#      cd ai-job-agent
#      bash install.sh
#
#  What it changes (and nothing else):
#    1. Installs: curl, ca-certificates, locales, tzdata, nano (apt).
#    2. Sets the time zone (default Europe/London) and generates English locales.
#    3. Installs Docker Engine + Compose plugin from get.docker.com (if missing).
#    4. Creates /opt/stacks/{ai,jobs,jobagent}, /opt/dockge and /opt/jobagent
#       and copies the project files there. Existing config.json, master.md,
#       answers.yaml and .env files are NEVER overwritten.
#    5. Creates /opt/stacks/jobagent/.env from .env.example (mode 600).
#    6. Starts Ollama (+ Open WebUI), NocoDB and Dockge, and downloads the AI model.
#    7. Builds the Job Finder container. It is only started when .env has been
#       filled in (otherwise you are told what to do next).
#
#  Options:
#    --minimal        skip Open WebUI and Dockge (recommended for 4-8 GB RAM)
#    --model NAME     AI model to download (default: qwen2.5:3b)
#    --no-model       do not download a model now
#    --tz ZONE        time zone, e.g. Europe/London (default)
#    -y               do not ask for confirmation
# ==============================================================================
set -euo pipefail

MODEL="qwen2.5:3b"
TZONE="Europe/London"
MINIMAL=0
PULL_MODEL=1
ASSUME_YES=0
SRC="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

while [ $# -gt 0 ]; do
  case "$1" in
    --minimal)  MINIMAL=1 ;;
    --model)    MODEL="${2:?--model needs a name}"; shift ;;
    --no-model) PULL_MODEL=0 ;;
    --tz)       TZONE="${2:?--tz needs a zone}"; shift ;;
    -y|--yes)   ASSUME_YES=1 ;;
    -h|--help)  sed -n '2,32p' "$0"; exit 0 ;;
    *) echo "Unknown option: $1 (use --help)"; exit 1 ;;
  esac
  shift
done

say()  { printf '\n\033[1;36m==> %s\033[0m\n' "$*"; }
```

```
ok()   { printf '    \033[32mOK\033[0m %s\n' "$*"; }
warn() { printf '    \033[33m!!\033[0m %s\n' "$*"; }
die()  { printf '\n\033[31mERROR:\033[0m %s\n' "$*"; exit 1; }

# ------------------------------------------------------------------ safety checks
[ "$(id -u)" -eq 0 ] || die "Please run as root (in a Proxmox container you already are root)."
[ -f "$SRC/stacks/jobagent/compose.yaml" ] || die "Run this from the unzipped ai-job-agent folder."
if command -v pveversion >/dev/null 2>&1; then
  die "This looks like the Proxmox HOST. Run the installer inside the container: pct enter <ID>"
fi
. /etc/os-release 2>/dev/null || true
case "${ID:-}" in
  debian|ubuntu) ok "Operating system: ${PRETTY_NAME:-$ID}" ;;
  *) warn "Untested operating system '${ID:-unknown}'. Debian 12/13 or Ubuntu 22.04/24.04
recommended." ;;
esac

MEM_MB=$(awk '/MemTotal/ {print int($2/1024)}' /proc/meminfo)
DISK_GB=$(df -Pm / | awk 'NR==2 {print int($4/1024)}')
ok "Memory: ${MEM_MB} MB, free disk: ${DISK_GB} GB"
[ "$MEM_MB" -lt 3500 ] && warn "Less than 4 GB RAM: use --minimal and a 1.5B model (see handbook,
chapter 3)."
[ "$DISK_GB" -lt 15 ] && warn "Less than 15 GB free disk: models and images may not fit."

echo
echo "This will install Docker and the AI Job Agent into /opt on THIS machine."
echo "Model: $MODEL   Time zone: $TZONE   Minimal: $([ $MINIMAL = 1 ] && echo yes || echo no)"
if [ "$ASSUME_YES" -ne 1 ]; then
  read -r -p "Continue? [y/N] " answer
  case "$answer" in y|Y|yes|YES) ;; *) echo "Cancelled."; exit 0 ;; esac
fi

# ------------------------------------------------------------------ 1-2 base system
say "Installing base packages"
export DEBIAN_FRONTEND=noninteractive
apt-get update -qq
apt-get install -y -qq curl ca-certificates locales tzdata nano >/dev/null
ok "curl, ca-certificates, locales, tzdata, nano"

say "Setting time zone and locales"
ln -sf "/usr/share/zoneinfo/$TZONE" /etc/localtime && echo "$TZONE" > /etc/timezone
sed -i 's/^# *\(en_GB.UTF-8 UTF-8\)/\1/; s/^# *\(en_US.UTF-8 UTF-8\)/\1/' /etc/locale.gen
locale-gen >/dev/null 2>&1 || true
ok "Time zone $TZONE, locales en_GB/en_US generated"

# ------------------------------------------------------------------ 3 docker
say "Installing Docker"
if command -v docker >/dev/null 2>&1 && docker compose version >/dev/null 2>&1; then
  ok "Docker already installed: $(docker --version)"
else
  curl -fsSL https://get.docker.com -o /tmp/get-docker.sh
  sh /tmp/get-docker.sh >/dev/null
  rm -f /tmp/get-docker.sh
  ok "$(docker --version)"
fi
systemctl enable --now docker >/dev/null 2>&1 || true
docker info >/dev/null 2>&1 || die "Docker is installed but not running. In Proxmox, enable
'nesting' for this container (handbook, Stage D)."

# ------------------------------------------------------------------ 4 files
say "Copying project files"
copy_new() {  # copy_new SOURCE DEST : copy only if DEST does not exist yet
  if [ -e "$2" ]; then warn "kept your existing $2"; else cp "$1" "$2"; ok "$2"; fi
}
mkdir -p /opt/stacks/ai /opt/stacks/jobs /opt/stacks/jobagent /opt/dockge \
         /opt/jobagent/app /opt/jobagent/cv /opt/jobagent/state /opt/jobagent/output
/opt/jobagent/tools

cp "$SRC/stacks/ai/compose.yaml"         /opt/stacks/ai/compose.yaml
```

```
cp "$SRC/stacks/jobs/compose.yaml"       /opt/stacks/jobs/compose.yaml
cp "$SRC/stacks/jobagent/compose.yaml"   /opt/stacks/jobagent/compose.yaml
cp "$SRC/stacks/jobagent/Dockerfile"     /opt/stacks/jobagent/Dockerfile
cp "$SRC/stacks/jobagent/.env.example"   /opt/stacks/jobagent/.env.example
cp "$SRC/dockge/compose.yaml"            /opt/dockge/compose.yaml
cp "$SRC/jobagent/app/finder.py"         /opt/jobagent/app/finder.py
cp "$SRC"/jobagent/tools/*.py            /opt/jobagent/tools/
ok "program files and compose files"
copy_new "$SRC/jobagent/app/config.json" /opt/jobagent/app/config.json
copy_new "$SRC/jobagent/cv/master.md"    /opt/jobagent/cv/master.md
copy_new "$SRC/jobagent/cv/answers.yaml" /opt/jobagent/cv/answers.yaml

# ------------------------------------------------------------------ 5 .env
ENV_FILE=/opt/stacks/jobagent/.env
copy_new /opt/stacks/jobagent/.env.example "$ENV_FILE"
chmod 600 "$ENV_FILE"
chown -R 1000:1000 /opt/jobagent   # uid 1000 = the user inside the optional n8n container
ok ".env permissions set to 600 (only root can read it)"

# ------------------------------------------------------------------ 6 services
say "Starting the AI stack"
cd /opt/stacks/ai
if [ "$MINIMAL" -eq 1 ]; then docker compose up -d ollama; else docker compose up -d; fi
ok "Ollama running$([ $MINIMAL = 1 ] || echo ', Open WebUI on port 3000')"

say "Starting the tracker (NocoDB)"
cd /opt/stacks/jobs && docker compose up -d
ok "NocoDB on port 8081"

if [ "$MINIMAL" -eq 0 ]; then
  say "Starting Dockge"
  cd /opt/dockge && docker compose up -d
  ok "Dockge on port 5001"
fi

if [ "$PULL_MODEL" -eq 1 ]; then
  say "Downloading AI model $MODEL (this can take 5-30 minutes)"
  for i in 1 2 3 4 5 6; do docker exec ollama ollama list >/dev/null 2>&1 && break; sleep 5; done
  docker exec ollama ollama pull "$MODEL"
  ok "Model $MODEL ready"
  if [ "$MODEL" != "qwen2.5:3b" ]; then
    sed -i "s|^SCORING_MODEL=.*|SCORING_MODEL=$MODEL|" "$ENV_FILE"
    ok "SCORING_MODEL set to $MODEL in .env"
  fi
fi

# ------------------------------------------------------------------ 7 job finder
say "Building the Job Finder container"
cd /opt/stacks/jobagent && docker compose build -q
ok "Image built"

MISSING=$(grep -E
'^(REED_API_KEY|TELEGRAM_TOKEN|TELEGRAM_CHAT_ID|NOCODB_TOKEN|NOCODB_TABLE_ID)=.*YOUR_' "$ENV_FILE" |
cut -d= -f1 | tr '\n' ' ' || true)
IP=$(hostname -I 2>/dev/null | awk '{print $1}')
if [ -z "$MISSING" ]; then
  docker compose up -d
  ok "Job Finder started (scheduler mode)"
else
  warn "Job Finder NOT started yet. Still to fill in $ENV_FILE: $MISSING"
fi

cat <<EOF

========================================================================
 Installation finished.
   NocoDB tracker : http://${IP:-YOUR_SERVER_IP}:8081
$( [ "$MINIMAL" -eq 0 ] && echo "   Open WebUI     : http://${IP:-YOUR_SERVER_IP}:3000" )
$( [ "$MINIMAL" -eq 0 ] && echo "   Dockge         : http://${IP:-YOUR_SERVER_IP}:5001" )
```

```
 (With the NAT network option, use the Proxmox host's address instead.)

 Next steps (handbook, Stage F-J):
   1. Create the NocoDB table and API token          (Stage F2)
   2. Create the Telegram bot                         (Stage G)
   3. Edit the secrets:   nano $ENV_FILE
   4. Edit your profile:  nano /opt/jobagent/cv/master.md
   5. Start / restart:    cd /opt/stacks/jobagent && docker compose up -d --force-recreate
   6. Check everything:   docker exec jobagent python /data/tools/check_setup.py --send
======================================================================
EOF
```

A.2 Optional NAT bridge script (run on the Proxmox host)

Package path: `host/setup-nat-bridge.sh` · 74 lines

HOST/SETUP-NAT-BRIDGE.SH

```
#!/usr/bin/env bash
# ============================================================================
#  OPTIONAL - Network Option B: private NAT bridge on the PROXMOX HOST.
#
#  Only needed on networks that block or "register" every new device
#  (many student halls, hotels, some workplaces). On a normal home router
#  you do NOT need this: use Network Option A (vmbr0 + DHCP) instead.
#
#  Run on the Proxmox HOST shell (prompt "root@<host>"), NOT inside a container:
#      bash setup-nat-bridge.sh
#
#  What it changes:
#    1. Backs up /etc/network/interfaces to /etc/network/interfaces.bak-<date>.
#    2. Appends a bridge "vmbr1" (10.10.10.1/24, no physical port) with:
#         - NAT (MASQUERADE) so containers on 10.10.10.0/24 reach the internet
#           through the host's own address,
#         - port forwarding of TCP 3000,5001,5678,8081 to 10.10.10.100.
#    3. Enables IP forwarding (/etc/sysctl.d/99-forward.conf).
#    4. Applies the network config with "ifreload -a".
#  To undo: copy the backup back over /etc/network/interfaces and reboot.
# ============================================================================
set -euo pipefail

IFACES="${IFACES:-/etc/network/interfaces}"
SYSCTL_FILE="${SYSCTL_FILE:-/etc/sysctl.d/99-forward.conf}"
SUBNET="${SUBNET:-10.10.10}"          # gives 10.10.10.1 (host) and 10.10.10.100 (container)
CT_IP="${CT_IP:-${SUBNET}.100}"
PORTS="${PORTS:-3000,5001,5678,8081}"
UPLINK="${UPLINK:-vmbr0}"
APPLY="${APPLY:-1}"                   # APPLY=0 writes files only (for testing)

die() { echo "ERROR: $*"; exit 1; }

[ "$(id -u)" -eq 0 ] || die "run as root"
if [ "$APPLY" = 1 ]; then
  command -v pveversion >/dev/null 2>&1 || die "this is not a Proxmox host. Run it on the host
shell, not inside a container."
fi
[ -f "$IFACES" ] || die "$IFACES not found"
grep -q "iface $UPLINK " "$IFACES" || die "uplink bridge $UPLINK not found in $IFACES"
if grep -q "^auto vmbr1" "$IFACES" || grep -q "iface vmbr1 " "$IFACES"; then
  die "vmbr1 already exists in $IFACES - nothing changed"
fi

echo "About to add vmbr1 (${SUBNET}.1/24) with NAT via $UPLINK and forward ports $PORTS to $CT_IP."
read -r -p "Continue? [y/N] " a
case "$a" in y|Y|yes) ;; *) echo "Cancelled."; exit 0 ;; esac

BACKUP="${IFACES}.bak-$(date +%Y%m%d-%H%M%S)"
cp "$IFACES" "$BACKUP"
```

```
echo "Backup saved: $BACKUP"

cat >> "$IFACES" <<EOF

auto vmbr1
iface vmbr1 inet static
        address ${SUBNET}.1/24
        bridge-ports none
        bridge-stp off
        bridge-fd 0
        post-up   iptables -t nat -A POSTROUTING -s ${SUBNET}.0/24 -o ${UPLINK} -j MASQUERADE
        post-down iptables -t nat -D POSTROUTING -s ${SUBNET}.0/24 -o ${UPLINK} -j MASQUERADE
        post-up   iptables -t nat -A PREROUTING -i ${UPLINK} -p tcp -m multiport --dports ${PORTS} -
j DNAT --to-destination ${CT_IP}
        post-down iptables -t nat -D PREROUTING -i ${UPLINK} -p tcp -m multiport --dports ${PORTS} -
j DNAT --to-destination ${CT_IP}
EOF
echo "net.ipv4.ip_forward=1" > "$SYSCTL_FILE"

if [ "$APPLY" = 1 ]; then
  sysctl -p "$SYSCTL_FILE"
  ifreload -a
  echo "Done. Check with:  ip addr show vmbr1   and   iptables -t nat -L -n"
else
  echo "APPLY=0: files written, nothing applied."
fi
```

A.3 AI stack: /opt/stacks/ai/compose.yaml

Package path: `stacks/ai/compose.yaml` · 40 lines

STACKS/AI/COMPOSE.YAML

```
# AI stack: Ollama (runs the AI models) + Open WebUI (optional chat website).
# Start with:  cd /opt/stacks/ai && docker compose up -d
services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    restart: unless-stopped
    ports:
      - "127.0.0.1:11434:11434"   # only reachable from this server (Ollama has no password)
    environment:
      - OLLAMA_KEEP_ALIVE=24h       # keep the model loaded in RAM (no reload delay)
      - OLLAMA_NUM_PARALLEL=1       # one request at a time: best for weak CPUs
    volumes:
      - ollama:/root/.ollama        # downloaded models are stored here
    networks:
      - shared

  open-webui:                       # optional: delete this block on 4 GB machines
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    restart: unless-stopped
    ports:
      - "3000:8080"
    environment:
      - OLLAMA_BASE_URL=http://ollama:11434
    volumes:
      - openwebui:/app/backend/data
    depends_on:
      - ollama
    networks:
      - shared

volumes:
  ollama:
  openwebui:
```

```
networks:
  shared:
    name: shared                    # other stacks join this network by name
```

A.4 Jobs stack: /opt/stacks/jobs/compose.yaml

Package path: `stacks/jobs/compose.yaml` · 52 lines

STACKS/JOBS/COMPOSE.YAML

```
# Jobs stack: NocoDB (job tracker, required).
# n8n and Gotenberg are optional extras for future parts of the project:
#   start only NocoDB:        docker compose up -d
#   start NocoDB + extras:    docker compose --profile extras up -d
services:
  nocodb:
    image: nocodb/nocodb:latest
    container_name: nocodb
    restart: unless-stopped
    ports:
      - "8081:8080"
    volumes:
      - nocodb:/usr/app/data
    networks:
      - shared

  n8n:                              # optional (profile "extras")
    image: docker.n8n.io/n8nio/n8n
    container_name: n8n
    profiles: ["extras"]
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - GENERIC_TIMEZONE=Europe/London
      - TZ=Europe/London
      - N8N_SECURE_COOKIE=false     # allows plain http on a private network only
      - N8N_RUNNERS_ENABLED=true
      - WEBHOOK_URL=${N8N_WEBHOOK_URL:-http://localhost:5678/}
      - NODE_FUNCTION_ALLOW_BUILTIN=*
    volumes:
      - n8n:/home/node/.n8n
      - /opt/jobagent:/data
    networks:
      - shared

  gotenberg:                        # optional (profile "extras"): HTML -> PDF, internal only
    image: gotenberg/gotenberg:8
    container_name: gotenberg
    profiles: ["extras"]
    restart: unless-stopped
    networks:
      - shared

volumes:
  nocodb:
  n8n:

networks:
  shared:
    external: true
```

A.5 Dockge: /opt/dockge/compose.yaml

Package path: `dockge/compose.yaml` · 16 lines

DOCKGE/COMPOSE.YAML

```
# Dockge: a web page for starting, stopping and editing the Docker stacks.
# Optional. Start with:  cd /opt/dockge && docker compose up -d
services:
  dockge:
    image: louislam/dockge:1
    container_name: dockge
    restart: unless-stopped
    ports:
      - "5001:5001"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - ./data:/app/data
      - /opt/stacks:/opt/stacks
    environment:
      - DOCKGE_STACKS_DIR=/opt/stacks
```

A.6 Job Finder image: /opt/stacks/jobagent/Dockerfile

Package path: `stacks/jobagent/Dockerfile` · 15 lines

STACKS/JOBAGENT/DOCKERFILE

```
# Small image that runs the Job Finder (finder.py).
# The program itself is NOT copied into the image: it lives in /opt/jobagent
# on the server and is mounted at /data, so you can edit it without rebuilding.
FROM python:3.12-slim

RUN apt-get update \
 && apt-get install -y --no-install-recommends tzdata \
 && rm -rf /var/lib/apt/lists/* \
 && pip install --no-cache-dir requests

ENV TZ=Europe/London \
    PYTHONUNBUFFERED=1

CMD ["python", "/data/app/finder.py"]
```

A.7 Job Finder stack: /opt/stacks/jobagent/compose.yaml

Package path: `stacks/jobagent/compose.yaml` · 16 lines

STACKS/JOBAGENT/COMPOSE.YAML

```
# Job Finder stack. Start with:  cd /opt/stacks/jobagent && docker compose up -d --build
services:
  jobagent:
    build: .
    container_name: jobagent
    restart: unless-stopped
    env_file: .env              # all secrets live here (chmod 600)
    volumes:
      - /opt/jobagent:/data     # app, cv, state and output folders
    networks:
      - shared

networks:
  shared:
    external: true              # created by the AI stack (/opt/stacks/ai)
```

A.8 Secrets template: /opt/stacks/jobagent/.env.example

Package path: `stacks/jobagent/.env.example` · 38 lines

STACKS/JOBAGENT/.ENV.EXAMPLE

```
# ====================================================================
#  AI Job Agent - secrets and settings
#  1. Copy this file:   cp .env.example .env
#  2. Protect it:       chmod 600 .env
#  3. Replace every YOUR_... value. Do not use quotes or spaces around "=".
#  4. Apply changes:    docker compose up -d --force-recreate
#  NEVER share, screenshot, upload or commit the real .env file.
# ====================================================================

# --- Job sources ------------------------------------------------------
# Reed: https://www.reed.co.uk/developers/jobseeker  (free key by email)
REED_API_KEY=YOUR_REED_API_KEY

# Adzuna: https://developer.adzuna.com  (optional; leave as-is to skip)
# APP_ID is the SHORT id (8 characters); APP_KEY is the long key.
ADZUNA_APP_ID=YOUR_ADZUNA_APP_ID
ADZUNA_APP_KEY=YOUR_ADZUNA_APP_KEY

# --- Telegram ---------------------------------------------------------
# Token from @BotFather; chat ID from tools/get_chat_id.py
TELEGRAM_TOKEN=YOUR_TELEGRAM_BOT_TOKEN
TELEGRAM_CHAT_ID=YOUR_TELEGRAM_CHAT_ID

# --- NocoDB tracker ---------------------------------------------------
# "nocodb" is the container name on the shared Docker network: keep it.
NOCODB_URL=http://nocodb:8080
NOCODB_TOKEN=YOUR_NOCODB_API_TOKEN
NOCODB_TABLE_ID=YOUR_NOCODB_TABLE_ID

# --- AI (Ollama) ------------------------------------------------------
# Keep http://ollama:11434 if Ollama runs in the AI stack on this server.
# To use Ollama on another, stronger computer: http://YOUR_OTHER_PC_IP:11434
OLLAMA_URL=http://ollama:11434
SCORING_MODEL=qwen2.5:3b

# --- Time zone (run_times in config.json use this clock) --------------
TZ=Europe/London
```

A.9 Search settings: /opt/jobagent/app/config.json

Package path: `jobagent/app/config.json` · 36 lines

JOBAGENT/APP/CONFIG.JSON

```
{
  "run_times": ["07:30", "18:30"],
  "search_keywords": [
    "SOC Analyst",
    "Security Operations Analyst",
    "Cyber Security Analyst",
    "Information Security Analyst",
    "Graduate Cyber Security",
    "Junior Penetration Tester",
    "Digital Forensics",
    "IT Security",
    "Systems Administrator",
    "IT Support Analyst"
  ],
  "location": "",
  "distance_miles": 20,
  "title_must_contain_one_of": [
    "SOC", "Security", "Cyber", "Forensic", "Penetration", "Pen Test",
    "Analyst", "Systems Administrator", "Sysadmin", "IT Support", "Infrastructure"
  ],
  "avoid_title_keywords": [
    "Senior", "Sr", "Lead", "Principal", "Head", "Manager", "Director",
```

```
    "Architect", "Chief", "VP"
  ],
  "skip_if_description_contains": [
    "SC clearance", "DV clearance", "SC cleared", "DV cleared",
    "security clearance is required",
    "unable to sponsor", "cannot sponsor", "will not sponsor", "no sponsorship",
    "British citizens only", "UK nationals only"
  ],
  "max_days_old": 3,
  "max_jobs_to_score_per_run": 40,
  "min_match_score": 70,
  "telegram_top_n": 15
}
```

A.10 Profile template: /opt/jobagent/cv/master.md

Package path: `jobagent/cv/master.md` · 63 lines

JOBAGENT/CV/MASTER.MD

```
# MASTER PROFILE (example - replace every line with your own TRUE facts)

<!--
RULES FOR THIS FILE
- Write down every TRUE fact about your education, work, projects and skills.
- Be specific: tools, dates, numbers, results. More true detail = better scoring.
- The AI is only allowed to select, reorder and rephrase what is written here.
  It must never add anything. If it is not in this file, it is not true.
- Do not put passwords, ID numbers, bank details or your home address here.
- Keep it under roughly 1,500 words so it fits comfortably in the AI's memory.
-->

## Contact (used later for CVs; not sent anywhere by the Job Finder)
- Name: YOUR_FULL_NAME
- Email: YOUR_EMAIL
- Phone: YOUR_PHONE
- Location: YOUR_TOWN, United Kingdom (willing to relocate: yes/no)
- LinkedIn: YOUR_LINKEDIN_URL
- GitHub / portfolio: YOUR_PORTFOLIO_URL

## Target
- Roles: entry-level / graduate SOC analyst, cyber security analyst, IT security
- Contract: full-time, permanent
- Work pattern: on-site, hybrid or remote

## Education
- MSc Example Subject, Example University (2025 - 2026)
  - Modules: Network Security, Digital Forensics, Cloud Security
  - Dissertation: one sentence on what you did and what you found
- BSc Example Subject, Example College (2021 - 2025)

## Experience
### IT Support Technician - Example Company Ltd (Jan 2025 - Jun 2025)
- Supported around 50 users with Microsoft 365 accounts, laptops and printers
- Reset passwords and managed accounts in Active Directory
- Wrote how-to guides that reduced repeat tickets

### Part-time Retail Assistant - Example Shop (2022 - 2024)
- Handled cash and customer complaints; trained two new starters

## Projects
### Home lab (Proxmox, Docker, self-hosted AI)
- Built a Proxmox server on an old laptop; runs Docker containers in an LXC
- Self-hosted job-search agent: Python, REST APIs, local LLM (Ollama), Telegram bot
- Configured NAT, port forwarding and Tailscale VPN

### Example security project
- What it does, which tools you used, what you measured, what you learned
```

```
## Skills
- Security tools: Wireshark, Nmap, Burp Suite (only list tools you have really used)
- Operating systems: Windows, Linux (Debian, Kali)
- Scripting: Python, Bash, PowerShell
- Other: Docker, Git, SQL

## Certifications
- None yet / list real certifications with dates

## Other facts recruiters ask about
- Right to work in the UK: describe your real situation in one line
- Notice period: e.g. 1 week
- Driving licence: yes/no
```

A.11 Fixed answers template: /opt/jobagent/cv/answers.yaml

Package path: `jobagent/cv/answers.yaml` · 44 lines

JOBAGENT/CV/ANSWERS.YAML

```
# FIXED APPLICATION ANSWERS (example - replace with your own real answers)
#
# These answers are written by YOU, once, and are never generated by AI.
# The current Job Finder does not read this file. It is prepared for the
# planned "apply" part of the project, which must only ever copy these
# exact answers into application forms, after you approve each application.

personal:
  full_name: "YOUR_FULL_NAME"
  email: "YOUR_EMAIL"
  phone: "YOUR_PHONE"
  location: "YOUR_TOWN, United Kingdom"
  linkedin: "YOUR_LINKEDIN_URL"

eligibility:
  right_to_work_uk: "Yes"            # answer truthfully for your situation
  requires_sponsorship_now: "No"
  requires_sponsorship_future: "No"
  notes: ""                          # e.g. visa type and expiry, if relevant

availability:
  notice_period: "1 week"
  employment_type: "Full-time"
  work_pattern: ["On-site", "Hybrid", "Remote"]
  willing_to_relocate: "Yes, anywhere in the UK"

salary:
  expected_min_gbp: 25000
  expected_max_gbp: 30000

targets:
  job_titles:
    - "SOC Analyst"
    - "Cyber Security Analyst"
    - "IT Security Analyst"
  skip_if_advert_contains:
    - "SC clearance"
    - "DV clearance"

rules:
  min_match_score: 70
  daily_apply_limit: 10
  mode: "approve-first"   # every application needs your explicit approval
```

A.12 The Job Finder: /opt/jobagent/app/finder.py

Package path: `jobagent/app/finder.py` · 522 lines

JOBAGENT/APP/FINDER.PY

```
#!/usr/bin/env python3
"""
AI Job Agent - Part 1: Job Finder
=================================

What this program does, in order:

  1. Waits until one of the times listed in config.json -> "run_times"
     (or runs immediately when started with --once).
  2. Downloads fresh job adverts from the Reed and Adzuna job APIs.
  3. Removes duplicates and adverts it has already processed before.
  4. Throws away adverts that clearly do not fit (wrong title, too senior,
     or containing a "skip" phrase such as a security-clearance requirement).
  5. Asks a local AI model (through Ollama) to score every remaining advert
     against YOUR master profile (cv/master.md), from 0 to 100.
  6. Saves every advert that scores at or above "min_match_score" into the
     NocoDB tracker table with Status = "Found".
  7. Sends you a Telegram summary of the best matches.

It never applies for jobs and never changes your CV. Those are planned
future parts (see the handbook roadmap).

Run modes:
  python finder.py           -> scheduler mode (stays running, runs at run_times)
  python finder.py --once    -> one full run now, then exit

All secrets come from environment variables (the .env file), never from
this file.
"""

import datetime as dt
import html
import json
import os
import re
import sys
import time

import requests

# ---------------------------------------------------------------------------
# Paths (inside the container, /data is the host folder /opt/jobagent)
# ---------------------------------------------------------------------------
DATA_DIR = os.environ.get("DATA_DIR", "/data")
CONFIG_FILE = os.path.join(DATA_DIR, "app", "config.json")
MASTER_FILE = os.path.join(DATA_DIR, "cv", "master.md")
SEEN_FILE = os.path.join(DATA_DIR, "state", "seen.json")

# ---------------------------------------------------------------------------
# Settings from the environment (.env)
# ---------------------------------------------------------------------------
REED_API_KEY = os.environ.get("REED_API_KEY", "").strip()
ADZUNA_APP_ID = os.environ.get("ADZUNA_APP_ID", "").strip()
ADZUNA_APP_KEY = os.environ.get("ADZUNA_APP_KEY", "").strip()
TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN", "").strip()
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID", "").strip()
NOCODB_URL = os.environ.get("NOCODB_URL", "http://nocodb:8080").rstrip("/")
NOCODB_TOKEN = os.environ.get("NOCODB_TOKEN", "").strip()
NOCODB_TABLE_ID = os.environ.get("NOCODB_TABLE_ID", "").strip()
OLLAMA_URL = os.environ.get("OLLAMA_URL", "http://ollama:11434").rstrip("/")
SCORING_MODEL = os.environ.get("SCORING_MODEL", "qwen2.5:3b").strip()

# Base URLs can be overridden for testing; normal users never change these.
```

```
REED_BASE = os.environ.get("REED_BASE_URL", "https://www.reed.co.uk/api/1.0").rstrip("/")
ADZUNA_BASE = os.environ.get("ADZUNA_BASE_URL", "https://api.adzuna.com/v1/api").rstrip("/")
TELEGRAM_BASE = os.environ.get("TELEGRAM_API_BASE", "https://api.telegram.org").rstrip("/")

# AI scoring limits (explained in the handbook, chapter "AI configuration")
AI_TIMEOUT_SECONDS = 360      # give up on one advert after 6 minutes
AI_NUM_CTX = 8192             # context window: master profile + advert must fit
AI_NUM_PREDICT = 300          # max answer length; stops endless-whitespace bug
DESCRIPTION_LIMIT = 2500      # characters of the advert sent to the AI

HTTP_TIMEOUT = 30
PLACEHOLDER_MARK = "YOUR_"


def log(message):
    stamp = dt.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{stamp}] {message}", flush=True)


def is_set(value):
    return bool(value) and PLACEHOLDER_MARK not in value


# ---------------------------------------------------------------------------
# Files
# ---------------------------------------------------------------------------
def load_config():
    with open(CONFIG_FILE, encoding="utf-8") as f:
        cfg = json.load(f)
    cfg.setdefault("run_times", ["07:30", "18:30"])
    cfg.setdefault("search_keywords", [])
    cfg.setdefault("location", "")
    cfg.setdefault("distance_miles", 20)
    cfg.setdefault("title_must_contain_one_of", [])
    cfg.setdefault("avoid_title_keywords", [])
    cfg.setdefault("skip_if_description_contains", [])
    cfg.setdefault("max_days_old", 3)
    cfg.setdefault("max_jobs_to_score_per_run", 40)
    cfg.setdefault("min_match_score", 70)
    cfg.setdefault("telegram_top_n", 15)
    return cfg


def load_master():
    with open(MASTER_FILE, encoding="utf-8") as f:
        return f.read()


def load_seen():
    try:
        with open(SEEN_FILE, encoding="utf-8") as f:
            return set(json.load(f))
    except FileNotFoundError:
        return set()
    except (json.JSONDecodeError, TypeError):
        log("WARNING: seen.json was unreadable, starting with an empty list")
        return set()


def save_seen(seen):
    os.makedirs(os.path.dirname(SEEN_FILE), exist_ok=True)
    tmp = SEEN_FILE + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(sorted(seen), f)
    os.replace(tmp, SEEN_FILE)  # atomic: never leaves a half-written file


# ---------------------------------------------------------------------------
# Helpers
```

```
# ---------------------------------------------------------------------------
def strip_html(text):
    text = re.sub(r"(?i)<br\s*/?>|</p>|</li>", "\n", text or "")
    text = re.sub(r"<[^>]+>", " ", text)
    text = html.unescape(text)
    text = re.sub(r"[ \t\r\f\v]+", " ", text)
    text = re.sub(r"\n\s*\n+", "\n", text)
    return text.strip()


def norm(text):
    return re.sub(r"[^a-z0-9]+", " ", (text or "").lower()).strip()


def contains_word(haystack, word):
    """Whole-word, case-insensitive match ('Sr' must not match 'Srinagar')."""
    return re.search(r"(?<![a-z0-9])" + re.escape(word.lower()) + r"(?![a-z0-9])",
                     haystack.lower()) is not None


def parse_reed_date(value):
    try:
        return dt.datetime.strptime(value, "%d/%m/%Y").date()
    except (TypeError, ValueError):
        return None


# ---------------------------------------------------------------------------
# Job sources
# ---------------------------------------------------------------------------
def fetch_reed(cfg):
    if not is_set(REED_API_KEY):
        log("Reed: no REED_API_KEY in .env, skipping Reed")
        return []
    jobs = []
    oldest = dt.date.today() - dt.timedelta(days=int(cfg["max_days_old"]))
    for keyword in cfg["search_keywords"]:
        params = {"keywords": keyword, "fullTime": "true", "resultsToTake": 100}
        if cfg["location"]:
            params["locationName"] = cfg["location"]
            params["distanceFromLocation"] = cfg["distance_miles"]
        try:
            r = requests.get(f"{REED_BASE}/search", params=params,
                             auth=(REED_API_KEY, ""), timeout=HTTP_TIMEOUT)
            r.raise_for_status()
            results = r.json().get("results", [])
        except Exception as exc:  # one bad keyword must not stop the run
            log(f"Reed: '{keyword}' failed: {exc}")
            continue
        for item in results:
            posted = parse_reed_date(item.get("date"))
            if posted and posted < oldest:
                continue
            jobs.append({
                "id": f"reed-{item.get('jobId')}",
                "source": "Reed",
                "source_job_id": item.get("jobId"),
                "title": item.get("jobTitle") or "",
                "company": item.get("employerName") or "",
                "location": item.get("locationName") or "",
                "url": item.get("jobUrl") or "",
                "description": strip_html(item.get("jobDescription") or ""),
            })
        log(f"Reed: '{keyword}' -> {len(results)} results")
    return jobs


def fetch_adzuna(cfg):
    if not (is_set(ADZUNA_APP_ID) and is_set(ADZUNA_APP_KEY)):
```

```
        log("Adzuna: ADZUNA_APP_ID or ADZUNA_APP_KEY missing in .env, skipping Adzuna")
        return []
    jobs = []
    for keyword in cfg["search_keywords"]:
        params = {
            "app_id": ADZUNA_APP_ID,
            "app_key": ADZUNA_APP_KEY,
            "what_phrase": keyword,
            "full_time": 1,
            "max_days_old": int(cfg["max_days_old"]),
            "results_per_page": 50,
            "content-type": "application/json",
        }
        if cfg["location"]:
            params["where"] = cfg["location"]
            params["distance"] = int(round(float(cfg["distance_miles"]) * 1.609))  # km
        try:
            r = requests.get(f"{ADZUNA_BASE}/jobs/gb/search/1", params=params,
                             timeout=HTTP_TIMEOUT)
            r.raise_for_status()
            results = r.json().get("results", [])
        except Exception as exc:
            log(f"Adzuna: '{keyword}' failed: {exc}")
            continue
        for item in results:
            jobs.append({
                "id": f"adzuna-{item.get('id')}",
                "source": "Adzuna",
                "source_job_id": item.get("id"),
                "title": strip_html(item.get("title") or ""),
                "company": (item.get("company") or {}).get("display_name", ""),
                "location": (item.get("location") or {}).get("display_name", ""),
                "url": item.get("redirect_url") or "",
                "description": strip_html(item.get("description") or ""),
            })
        log(f"Adzuna: '{keyword}' -> {len(results)} results")
    return jobs


def fetch_reed_details(job):
    """Reed search results only contain a short snippet; fetch the full advert."""
    try:
        r = requests.get(f"{REED_BASE}/jobs/{job['source_job_id']}",
                         auth=(REED_API_KEY, ""), timeout=HTTP_TIMEOUT)
        r.raise_for_status()
        full = strip_html(r.json().get("jobDescription") or "")
        if full:
            job["description"] = full
    except Exception as exc:
        log(f"Reed details failed for {job['id']}: {exc} (using snippet)")
    return job


# ---------------------------------------------------------------------------
# Filtering
# ---------------------------------------------------------------------------
def dedupe(jobs):
    out, ids, pairs = [], set(), set()
    for job in jobs:
        pair = (norm(job["title"]), norm(job["company"]))
        if job["id"] in ids or pair in pairs:
            continue
        ids.add(job["id"])
        pairs.add(pair)
        out.append(job)
    return out


def rule_check(job, cfg):
```

```
    """Returns None if the job passes, otherwise a short reason string."""
    title = job["title"]
    wanted = cfg["title_must_contain_one_of"]
    if wanted and not any(w.lower() in title.lower() for w in wanted):
        return "title not relevant"
    for word in cfg["avoid_title_keywords"]:
        if contains_word(title, word):
            return f"seniority word '{word}'"
    desc = job["description"].lower()
    for phrase in cfg["skip_if_description_contains"]:
        if phrase.lower() in desc:
            return f"skip phrase '{phrase}'"
    return None


# ---------------------------------------------------------------------------
# AI scoring (Ollama)
# ---------------------------------------------------------------------------
SYSTEM_INSTRUCTIONS = """You are an experienced UK recruiter screening job adverts for ONE
candidate.
The candidate's complete, truthful profile is below. Judge only from facts in the profile.
Never assume skills, experience or qualifications that the profile does not state.

Score how well the candidate fits the advert from 0 to 100:
  90-100 excellent fit, apply today
  70-89  good fit, worth applying
  50-69  partial fit, important gaps
  0-49   poor fit

Reply with JSON only, exactly in this shape:
{"score": <integer 0-100>,
 "level_fit": "<too junior | good | too senior>",
 "reasons": ["<short reason>", "<short reason>"],
 "gaps": ["<short gap>"],
 "sponsorship_red_flag": <true if the advert says it cannot sponsor visas or requires
                          citizenship/security clearance the profile does not have, else false>}

=== CANDIDATE PROFILE ===
"""


def score_job(job, master):
    # The profile goes FIRST and never changes between adverts, so Ollama can
    # reuse it (prompt-prefix caching). Only the advert part changes.
    system_prompt = SYSTEM_INSTRUCTIONS + master
    advert = (
        f"JOB TITLE: {job['title']}\n"
        f"COMPANY: {job['company']}\n"
        f"LOCATION: {job['location']}\n"
        f"DESCRIPTION:\n{job['description'][:DESCRIPTION_LIMIT]}"
    )
    payload = {
        "model": SCORING_MODEL,
        "stream": False,
        "format": "json",
        "messages": [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": advert},
        ],
        "options": {"num_ctx": AI_NUM_CTX, "temperature": 0.1,
                    "num_predict": AI_NUM_PREDICT},
    }
    r = requests.post(f"{OLLAMA_URL}/api/chat", json=payload, timeout=AI_TIMEOUT_SECONDS)
    r.raise_for_status()
    content = r.json().get("message", {}).get("content", "")
    match = re.search(r"\{.*\}", content, re.S)
    if not match:
        raise ValueError(f"AI did not return JSON: {content[:120]!r}")
    result = json.loads(match.group(0))
```

```
    score = int(float(result.get("score", 0)))
    result["score"] = max(0, min(100, score))
    result["reasons"] = [str(x) for x in (result.get("reasons") or [])][:5]
    result["gaps"] = [str(x) for x in (result.get("gaps") or [])][:5]
    flag = result.get("sponsorship_red_flag", False)
    result["sponsorship_red_flag"] = flag is True or str(flag).lower() == "true"
    result["level_fit"] = str(result.get("level_fit", ""))
    return result


# ---------------------------------------------------------------------------
# Outputs: NocoDB tracker and Telegram
# ---------------------------------------------------------------------------
def save_to_nocodb(job, result):
    if not (is_set(NOCODB_TOKEN) and is_set(NOCODB_TABLE_ID)):
        log("NocoDB: token or table ID missing in .env, not saving")
        return False
    notes = (
        f"Level fit: {result['level_fit']}\n"
        f"Reasons: {'; '.join(result['reasons'])}\n"
        f"Gaps: {'; '.join(result['gaps'])}\n"
        f"Location: {job['location']}"
    )
    record = {
        "Company": job["company"],
        "Role": job["title"],
        "Link": job["url"],
        "Source": job["source"],
        "Match Score": result["score"],
        "Status": "Found",
        "Notes": notes,
        "Job ID": job["id"],
    }
    try:
        r = requests.post(
            f"{NOCODB_URL}/api/v2/tables/{NOCODB_TABLE_ID}/records",
            headers={"xc-token": NOCODB_TOKEN}, json=record, timeout=HTTP_TIMEOUT)
        r.raise_for_status()
        return True
    except Exception as exc:
        log(f"NocoDB: could not save {job['id']}: {exc}")
        return False


def send_telegram(text):
    if not (is_set(TELEGRAM_TOKEN) and is_set(TELEGRAM_CHAT_ID)):
        log("Telegram: token or chat ID missing in .env, message not sent")
        return False
    ok = True
    chunks = [text[i:i + 3900] for i in range(0, len(text), 3900)] or [""]
    for chunk in chunks:
        try:
            r = requests.post(
                f"{TELEGRAM_BASE}/bot{TELEGRAM_TOKEN}/sendMessage",
                json={"chat_id": TELEGRAM_CHAT_ID, "text": chunk,
                      "parse_mode": "HTML", "disable_web_page_preview": True},
                timeout=HTTP_TIMEOUT)
            r.raise_for_status()
        except Exception as exc:
            # never print the token: the URL contains it
            log(f"Telegram: send failed ({type(exc).__name__})")
            ok = False
    return ok


def telegram_summary(matches, stats, cfg):
    e = html.escape
    if not matches:
        return (f"<b>Job Finder</b>: run finished.\n"
```

```
                f"Fetched {stats['fetched']}, passed filters {stats['passed']}, "
                f"scored {stats['scored']}.\n"
                f"No new jobs above {cfg['min_match_score']} this time.")
    top = sorted(matches, key=lambda m: m[1]["score"], reverse=True)[:cfg["telegram_top_n"]]
    lines = [f"<b>Job Finder</b>: {len(matches)} new match(es) "
             f"(score {cfg['min_match_score']}+)",
             f"Fetched {stats['fetched']} | passed filters {stats['passed']} | "
             f"scored {stats['scored']}", ""]
    for job, res in top:
        reason = res["reasons"][0] if res["reasons"] else ""
        lines.append(f"<b>{res['score']}</b> - {e(job['title'])} @ {e(job['company'])}")
        if job["location"]:
            lines.append(f"   {e(job['location'])} ({e(job['source'])})")
        if reason:
            lines.append(f"   {e(reason)}")
        if job["url"]:
            lines.append(f"   <a href=\"{e(job['url'], quote=True)}\">Open advert</a>")
        lines.append("")
    lines.append("All matches are saved in your NocoDB tracker with Status = Found.")
    return "\n".join(lines)


# ---------------------------------------------------------------------------
# One full run
# ---------------------------------------------------------------------------
def run_once():
    cfg = load_config()
    master = load_master()
    seen = load_seen()
    log(f"Run started (model {SCORING_MODEL}, threshold {cfg['min_match_score']})")

    jobs = dedupe(fetch_reed(cfg) + fetch_adzuna(cfg))
    fetched = len(jobs)
    jobs = [j for j in jobs if j["id"] not in seen]
    log(f"Fetched {fetched} unique jobs, {len(jobs)} not seen before")

    candidates = []
    for job in jobs:
        reason = rule_check(job, cfg)
        if reason:
            seen.add(job["id"])  # rejected by rules: never look at it again
        else:
            candidates.append(job)
    save_seen(seen)
    log(f"{len(candidates)} jobs passed the filters")

    limit = int(cfg["max_jobs_to_score_per_run"])
    if len(candidates) > limit:
        log(f"Scoring the first {limit}; the other {len(candidates) - limit} "
            f"stay unseen and carry over to the next run")
    to_score = candidates[:limit]

    matches, scored = [], 0
    for n, job in enumerate(to_score, 1):
        if job["source"] == "Reed":
            fetch_reed_details(job)
            # the full advert may contain a skip phrase the snippet did not
            reason = rule_check(job, cfg)
            if reason:
                log(f"[{n}/{len(to_score)}] skipped after full text: {reason}")
                seen.add(job["id"])
                save_seen(seen)
                continue
        started = time.time()
        try:
            result = score_job(job, master)
        except Exception as exc:
            # NOT marked as seen, so it will be tried again next run
            log(f"[{n}/{len(to_score)}] scoring failed for {job['title']!r}: {exc}")
```

```
            continue
        scored += 1
        seen.add(job["id"])  # mark seen only AFTER a successful score
        save_seen(seen)
        took = time.time() - started
        log(f"[{n}/{len(to_score)}] {result['score']:>3} | {job['title']} @ "
            f"{job['company']} ({took:.0f}s)")
        if result["sponsorship_red_flag"]:
            log("      dropped: sponsorship/clearance red flag")
            continue
        if result["score"] >= int(cfg["min_match_score"]):
            save_to_nocodb(job, result)
            matches.append((job, result))

    stats = {"fetched": fetched, "passed": len(candidates), "scored": scored}
    send_telegram(telegram_summary(matches, stats, cfg))
    log(f"Run finished: {scored} scored, {len(matches)} saved as matches")
    return matches


# ---------------------------------------------------------------------------
# Scheduler
# ---------------------------------------------------------------------------
def scheduler():
    log("Scheduler started. Checking the clock every 20 seconds.")
    done = set()  # (date, "HH:MM") pairs already run
    while True:
        try:
            cfg = load_config()
            now = dt.datetime.now()
            slot = now.strftime("%H:%M")
            key = (now.date().isoformat(), slot)
            if slot in cfg["run_times"] and key not in done:
                done.add(key)
                run_once()
        except Exception as exc:  # keep the scheduler alive no matter what
            log(f"ERROR during run: {exc}")
        time.sleep(20)


if __name__ == "__main__":
    if "--once" in sys.argv:
        run_once()
    else:
        scheduler()
```

A.13 Helper: /opt/jobagent/tools/get_chat_id.py

Package path: `jobagent/tools/get_chat_id.py` · 56 lines

JOBAGENT/TOOLS/GET_CHAT_ID.PY

```
#!/usr/bin/env python3
"""
Helper: find your Telegram chat ID.

Before running this:
  1. Put your bot token in /opt/stacks/jobagent/.env as TELEGRAM_TOKEN=...
  2. Open Telegram, find YOUR bot by its username, press Start and send it "hello".

Run (on the server, inside the agent container):
  docker exec jobagent python /data/tools/get_chat_id.py

It prints the chat ID of everyone who recently messaged the bot. Copy YOUR
number into .env as TELEGRAM_CHAT_ID=... and restart the agent.
This script only reads; it does not send anything.
"""
import os
```

```
import sys

import requests

TOKEN = os.environ.get("TELEGRAM_TOKEN", "").strip()
BASE = os.environ.get("TELEGRAM_API_BASE", "https://api.telegram.org").rstrip("/")

if not TOKEN or "YOUR_" in TOKEN:
    sys.exit("TELEGRAM_TOKEN is not set in .env yet. Add it, then run: "
             "cd /opt/stacks/jobagent && docker compose up -d --force-recreate")

try:
    r = requests.get(f"{BASE}/bot{TOKEN}/getUpdates", timeout=30)
except requests.RequestException as exc:
    sys.exit(f"Could not reach Telegram ({type(exc).__name__}). Check the internet connection.")

if r.status_code == 401:
    sys.exit("Telegram says the token is wrong (401 Unauthorized). Copy it again from @BotFather.")
if r.status_code == 409:
    sys.exit("Telegram says another program is reading this bot's messages (409 Conflict). "
             "Stop that program (or remove its webhook) and try again.")
r.raise_for_status()

found = {}
for update in r.json().get("result", []):
    msg = update.get("message") or update.get("edited_message") or {}
    chat = msg.get("chat") or {}
    if "id" in chat:
        who = chat.get("username") or chat.get("first_name") or chat.get("title") or "?"
        found[chat["id"]] = (who, chat.get("type", "?"))

if not found:
    print("No messages found. Open your bot in Telegram, press Start, send 'hello', "
          "wait 5 seconds and run this again.")
else:
    print("Chats that recently messaged your bot:")
    for chat_id, (who, kind) in found.items():
        print(f"  chat ID: {chat_id:<16} name: {who:<20} type: {kind}")
    print("\nPut YOUR chat ID (type 'private') into .env as TELEGRAM_CHAT_ID=...")
```

A.14 Helper: /opt/jobagent/tools/check_setup.py

Package path: `jobagent/tools/check_setup.py` · 138 lines

JOBAGENT/TOOLS/CHECK_SETUP.PY

```
#!/usr/bin/env python3
"""
Helper: test every connection the Job Finder needs, one by one.

Run (on the server):
  docker exec jobagent python /data/tools/check_setup.py

Each line ends in OK, SKIP or FAIL with a hint. Add --send to also send a
test message to your Telegram. Secrets are never printed.
"""
import json
import os
import sys

import requests

E = os.environ.get
DATA = E("DATA_DIR", "/data")
REED_BASE = E("REED_BASE_URL", "https://www.reed.co.uk/api/1.0").rstrip("/")
ADZUNA_BASE = E("ADZUNA_BASE_URL", "https://api.adzuna.com/v1/api").rstrip("/")
TG_BASE = E("TELEGRAM_API_BASE", "https://api.telegram.org").rstrip("/")
failures = 0
```

```
def is_set(name):
    v = (E(name) or "").strip()
    return bool(v) and "YOUR_" not in v


def report(name, status, hint=""):
    global failures
    if status == "FAIL":
        failures += 1
    print(f"  {name:<22} {status:<5} {hint}")


def check(name, func):
    try:
        status, hint = func()
    except requests.RequestException as exc:
        status, hint = "FAIL", f"cannot connect ({type(exc).__name__})"
    except Exception as exc:
        status, hint = "FAIL", str(exc)[:120]
    report(name, status, hint)


def files():
    for rel in ("app/config.json", "cv/master.md"):
        if not os.path.isfile(os.path.join(DATA, rel)):
            return "FAIL", f"missing {rel} in /opt/jobagent"
    with open(os.path.join(DATA, "app/config.json"), encoding="utf-8") as f:
        cfg = json.load(f)
    with open(os.path.join(DATA, "cv/master.md"), encoding="utf-8") as f:
        words = len(f.read().split())
    hint = f"{len(cfg.get('search_keywords', []))} keywords, master.md {words} words"
    return "OK", hint


def reed():
    if not is_set("REED_API_KEY"):
        return "FAIL", "REED_API_KEY not set in .env"
    r = requests.get(f"{REED_BASE}/search", params={"keywords": "analyst", "resultsToTake": 1},
                     auth=(E("REED_API_KEY").strip(), ""), timeout=30)
    if r.status_code == 401:
        return "FAIL", "key rejected (401): copy it again from the Reed email"
    r.raise_for_status()
    return "OK", f"{r.json().get('totalResults', '?')} jobs available"


def adzuna():
    if not (is_set("ADZUNA_APP_ID") and is_set("ADZUNA_APP_KEY")):
        return "SKIP", "not configured (optional)"
    r = requests.get(f"{ADZUNA_BASE}/jobs/gb/search/1",
                     params={"app_id": E("ADZUNA_APP_ID").strip(),
                             "app_key": E("ADZUNA_APP_KEY").strip(),
                             "what": "analyst", "results_per_page": 1}, timeout=30)
    if r.status_code in (401, 403):
        return "FAIL", "rejected: APP_ID must be the short 8-character id"
    r.raise_for_status()
    return "OK", f"{r.json().get('count', '?')} jobs available"


def ollama():
    url = E("OLLAMA_URL", "http://ollama:11434").rstrip("/")
    model = E("SCORING_MODEL", "qwen2.5:3b")
    r = requests.get(f"{url}/api/tags", timeout=15)
    r.raise_for_status()
    names = [m.get("name", "") for m in r.json().get("models", [])]
    if model not in names and f"{model}:latest" not in names:
        return "FAIL", f"model {model} not downloaded: docker exec ollama ollama pull {model}"
    return "OK", f"{model} is installed"
```

```
def nocodb():
    if not (is_set("NOCODB_TOKEN") and is_set("NOCODB_TABLE_ID")):
        return "FAIL", "NOCODB_TOKEN or NOCODB_TABLE_ID not set in .env"
    url = E("NOCODB_URL", "http://nocodb:8080").rstrip("/")
    r = requests.get(f"{url}/api/v2/tables/{E('NOCODB_TABLE_ID').strip()}/records",
                     headers={"xc-token": E("NOCODB_TOKEN").strip()},
                     params={"limit": 1}, timeout=15)
    if r.status_code in (401, 403):
        return "FAIL", "token rejected: create a new API token in NocoDB"
    if r.status_code == 404:
        return "FAIL", "table not found: check NOCODB_TABLE_ID"
    r.raise_for_status()
    return "OK", "tracker table reachable"


def telegram():
    if not (is_set("TELEGRAM_TOKEN") and is_set("TELEGRAM_CHAT_ID")):
        return "FAIL", "TELEGRAM_TOKEN or TELEGRAM_CHAT_ID not set in .env"
    base = f"{TG_BASE}/bot{E('TELEGRAM_TOKEN').strip()}"
    r = requests.get(f"{base}/getMe", timeout=30)
    if r.status_code == 401:
        return "FAIL", "token rejected: copy it again from @BotFather"
    r.raise_for_status()
    bot = r.json().get("result", {}).get("username", "?")
    if "--send" in sys.argv:
        s = requests.post(f"{base}/sendMessage",
                          json={"chat_id": E("TELEGRAM_CHAT_ID").strip(),
                                "text": "Job agent online - test message"}, timeout=30)
        if s.status_code == 400:
            return "FAIL", "bot works but chat ID is wrong, or you never pressed Start"
        s.raise_for_status()
        return "OK", f"bot @{bot}; test message sent"
    return "OK", f"bot @{bot} (add --send to send a test message)"


print("AI Job Agent - setup check")
check("Files", files)
check("Reed API", reed)
check("Adzuna API", adzuna)
check("Ollama + model", ollama)
check("NocoDB tracker", nocodb)
check("Telegram", telegram)
print("All required checks passed." if failures == 0 else f"{failures} check(s) failed - see hints
above.")
sys.exit(1 if failures else 0)
```

APPENDIX B

Command cheat sheet

Proxmox host

Command	What it does
`pct list`	List containers and their state
`pct enter 100` / `exit`	Open / leave the container's shell
`pct start 100` / `pct shutdown 100`	Start / cleanly stop the container
`pct push 100 SRC DEST` / `pct pull 100 SRC DEST`	Copy a file into / out of the container
`pveam update`, `pveam available --section system`	Refresh / list container templates
`vzdump 100 --mode snapshot --compress zstd --storage local`	Back up the container
`apt update && apt full-upgrade -y`	Update Proxmox
`hostname -I`, `ip addr`, `ip route`	Show addresses and gateway
`ifreload -a`	Apply `/etc/network/interfaces`
`systemctl restart pveproxy`	Restart the web interface
`iptables -t nat -L -n -v`	Show NAT and port-forward rules (Option B)
`cat /proc/cmdline`	Show kernel options (check autosuspend fix)
`smartctl -a [-d sat] /dev/sdX`	Disk health report
`reboot`, `shutdown now`, `shutdown -r +10`, `shutdown -c`	Restart, power off, delayed restart, cancel

Container: Docker and the Job Finder

Command	What it does
`docker ps` / `docker ps -a`	Running apps / all apps including stopped
`docker logs -f jobagent`	Follow the scheduler's log
`docker exec jobagent python /data/app/finder.py --once`	Run a search now (foreground)
`docker exec -d jobagent sh -c "python /data/app/finder.py --once >> /data/state/run.log 2>&1"`	Run a search now (background)
`tail -f /opt/jobagent/state/run.log`	Follow a background run

Command	What it does
`docker exec jobagent python /data/tools/check_setup.py [--send]`	Test all connections
`docker exec jobagent python /data/tools/get_chat_id.py`	Find your Telegram chat ID
`docker restart jobagent`	Apply config.json / master.md / finder.py changes
`cd /opt/stacks/jobagent && docker compose up -d --force-recreate`	Apply .env changes
`cd /opt/stacks/STACK && docker compose pull && docker compose up -d`	Update a stack's apps
`docker exec ollama ollama list` / `pull NAME` / `rm NAME`	Manage AI models
`docker logs ollama --tail 30 2>&1 | grep "task.n_tokens" | tail -1`	Prompt size of the last AI request
`docker stats --no-stream`	Memory/CPU per app
`docker system df`, `docker image prune -f`	Docker disk use / clean up
`python3 -m json.tool /opt/jobagent/app/config.json`	Check config.json is valid
`rm /opt/jobagent/state/seen.json`	Forget all processed adverts
`tailscale ip -4`, `tailscale status`	Tailscale address / connected devices

APPENDIX C

Services, ports and file locations

Services

Name	Image	Stack folder	Port	Data	Needed
Proxmox web UI	(host)	-	8006 (https)	-	Yes
jobagent	built from Dockerfile	`/opt/stacks/jobagent`	none	`/opt/jobagent`	Yes
ollama	`ollama/ollama`	`/opt/stacks/ai`	11434 (127.0.0.1 only)	volume `ai_ollama`	Yes
nocodb	`nocodb/nocodb`	`/opt/stacks/jobs`	8081	volume `jobs_nocodb`	Yes
open-webui	`ghcr.io/open-webui/open-webui:main`	`/opt/stacks/ai`	3000	volume `ai_openwebui`	Optional
dockge	`louislam/dockge:1`	`/opt/dockge`	5001	`/opt/dockge/data`	Optional
n8n	`docker.n8n.io/n8nio/n8n`	`/opt/stacks/jobs` (profile extras)	5678	volume `jobs_n8n`	Optional
gotenberg	`gotenberg/gotenberg:8`	`/opt/stacks/jobs` (profile extras)	internal only	-	Optional

Inside Docker, apps find each other by name on the `shared` network: `http://ollama:11434`, `http://nocodb:8080`, `http://gotenberg:3000`.

Important files

File	Purpose	Where
`/etc/network/interfaces`	Host network (vmbr0, optional vmbr1)	Host
`/etc/hosts`	Must contain the host's current IP	Host
`/etc/sysctl.d/99-forward.conf`	IP forwarding (Option B)	Host
`/etc/pve/lxc/100.conf`	Container settings incl. TUN lines	Host
`/etc/systemd/logind.conf`	Lid behaviour	Host
`/etc/default/grub` or `/etc/kernel/cmdline`	Kernel options (USB autosuspend)	Host
`/var/lib/vz/dump/`	Backups	Host

File	Purpose	Where
`/opt/stacks/jobagent/.env`	All secrets (chmod 600)	Container
`/opt/jobagent/app/config.json`	Search settings	Container
`/opt/jobagent/app/finder.py`	The program	Container
`/opt/jobagent/cv/master.md`	Your profile	Container
`/opt/jobagent/cv/answers.yaml`	Fixed answers (planned use)	Container
`/opt/jobagent/state/seen.json`, `run.log`	Memory of processed adverts; background-run log	Container

APPENDIX D

Glossary

Term	Meaning
AI model	The file of learned patterns that generates text. Measured in billions of parameters ("3b").
API / API key	A door into a service for programs / the secret code that identifies you to it.
BIOS / UEFI	The settings program that runs before the operating system.
Bridge (vmbr0, vmbr1)	A virtual network switch inside Proxmox.
Chat ID	The number identifying your Telegram chat with the bot.
CIDR (/24)	Short way to write the size of a network; /24 = 255.255.255.0.
Container (LXC)	A lightweight isolated Linux system sharing the host's kernel.
Container (Docker)	A sealed box running one application from an image.
Context window (num_ctx)	How much text, in tokens, the AI can consider at once.
CPU / core / thread	The processor / its independent workers / virtual workers (two per core on many chips).
DHCP	Automatic handing-out of IP addresses by the router.
DNAT / port forwarding	Redirecting traffic arriving on a port to another address.
DNS	The internet's phone book: turns names into IP addresses.
Docker Compose / compose.yaml	Tool and recipe file that start a group of Docker apps (a stack).
.env	A text file of settings and secrets read by a Docker app.
Gateway	The router; the way out to the internet.
Host	The Proxmox machine that runs the containers.
Image (Docker)	A ready-made package from which Docker creates containers.
IP address	A device's number on a network.
ISO	An image file of an installation disc.
JSON / YAML / Markdown	Simple text formats for data / settings / formatted documents.
Kernel	The core of an operating system.
Localhost / 127.0.0.1	"This same machine".
MAC address	A network card's hardware identifier.
NAT / MASQUERADE	Letting many devices share one address to reach the internet.
Nesting	Running containers (Docker) inside a container (LXC).
Ollama	The program that runs AI models locally.

Term	Meaning
Port	A numbered door on an IP address for one service.
Prompt	(1) The text before the cursor in a terminal. (2) The text sent to an AI.
Proxmox VE	The free virtualisation operating system used as the host.
RAM	The computer's working memory.
Root	The administrator account on Linux.
SSH / scp	Secure remote terminal / secure file copy.
SSD / HDD	Solid-state drive (fast, no moving parts) / spinning hard disk.
Stack	A group of Docker apps started from one compose file.
Tailscale / tailnet	A private encrypted network between your own devices.
Template (CT template)	A ready-made Linux system used to create containers.
Token	(1) A secret password for a program (bot token, API token). (2) A piece of a word, as AI models count text.
USB autosuspend	Linux power saving that can switch off idle USB devices.
Virtual machine (VM)	A complete simulated computer.
VT-x / AMD-V	Processor features for virtualisation.
vzdump	Proxmox's backup tool.

APPENDIX E

Final checklist

Print this page and tick each box as you go.

Hardware ready (Stage B)

- [ ] Virtualisation enabled in BIOS
- [ ] Files backed up, disk can be erased
- [ ] On charger, wired Ethernet
- [ ] Free IP address and gateway noted

Proxmox installed (Stage C)

- [ ] Web UI opens at :8006
- [ ] No-subscription repository, updated
- [ ] Lid ignored (laptops)
- [ ] USB autosuspend off, checked in /proc/cmdline

Server environment (Stage D)

- [ ] amd64 Debian 13 template
- [ ] Network option chosen (A or B)
- [ ] Container created: nesting, keyctl, start at boot
- [ ] TUN lines added (for Tailscale)
- [ ] apt update works, ping works

Dependencies (Stage E)

- [ ] Docker: hello-world works
- [ ] Tailscale connected (optional)

Project (Stage F)

- [ ] Package copied and unzipped
- [ ] install.sh completed
- [ ] docker ps shows apps Up
- [ ] .env is mode 600

Tracker (Stage F2)

- [ ] NocoDB admin account
- [ ] Base "Job Agent", table "Applications", 10 fields
- [ ] API token + table ID in .env

Telegram (Stage G)

- [ ] Bot created, groups disabled
- [ ] Token + chat ID in .env
- [ ] Test message received

AI and sources (Stage H)

- [ ] Reed key (and Adzuna, optional)
- [ ] Model downloaded
- [ ] check_setup.py: all required OK

Your data (Stage J)

- [ ] master.md replaced with your true profile
- [ ] answers.yaml filled in truthfully
- [ ] config.json keywords, filters, location set

Tested

- [ ] Manual run completed
- [ ] Telegram summary received
- [ ] Matches visible in NocoDB
- [ ] Reviewed a match, updated Status (your approval step)
- [ ] Applied manually and recorded it
- [ ] Reboot test: everything back up

Protected

- [ ] Backup job scheduled, copy kept off the server
- [ ] No router port forwards; remote access via Tailscale
- [ ] No secrets shared anywhere

APPENDIX F

Useful links and index

Official links

What	Address
Proxmox VE download	https://www.proxmox.com/en/downloads
Proxmox documentation	https://pve.proxmox.com/pve-docs/
Proxmox system requirements	https://pve.proxmox.com/wiki/System_Requirements
Rufus (Windows USB writer)	https://rufus.ie
balenaEtcher	https://etcher.balena.io
Debian	https://www.debian.org
Docker Engine install	https://docs.docker.com/engine/install/
Tailscale	https://tailscale.com
Ollama and model library	https://ollama.com
Open WebUI	https://docs.openwebui.com
NocoDB	https://nocodb.com/docs
Dockge	https://github.com/louislam/dockge
Reed Jobseeker API	https://www.reed.co.uk/developers/jobseeker
Adzuna developer API	https://developer.adzuna.com
Telegram bots (official)	https://core.telegram.org/bots

Links checked November 2026. Websites change; if a link moves, search for the project name.

Index

.env secrets file 46
Adzuna keys 55
answers.yaml 47
APIs explained 12
Approve / reject (planned) 53
Architecture diagram 12
arm64 template mistake 34
Backups 77
BIOS / UEFI settings 24
BotFather 51
Chat ID 52
Cheat sheet 108
check_setup.py 56
config.json settings 46
Container, creating 36
Copy and paste in terminals 21
Disk full 75
Docker, installing 40
Dockge 61
File locations 110
Final checklist 114
Glossary 112
Hardware profiles (4/8/16 GB) 16
Input/output error 67
install.sh 44
Laptop lid 31

Logs 60
Low-memory settings 18
Manual installation 45
master.md profile 63
Models, choosing 18
nano editor 22
NAT bridge (Option B) 35
Network options A and B 15
NocoDB table and fields 49
NocoDB token and table ID 50
Open WebUI speed fix 57
Ports 110
Proxmox installer screens 28
Proxmox update repositories 30
Reed API key 55
Responsible use 81
Restoring backups 78
Roadmap 82
Running a search by hand 59
Schedule (run_times) 59
Secrets, rotating 79
Security rules 80
SSH 23
Stopping and restarting 61
Tailscale 41
Troubleshooting 67
Truthful applications 64
Updating 76
USB autosuspend 31
USB stick, bootable 27
Virtualisation check 24
Web interface, first login 29